

Processamento de Imagens

Luiz Eduardo da Silva

2026-03-14

Índice

Prefácio	3
1 Introdução ao Processamento Digital de Imagens	5
1.1 Apresentação	5
1.2 O que é uma Imagem Digital	5
1.3 Processamento de Imagens, Análise de Imagens e Visão Computacional	6
1.4 Origem e Evolução do Processamento Digital de Imagens	6
1.5 Imagens e o Espectro Eletromagnético	8
1.6 Passos Fundamentais do Processamento Digital de Imagens	9
1.7 Organização do Livro	10
2 Amostragem e Quantização	11
2.1 Elementos da Percepção Visual	11
2.1.1 Sistema de Processamento de Imagens	11
2.1.2 O Olho Humano e a Formação da Imagem	11
2.1.3 Fóvea, Cones e Bastonetes	13
2.2 Ilusões de Ótica e Percepção	13
2.3 Imagens Digitais e Sensores	15
2.3.1 Espectro Visível	15
2.3.2 Sensores de Imagem	15
2.4 Amostragem	17
2.4.1 Conceito de Amostragem	17
2.4.2 Resolução Espacial	17
2.5 Quantização	18
2.5.1 Conceito de Quantização	18
2.5.2 Profundidade de Bits	19
2.6 Representação da Imagem Digital	19
2.7 Os formatos PBM, PGM e PPM de imagens	22
2.7.1 Para ler um arquivo PNM	23
2.7.2 Salvar um arquivo PNM	25
2.8 Considerações Finais	27
3 Melhoramento de Imagens	28
3.1 O pixel	28
3.1.1 Vizinhança dos Pixels	28

3.1.2	Medidas de Distância dos pixels	29
3.1.3	Algoritmo: Transformada de Distância em Imagem Binária	30
3.1.4	Adjacência e Conectividade dos Pixels	33
3.1.5	Região e Borda	33
3.1.6	Algoritmo: Rotulagem de Componentes Conectados	35
3.2	Conceito de Melhoramento de Imagens	37
3.3	Domínios de Processamento	37
3.4	Melhoramento no Domínio Espacial	38
3.5	Transformações de Intensidade	38
3.5.1	Transformação Linear	39
3.5.2	Transformações Logarítmicas	39
3.5.3	Transformações de Potência	39
3.6	Histograma de Imagens	40
3.7	Equalização de Histograma	40
3.8	Filtragem	42
3.8.1	Convolução	42
3.8.2	Equação da Convolução	43
3.8.3	Pseudocódigo da Convolução	43
3.8.4	Operações lineares e não lineares	44
3.8.5	Exemplos de Filtros	44
3.9	Considerações sobre Ruído	46
3.9.1	Ruído Sal e Pimenta	46
3.10	Considerações Finais	48
4	Transformada de Fourier	49
4.1	Domínio Espacial e Domínio da Frequência	49
4.2	Fundamento	49
4.2.1	Mudança de domínio como estratégia de simplificação	49
4.2.2	Histórico	51
4.2.3	A ideia fundamental de Fourier	52
4.3	Transformada de Fourier Unidimensional	53
4.3.1	Definição Matemática	53
4.3.2	Interpretação Física e Intuitiva	54
4.3.3	Transformada Inversa de Fourier Unidimensional	54
4.3.4	Transformada Discreta de Fourier para uma Variável	56
4.4	A Transformada de Fourier Bidimensional	57
4.4.1	Motivação para o Domínio da Frequência	57
4.4.2	Definição Matemática	57
4.4.3	Interpretação do Espectro de Fourier Bidimensional	58
4.4.4	Frequência Espacial e Orientação	58
4.4.5	Transformada Inversa de Fourier Bidimensional	59
4.4.6	Transformada Discreta de Fourier Bidimensional	59

4.5	Transformada Rápida de Fourier (FFT)	59
4.5.1	Desenvolvimento Matemático	60
4.5.2	Restrição	61
4.5.3	Reordenação Bit-Reversa (Bit Reversal)	62
4.5.4	Estrutura Butterfly	62
4.5.5	Centralização do Espectro	65
4.6	Filtragem no Domínio da Frequência	66
4.7	Vantagens e Limitações	70
4.8	Considerações Finais	70
5	Compressão de Imagens	71
5.1	Conceitos Básicos de Compressão	72
5.1.1	Redundância de Dados	73
5.1.2	Interpretação	74
5.2	Tipos de Redundância em Imagens	74
5.2.1	Redundância Espacial	74
5.2.2	Redundância Estatística	76
5.2.3	Redundância Psicovisual	77
5.3	Compressão sem Perda	78
5.4	Compressão com Perda	78
5.4.1	Critério de Fidelidade	78
5.5	Transformações Utilizadas na Compressão	80
5.6	Codificação Huffman	80
5.7	Codificação LZW	84
5.8	Padrões de Compressão de Imagens	87
5.8.1	Formato JPEG	88
5.8.2	Formato Base-64	96
5.9	Considerações Finais	98
6	Imagens Coloridas	99
6.1	Percepção da Cor	99
6.2	Conceitos Básicos de Cor	99
6.3	Modelo de Cor RGB	100
6.4	Modelo de Cor CMY e CMYK	100
6.5	Modelo de Cor HSV	100
6.6	Modelo de Cor HSI	101
6.7	Representação de Imagens Coloridas	101
6.8	Aplicações de Imagens Coloridas	101
6.9	Considerações Finais	101
7	Morfologia Matemática Binária	102
7.1	Imagens Binárias	102
7.2	Conceitos Fundamentais da Morfologia Matemática	102

7.3	Elemento Estruturante	103
7.4	Dilatação	103
7.5	Erosão	103
7.6	Abertura	104
7.7	Fechamento	104
7.8	Propriedades das Operações Morfológicas	104
7.9	Aplicações da Morfologia Binária	105
7.10	Considerações Finais	105
8	Morfologia Matemática em Tons de Cinza	106
8.1	Imagens em Tons de Cinza	106
8.2	Fundamentos da Morfologia em Tons de Cinza	106
8.3	Elemento Estruturante	107
8.4	Dilatação em Tons de Cinza	107
8.5	Erosão em Tons de Cinza	107
8.6	Abertura em Tons de Cinza	108
8.7	Fechamento em Tons de Cinza	108
8.8	Propriedades das Operações Morfológicas	108
8.9	Aplicações da Morfologia em Tons de Cinza	109
8.10	Considerações Finais	109
9	Segmentação de Imagens	110
9.1	Conceito de Segmentação	110
9.2	Abordagens Gerais para Segmentação	110
9.3	Segmentação Baseada em Descontinuidades	111
9.3.1	Detecção de Bordas	111
9.3.2	Operadores de Detecção	111
9.4	Segmentação Baseada em Similaridades	111
9.4.1	Limiarização	111
9.4.2	Limiarização Global e Adaptativa	112
9.5	Segmentação por Crescimento de Regiões	112
9.6	Segmentação por Divisão e Fusão	112
9.7	Segmentação Baseada em Morfologia	112
9.8	Desafios da Segmentação	112
9.9	Considerações Finais	113
10	Representação e Descrição de Imagens	114
10.1	Representação de Regiões	114
10.2	Representação por Contorno	114
10.2.1	Codificação por Cadeia	115
10.2.2	Assinaturas de Forma	115
10.3	Representação por Região	115
10.4	Descrição de Regiões	115

10.5	Descritores Geométricos	116
10.6	Momentos de Imagem	116
10.7	Descritores de Textura	116
10.8	Normalização e Invariância	117
10.9	Aplicações de Representação e Descrição	117
10.10	Considerações Finais	117
11	Reconhecimento e Interpretação de Imagens Coloridas	118
11.1	Imagens Digitais e Informação de Cor	118
11.2	Modelos de Cor	118
11.3	Reconhecimento de Padrões em Imagens Coloridas	119
11.4	Segmentação Baseada em Cor	119
11.5	Aplicações da Interpretação de Imagens Coloridas	119
11.6	Considerações Finais	119
12	Introdução ao OpenCV	120
12.1	Visão Computacional e OpenCV	120
12.2	Estrutura da Biblioteca OpenCV	120
12.3	Leitura e Exibição de Imagens	120
12.4	Representação de Imagens no OpenCV	121
12.5	Redimensionamento de Imagens	121
12.6	Filtragem e Redução de Ruído	121
12.7	Detecção de Bordas	122
12.8	Limiarização (Threshold)	122
12.9	Aplicações Práticas do OpenCV	122
12.10	Considerações Finais	122
13	Conclusão	123
	Referências	124

Prefácio

Este livro não tem a pretensão de abordar em profundidade todos os temas relacionados ao Processamento Digital de Imagens (PDI). Trata-se de uma obra introdutória, cujo objetivo principal é apresentar os conceitos fundamentais da área de forma acessível e organizada, servindo como um ponto de partida para estudantes e interessados no assunto.

Nos capítulos seguintes estão reunidos materiais que foram desenvolvidos e utilizados pelo autor ao longo de vários anos em atividades de ensino. Esses conteúdos foram selecionados com base na experiência em sala de aula e correspondem ao conjunto de conceitos considerados essenciais para uma compreensão inicial do processamento de imagens. A proposta é oferecer uma visão clara dos principais fundamentos, acompanhada de exemplos e ilustrações que facilitem a compreensão.

Para um estudo mais aprofundado do tema, recomenda-se a consulta a obras clássicas da área, que tratam o assunto com maior rigor teórico e abrangência. Entre os textos mais conhecidos está *Digital Image Processing*, de Rafael C. Gonzalez e Richard E. Woods (Gonzalez e Woods (2010), Gonzalez e Woods (2018)), amplamente utilizado em cursos de graduação e pós-graduação. Outras referências importantes incluem *Digital Image Processing* de William K. Pratt e *Fundamentals of Digital Image Processing* de Anil K. Jain (Pratt (2007), Jain (1989)), entre outros (Sonka, Hlavac, e Boyle (2014), Szeliski (2022)), que também são obras tradicionais na formação de pesquisadores e profissionais da área. Essas leituras complementam e aprofundam os conceitos apresentados neste livro.

Este é o livro gerado a partir dos slides da disciplina de **Processamento de Imagens**. A geração foi realizada usando a ferramenta **Quarto book** com o auxílio do Chat-GPT 5.2, mas todo o conteúdo foi verificado e aprovado pelo autor.

Para aprender mais sobre **Quarto book** visite: <https://quarto.org/docs/books>.

Alguns exemplos do livro serão apresentados usando p5.js. A **p5.js** é uma biblioteca JavaScript de código aberto, inspirada na linguagem *Processing*, concebida para tornar a programação visual e interativa mais acessível. Ao abstrair a complexidade do desenvolvimento web, ela permite a criação de gráficos, animações e interações de forma simples e intuitiva, favorecendo tanto iniciantes quanto usuários experientes.

Por sua facilidade de uso e execução direta no navegador, a p5.js é amplamente adotada em contextos educacionais, visualização de dados e prototipagem, constituindo uma ferramenta especialmente adequada ao ensino e à experimentação em **Processamento de Imagens**, ao integrar programação, visualização e interatividade em um mesmo ambiente.

Exemplo de código embutido usando p5js:

 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/J9qjHybzR>



Para aprender mais sobre **p5.js** visite: <https://editor.p5js.org/about>.

Para a apresentação e visualização de algumas equações e conceitos matemáticos, foi utilizado o software **GeoGebra**. Essa ferramenta permite explorar de forma interativa gráficos, funções e relações matemáticas, facilitando a compreensão de conceitos importantes apresentados ao longo do texto. O uso de recursos interativos contribui para tornar a visualização das equações mais clara e intuitiva para o leitor. Para saber mais sobre essa ferramenta e acessar seus recursos, visite o site oficial do GeoGebra: <https://www.geogebra.org>.

As figuras utilizadas neste livro foram construídas com o auxílio do **Excalidraw**, uma ferramenta de desenho vetorial simples e intuitiva, bastante adequada para a criação de diagramas e ilustrações didáticas. O Excalidraw permite produzir gráficos e esquemas com um estilo visual claro, favorecendo a comunicação de ideias e conceitos apresentados no texto. Mais informações sobre essa ferramenta podem ser encontradas em: <https://excalidraw.com>.

1 Introdução ao Processamento Digital de Imagens

1.1 Apresentação

Um curso sobre **Processamento Digital de Imagens (PDI)** integra a formação em **Ciência da Computação** e tem como objetivo estudar técnicas computacionais para aquisição, representação, transformação e interpretação de imagens digitais. Ao longo do curso, o estudante será introduzido aos fundamentos teóricos e práticos que permitem extrair informações relevantes de imagens por meio de algoritmos implementados em computadores digitais.

O processamento de imagens é uma área multidisciplinar, com aplicações que vão desde sistemas simples de melhoria visual até sistemas complexos de reconhecimento automático, análise de cenas e apoio à tomada de decisão.

1.2 O que é uma Imagem Digital

Uma imagem pode ser definida matematicamente como uma função bidimensional $f(x, y)$, em que x e y representam coordenadas espaciais em um plano, e o valor de $f(x, y)$ corresponde à **intensidade luminosa** ou **nível de cinza** naquele ponto.

Quando as coordenadas espaciais e os valores de intensidade assumem valores **discretos e finitos**, a imagem é denominada **imagem digital**. Nesse contexto, uma imagem digital é formada por um conjunto finito de elementos organizados em uma grade, chamados de **pixels** (*picture elements*), sendo cada pixel identificado por sua posição $((x, y))$ e por um valor associado de intensidade.

O **Processamento Digital de Imagens** refere-se, portanto, ao conjunto de técnicas computacionais aplicadas sobre essas imagens digitais com o objetivo de melhorar sua qualidade, extrair informações ou permitir sua interpretação automática.

1.3 Processamento de Imagens, Análise de Imagens e Visão Computacional

O PDI está intimamente relacionado a outras duas áreas: **Análise de Imagens** e **Visão Computacional**. Embora relacionadas, essas áreas possuem focos distintos:

- **Processamento de Imagens:** concentra-se na transformação da imagem, tendo como saída outra imagem (por exemplo, redução de ruído ou melhoria de contraste).
- **Análise de Imagens:** busca extrair atributos e descrições relevantes dos objetos presentes na imagem.
- **Visão Computacional:** tem como objetivo final atribuir significado às imagens, simulando aspectos da percepção visual humana.

Esses processos podem ser organizados em três níveis:

- **Nível baixo:** operações básicas, como filtragem de ruído e realce de contraste;
- **Nível médio:** segmentação e descrição de objetos;
- **Nível alto:** interpretação e reconhecimento, ou seja, “dar sentido” aos objetos detectados.

1.4 Origem e Evolução do Processamento Digital de Imagens

Os primeiros registros associados ao processamento de imagens remontam ao início do século XX. Em 1921, imagens foram transmitidas entre Londres e Nova York por meio de cabos submarinos, reduzindo drasticamente o tempo de envio de fotografias. Posteriormente, técnicas como o uso de **fitas perfuradas** permitiram melhorias manuais nas imagens recebidas.



Figura 1.1: Primeira imagem transmitida

Na década de 1960, o avanço do PDI esteve fortemente ligado à **corrida espacial**, com o uso de computadores para melhorar imagens capturadas por sondas espaciais, como as primeiras

imagens digitais da Lua. Já na década de 1970, a área ganhou grande impulso na **medicina**, com o surgimento da tomografia computadorizada, que possibilitou a reconstrução de imagens internas do corpo humano a partir de dados capturados por sensores.



Figura 1.2: Imagem obtida com fita perfurada



Figura 1.3: Transmitida com 16 tons de cinza

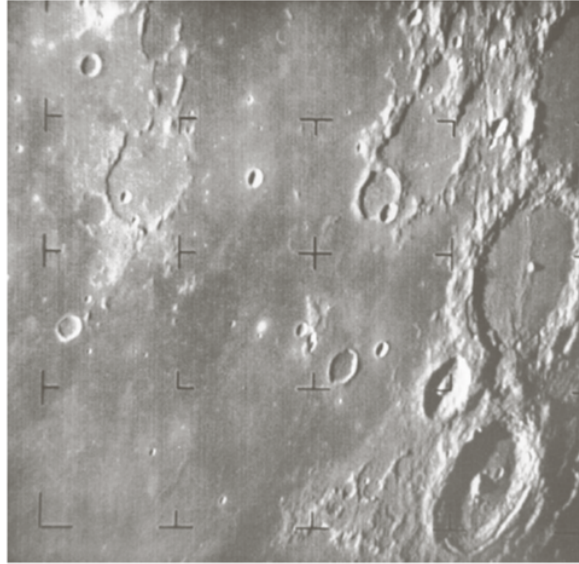


Figura 1.4: Transmitida com 16 tons de cinza

1.5 Imagens e o Espectro Eletromagnético

As imagens utilizadas em PDI não se limitam à luz visível. Elas podem ser obtidas em diferentes faixas do **espectro eletromagnético**, incluindo:

- raios gama e raios X (aplicações médicas e industriais),
- ultravioleta (análise de materiais e superfícies),
- visível (fotografia, vigilância, inspeção),
- infravermelho (sensoriamento térmico e satélites),
- micro-ondas e ondas de rádio (radar, ressonância magnética),
- além de imagens sísmicas e microscopia eletrônica.

Essa diversidade amplia significativamente o campo de aplicações do processamento digital de imagens.

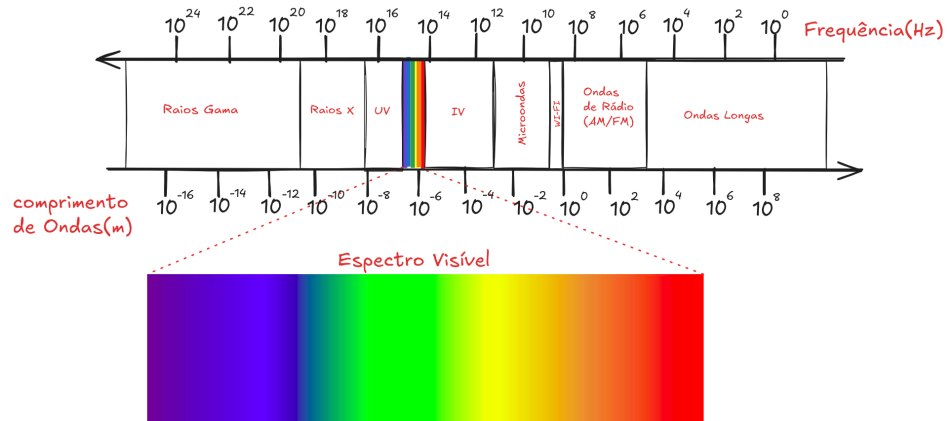


Figura 1.5: Espectro Eletromagnético (fonte: Khan Academy)

1.6 Passos Fundamentais do Processamento Digital de Imagens

De forma geral, um sistema de PDI envolve etapas fundamentais como:

1. **Aquisição da imagem** por sensores;
2. **Pré-processamento**, incluindo realce e restauração;
3. **Segmentação**, para separar objetos de interesse;
4. **Representação e descrição**, por meio de atributos;
5. **Reconhecimento e interpretação**, apoiados por uma base de conhecimento.

Essas etapas são suportadas por componentes como sensores de imagem, hardware especializado, software de processamento, sistemas de armazenamento e dispositivos de visualização.

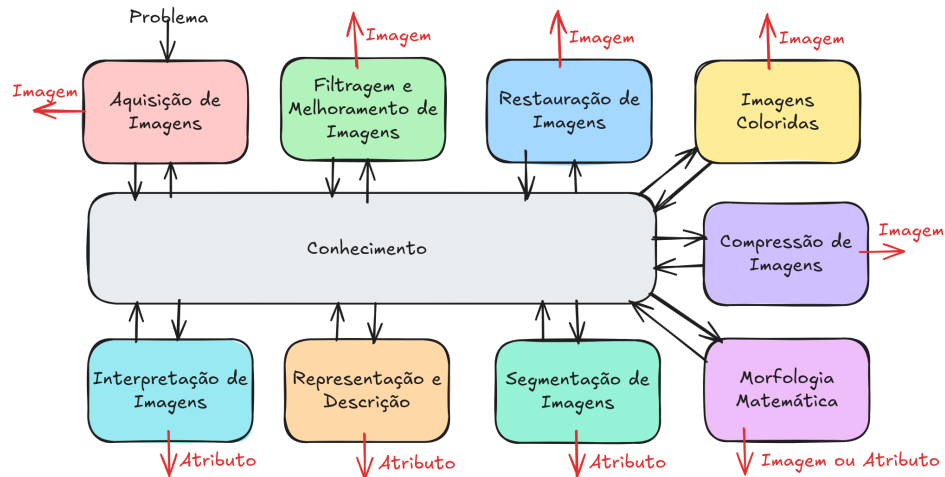


Figura 1.6: Passos Fundamentais do Processamento de Imagens

1.7 Organização do Livro

Este livro abordará os conceitos fundamentais do processamento digital de imagens, estabelecendo as bases matemáticas, computacionais e conceituais necessárias para disciplinas mais avançadas, como visão computacional e aprendizado de máquina aplicado a imagens. Ao final, espera-se que o aluno seja capaz de compreender, implementar e analisar algoritmos básicos de PDI, reconhecendo suas aplicações práticas em diferentes áreas do conhecimento.

2 Amostragem e Quantização

A representação digital de imagens é um dos pilares fundamentais do Processamento Digital de Imagens (PDI). Para que uma cena do mundo real possa ser manipulada por um computador, é necessário transformá-la de uma forma contínua, tanto no espaço quanto na intensidade luminosa, em uma representação discreta. Esse processo ocorre por meio de duas etapas essenciais: **amostragem** e **quantização**.

Neste capítulo são apresentados os fundamentos conceituais dessas etapas, relacionando-as à percepção visual humana, aos sensores de aquisição de imagens e aos modelos matemáticos utilizados para representar imagens digitais.

2.1 Elementos da Percepção Visual

2.1.1 Sistema de Processamento de Imagens

Um sistema de processamento de imagens pode ser descrito como um conjunto de etapas que envolvem **aquisição**, **processamento**, **armazenamento** e **saída** de imagens, conforme ilustrado em Figura 2.1. A aquisição é realizada por dispositivos como câmeras de vídeo e scanners, enquanto o processamento ocorre em um computador. A saída pode ser exibida em monitores, impressoras ou outros dispositivos gráficos, e o armazenamento é feito em mídias digitais.

Essa estrutura é inspirada, em muitos aspectos, no próprio sistema visual humano, que serve como referência para o desenvolvimento de modelos computacionais de visão.

2.1.2 O Olho Humano e a Formação da Imagem

O olho humano funciona como um sofisticado sistema óptico (Figura 2.2). A luz proveniente dos objetos atravessa a córnea e o cristalino, sendo focalizada sobre a retina. Na retina encontram-se os fotorreceptores, responsáveis por converter a energia luminosa em sinais elétricos que são transmitidos ao cérebro pelo nervo óptico.

A imagem formada na retina é invertida, e o cérebro realiza a interpretação e a reconstrução da cena observada. Esse processo evidencia que a percepção visual não é apenas um fenômeno físico, mas também cognitivo.

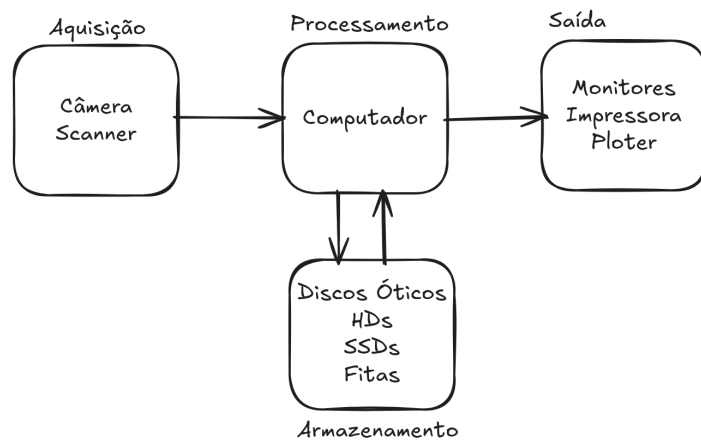


Figura 2.1: Elementos de um sistema de processamento de imagens

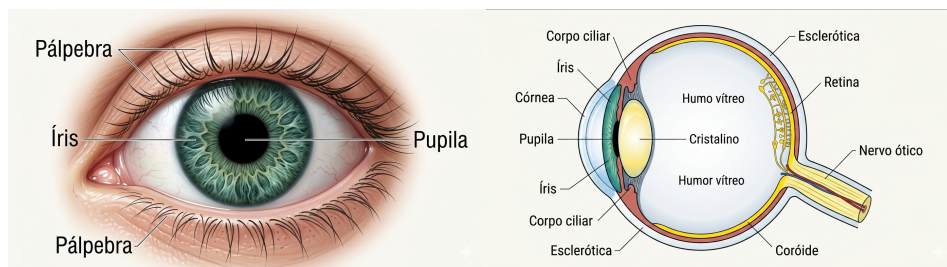


Figura 2.2: Olho humano

2.1.3 Fóvea, Cones e Bastonetes

A retina possui duas classes principais de fotorreceptores: **bastonetes** e **cones**, Figura 2.3. Os bastonetes são altamente sensíveis à luz e estão associados à percepção de brilho, enquanto os cones são responsáveis pela percepção de cores.

A região da **fóvea**, uma pequena depressão circular localizada no centro da retina, concentra uma grande densidade de cones, sendo responsável pela alta acuidade visual. Essa característica inspira o projeto de sensores de imagem, como os dispositivos CCD e CMOS, que utilizam matrizes de elementos sensíveis à luz para capturar imagens digitais.

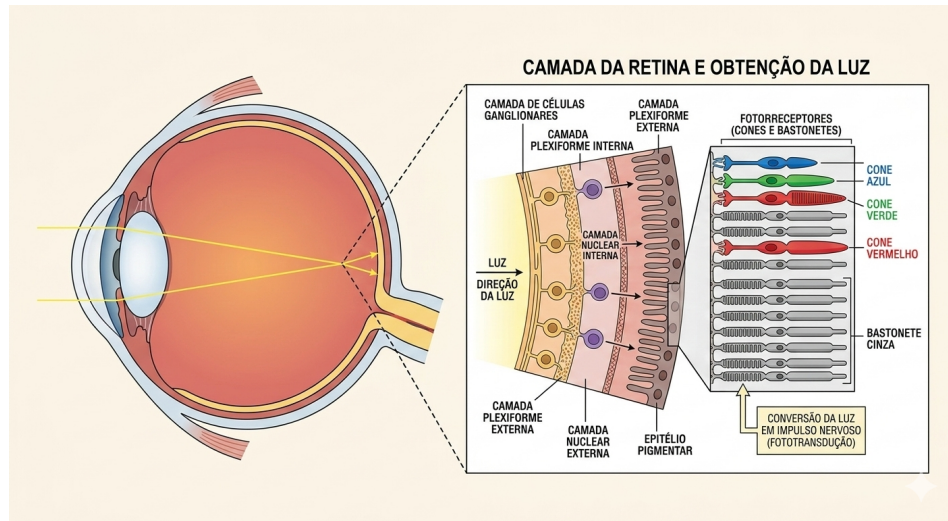


Figura 2.3: Bastonetes e Cones

2.2 Ilusões de Ótica e Percepção

O estudo das ilusões de ótica revela limitações e particularidades do sistema visual humano. Em situações onde o cérebro não consegue interpretar corretamente os estímulos visuais, ele tende a completar a informação com base em experiências prévias e padrões memorizados.

Fenômenos como o efeito Mach (Figura 2.4), a percepção de contraste e distorções associadas a linhas e formas (Figura 2.5) demonstram que a intensidade percebida nem sempre corresponde à intensidade física real. Esses efeitos são relevantes em PDI, pois técnicas de realce e segmentação exploram princípios semelhantes aos do sistema visual humano.

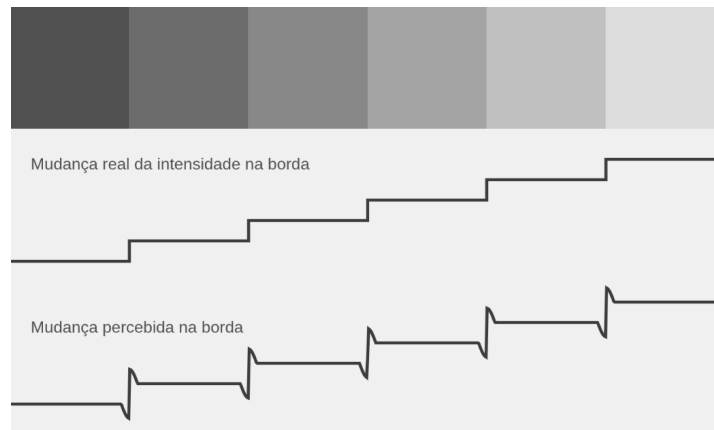


Figura 2.4: Efeito mach

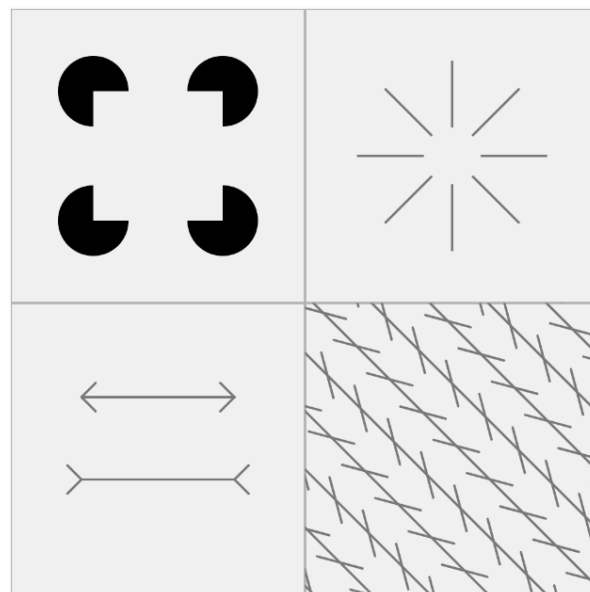


Figura 2.5: Efeito de linhas

2.3 Imagens Digitais e Sensores

2.3.1 Espectro Visível

O espectro visível (Figura 2.6) corresponde a uma pequena faixa do espectro eletromagnético, compreendida aproximadamente entre os comprimentos de onda do violeta ao vermelho. Embora estreita, essa faixa é suficiente para representar uma grande variedade de informações visuais utilizadas em aplicações cotidianas.

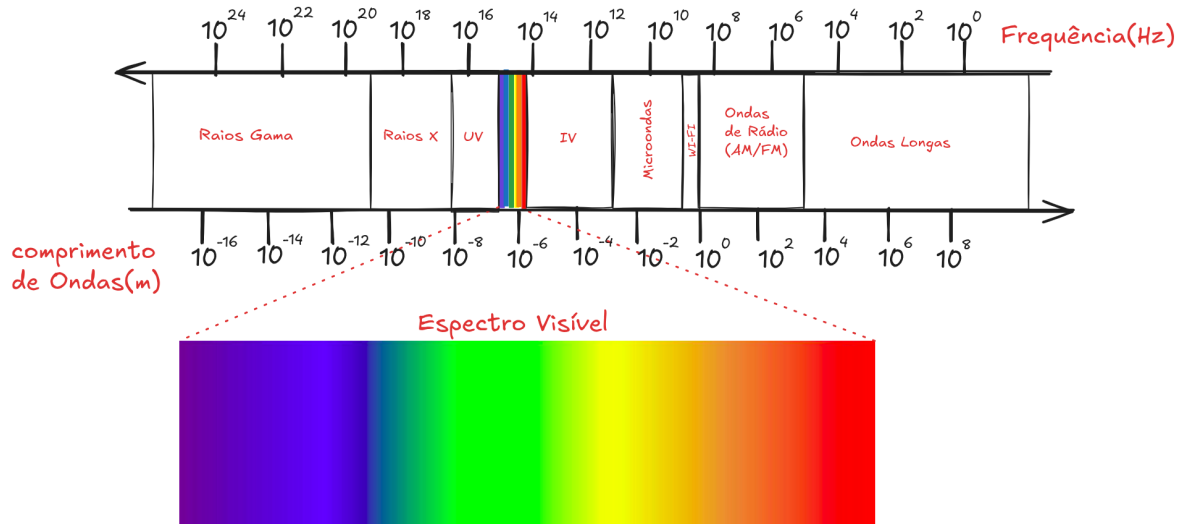


Figura 2.6: Espectro visível da luz

2.3.2 Sensores de Imagem

A aquisição de imagens digitais é realizada por sensores que convertem energia luminosa em sinais elétricos. Um sensor simples consiste em um material sensível à luz que gera uma tensão proporcional à intensidade luminosa incidente.

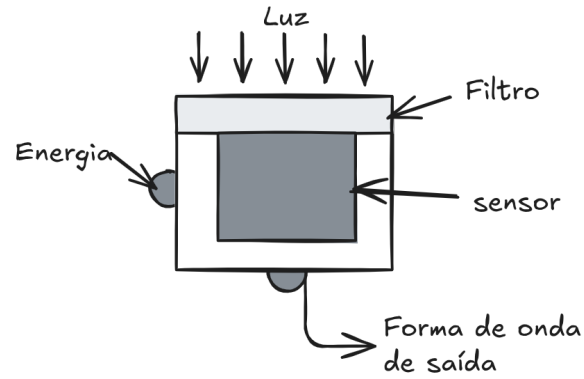


Figura 2.7: Sensores para aquisição de imagens

Os sensores podem ser classificados em:

- **Sensores lineares**, que capturam imagens linha a linha;
- **Sensores matriciais**, que utilizam uma matriz bidimensional de elementos sensíveis à luz, sendo os mais comuns em câmeras digitais modernas.

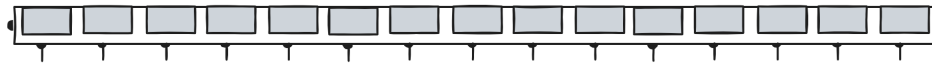


Figura 2.8: Sensor Linear

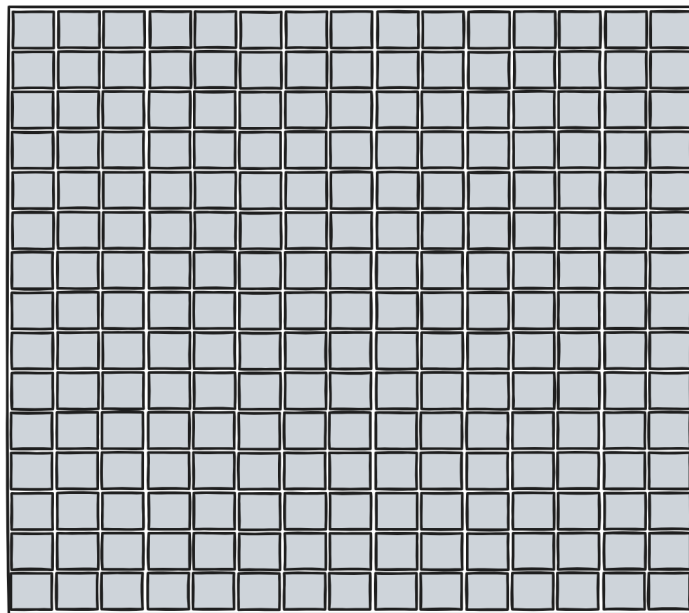


Figura 2.9: Sensor Matricial

2.4 Amostragem

2.4.1 Conceito de Amostragem

A amostragem corresponde à discretização espacial da imagem. Uma cena contínua é convertida em um conjunto finito de pontos, organizados em uma grade regular. Cada ponto da grade representa um **pixel** da imagem digital.

Matematicamente, considera-se uma função contínua de intensidade $f(x, y)$, que, após a amostragem, passa a ser definida apenas em posições discretas de x e y . A Figura 2.10, apresenta um exemplo do processo de amostragem e quantização para um sinal unidimensional em vermelho.

2.4.2 Resolução Espacial

A resolução espacial de uma imagem está relacionada ao número de amostras por unidade de comprimento, sendo frequentemente expressa em **DPI (dots per inch)**. Quanto maior a resolução, maior o número de pixels utilizados para representar a imagem e maior o nível de detalhe visual.

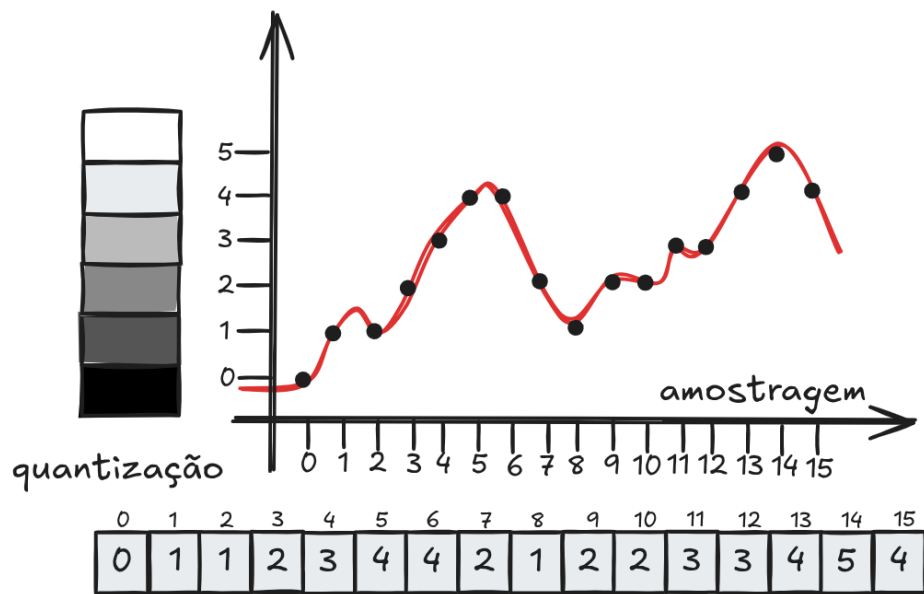


Figura 2.10: Ilustração dos processos de amostragem e quantização de um sinal contínuo.

Imagens com o mesmo tamanho físico podem apresentar resoluções diferentes, resultando em níveis distintos de nitidez. Da mesma forma, imagens com a mesma resolução podem ter tamanhos físicos diferentes.

2.5 Quantização

2.5.1 Conceito de Quantização

A quantização refere-se à discretização dos valores de intensidade da imagem. Enquanto a amostragem define *onde* a imagem é medida, a quantização define *quais valores* essas medições podem assumir.

Sejam (L) os níveis possíveis de intensidade, normalmente escolhidos como uma potência de dois:

$$L = 2^k$$

onde (k) é o número de bits utilizados para representar cada pixel.

2.5.2 Profundidade de Bits

A profundidade de bits influencia diretamente a qualidade visual da imagem. Por exemplo:

- 8 bits por pixel permitem 256 níveis de cinza;
- 4 bits por pixel permitem 16 níveis;
- 2 bits por pixel permitem apenas 4 níveis.

Reduções na quantização podem introduzir efeitos visuais indesejados, como perda de detalhes e aparecimento de bandas de intensidade.

2.6 Representação da Imagem Digital

Uma imagem digital pode ser representada como uma **matriz numérica**, em que cada elemento corresponde ao valor de intensidade de um pixel. A Figura 2.11, ilustra os valores dos pixels (1 ou 0) numa imagem binária. Para imagens em tons de cinza, essa matriz contém valores escalares, conforme ilustrado na Figura 2.12.



Figura 2.11: Ilustração dos pixels numa Imagem Binária

O exemplo interativo seguinte, apresenta uma ferramenta de zoom para melhor visualizar a representação matricial para imagens em tons de cinza. Para interação, movimente o mouse sobre a imagem à esquerda para selecionar uma região. Na imagem à direita será apresentada uma amostra ampliada (zoom), dos pixels correspondentes na região selecionada.

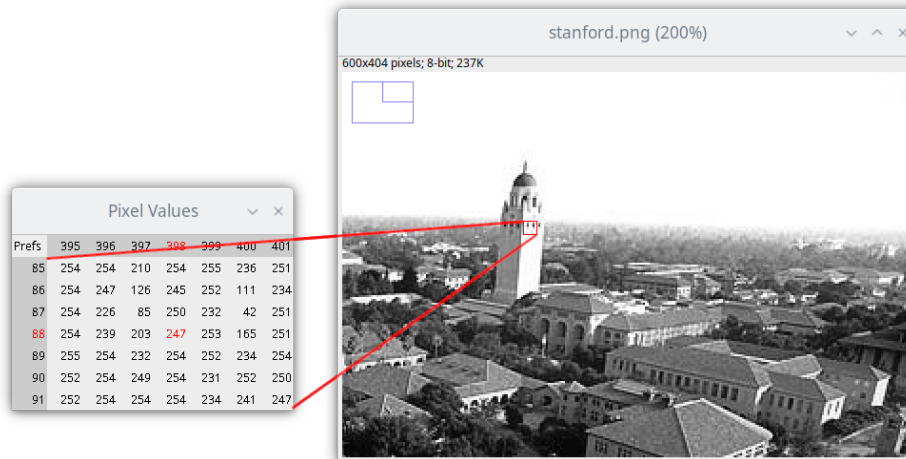


Figura 2.12: Ilustração dos pixels numa Imagem em Tons de Cinza

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/bDwv1wtRH>



Para imagens coloridas, a representação pode envolver:

- tabelas de cores;
- camadas de cores, como os canais vermelho, verde e azul (RGB). (Figura 2.13)

Além da representação matricial, a imagem também pode ser interpretada como uma superfície tridimensional, na qual as coordenadas espaciais definem a posição do pixel e a intensidade define a altura da superfície (Figura 2.14).

O exemplo interativo seguinte permite uma visualização de uma imagem como superfície:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/7vIY9trVV>

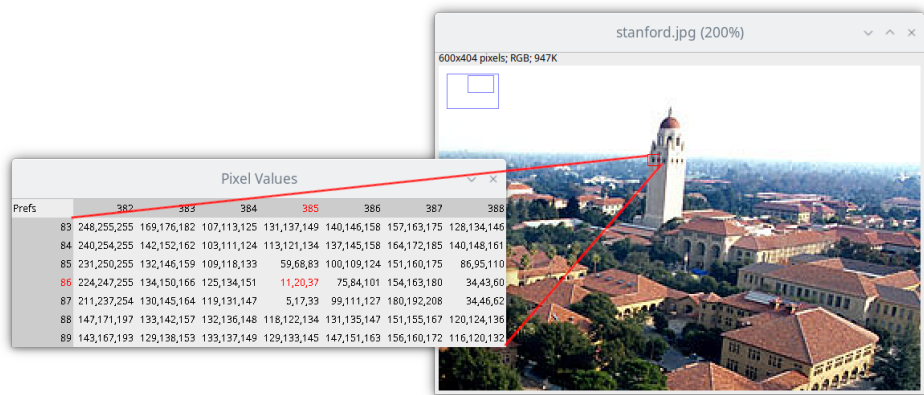


Figura 2.13: Ilustração dos pixels numa Imagem Colorida

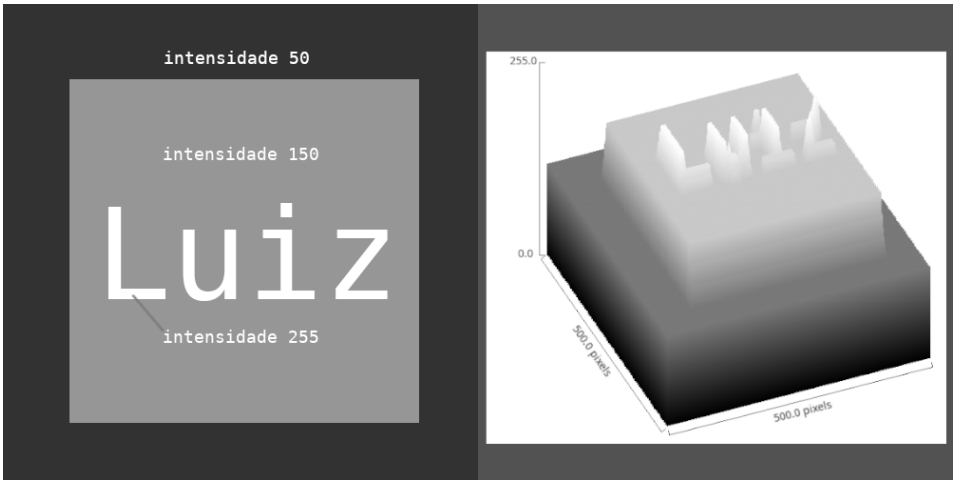


Figura 2.14: Ilustração da imagem com superfície



2.7 Os formatos PBM, PGM e PPM de imagens

Os formatos **PBM**, **PGM** e **PPM** pertencem à família **Netpbm** e são amplamente utilizados no ensino e na pesquisa em **Processamento Digital de Imagens**, principalmente por sua simplicidade e transparência na representação dos dados. Esses formatos descrevem explicitamente os valores dos pixels, sem empregar técnicas sofisticadas de compressão, o que facilita a compreensão de como uma imagem digital é armazenada e manipulada. Cada formato é voltado para um tipo específico de imagem: binária, em escala de cinza ou colorida.

O formato **PBM (Portable Bitmap)** é utilizado para representar **imagens binárias**, nas quais cada pixel assume apenas dois valores possíveis. Tipicamente, o valor 0 representa a cor branca e o valor 1 representa a cor preta. Por esse motivo, o PBM é especialmente adequado para máscaras, imagens segmentadas e resultados de processos de limiarização. Por trabalhar com apenas um bit por pixel, esse formato é conceitualmente simples e frequentemente empregado em exemplos introdutórios de binarização e análise lógica de imagens.

```
P1
5 5
0 0 1 0 0
0 1 1 1 0
1 1 1 1 1
0 1 1 1 0
0 0 1 0 0
```

O formato **PGM (Portable Graymap)** é destinado à representação de **imagens em escala de cinza**. Nesse caso, cada pixel é descrito por um único valor numérico que indica seu nível de intensidade luminosa, geralmente variando entre 0 (preto) e 255 (branco). Imagens PGM são muito utilizadas em processamento de imagens por permitirem uma análise direta dos níveis de cinza, sendo ideais para o estudo de filtros, detecção de bordas, realce de contraste e transformações pontuais. Sua estrutura simples torna explícita a relação entre valores numéricos e intensidades visuais.

```
P2
5 5
255
0 50 100 150 200
50 100 150 200 255
100 150 200 255 200
150 200 255 200 150
200 255 200 150 100
```

Já o formato **PPM (Portable Pixmap)** é empregado para **imagens coloridas**, representadas no modelo **RGB**. Cada pixel é descrito por três valores inteiros correspondentes às componentes vermelha (Red), verde (Green) e azul (Blue), normalmente também no intervalo de 0 a 255. Dessa forma, o PPM permite representar uma ampla gama de cores, sendo frequentemente usado como formato intermediário em conversões e experimentos. Apesar de gerar arquivos maiores, sua clareza estrutural facilita o entendimento da composição das imagens coloridas.

```
P3
3 2
255
255 0 0 0 255 0 0 0 255
255 255 0 0 255 255 255 0 255
```

Os três formatos admitem duas variações: uma versão **ASCII**, legível por humanos, e uma versão **binária**, mais compacta e eficiente em termos de armazenamento. As versões ASCII são particularmente úteis em contextos didáticos, pois permitem a inspeção direta dos valores dos pixels em um editor de texto. Já as versões binárias são mais adequadas para aplicações práticas, devido ao menor tamanho dos arquivos e à maior velocidade de leitura e escrita.

Em conjunto, os formatos PBM, PGM e PPM desempenham um papel fundamental no ensino do Processamento Digital de Imagens, pois tornam explícita a relação entre a representação numérica e a aparência visual das imagens. Ao trabalhar com esses formatos, o estudante pode compreender de forma concreta conceitos como pixel, nível de cinza, cor e quantização, estabelecendo uma base sólida para o estudo de técnicas mais avançadas.

2.7.1 Para ler um arquivo PNM

Abaixo o código em linguagem C para ler arquivo nesses formatos

```

/*-----
 * Read pnm ascii image
 * Params (in):
 *   name = image file name
 *   tp = image type (BW, GRAY or COLOR)
 * Returns:
 *   image structure
 *-----+*/
void erro(char *msg)
{
    puts(msg);
    exit(1);
}

image img_get(char *name, int tp)
{
    char line[200];
    int nr, nc, ml;
    image img;
    FILE *f;
    f = fopen(name, "rt");
    if (!f)
        erro("Abertura do arquivo de imagem");
    fgets(line, 200, f); // leitura do cabecaho
    if (line[0] != 'P' || line[1] != (tp + '0'))
        erro("Tipo do arquivo errado!");
    fgets(line, 200, f); // leitura dos comentario
    while (line[0] == '#')
        fgets(line, 200, f);
    sscanf(line, "%d %d", &nc, &nr); // leitura do numero de colunas e numero de linhas
    if (tp != BW)
        fscanf(f, "%d", &ml); // leitura máxima intensidade
    else
        ml = 1;
    img = malloc(sizeof(struct image));
    img->nr = nr;
    img->nc = nc;
    img->ml = ml;
    img->tp = tp;
    img->px = malloc(nr * nc * sizeof(int));
    for (int i = 0; i < nr * nc; i++)
    {

```



```

    if (tp != COLOR)
    {
        int k;
        fscanf(f, "%d", &k);
        img->px[i] = k;
    }
    else
    {
        int r, g, b;
        fscanf(f, "%d %d %d", &r, &g, &b);
        img->px[i] = (r << 16) | (g << 8) | b;
    }
}
fclose(f);
return img;
}

```

2.7.2 Salvar um arquivo PNM

Abaixo o código para gravar a imagem nestes formatos.

```

/*-----
 * Write pnm image
 * Params:
 *   img = image structure
 *   name = image file name
 *   tp = image type (BW, GRAY or COLOR)
 *-----*/
void img_put(image img, char *name, int tp)
{
    int count = 0;
    FILE *f = fopen(name, "wt");
    if (!f)
        erro("Erro na criação do arquivo de imagem");
    fprintf(f, "P%c\n", tp + '0');
    fputs("# Imagem gerada por Luiz Eduardo\n", f);
    fprintf(f, "%d %d\n", img->nc, img->nr);
    if (tp != BW)
        fprintf(f, "%d\n", img->ml);
    for (int i = 0; i < img->nr * img->nc; i++)
    {

```

```

    if (tp != COLOR)
    {
        int k = img->px[i];
        fprintf(f, "%3d ", k);
    }
    else
    {
        int r = (img->px[i] >> 16) & (0xFF);
        int g = (img->px[i] >> 8) & (0xFF);
        int b = img->px[i] & (0xFF);
        fprintf(f, "%d %d %d ", r, g, b);
    }
    count++;
    if (count > 50)
    {
        fprintf(f, "\n");
        count = 0;
    }
}
}

```

Onde a imagem (*image*) está representada na seguinte estrutura:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct image
{
    int nr, // numero de linhas
        nc, // numero de colunas
        ml, // maximo nivel de cinza
        tp; // tipo (BW, GRAY, COLOR)
    int *px; // vetor de pixels (nr * nc)
} *image;

enum
{
    BW = 1,
    GRAY,
    COLOR
};

```

Um programa para abrir uma imagem em formato PGM, calcular o negativo da imagem, salvar o resultado e chamar um visualizador de imagem para apresentar o resultado é o seguinte:

```
int main(int argc, char *argv[])
{
    image In;
    if (argc < 2)
    {
        puts("Tem que passar o nome do arquivo de imagem!");
        exit(1);
    }
    In = img_get(argv[1], GRAY);
    for (int i = 0; i < In->nr * In->nc; i++)
        In->px[i] = 255 - In->px[i];
    img_put(In, "saida.pgm", GRAY);
    system("eog saida.pgm &");
}
```

2.8 Considerações Finais

A amostragem e a quantização constituem a base da representação digital de imagens. A escolha adequada da resolução espacial e da profundidade de quantização é fundamental para garantir um equilíbrio entre qualidade visual, custo computacional e requisitos de armazenamento.

A compreensão desses conceitos é essencial para o estudo de técnicas mais avançadas de processamento de imagens, como filtragem, segmentação, compressão e reconhecimento de padrões, abordadas nos capítulos seguintes.

3 Melhoramento de Imagens

O melhoramento de imagens é uma das etapas mais importantes do Processamento Digital de Imagens (PDI). Diferentemente de outras áreas, como segmentação ou reconhecimento de padrões, o objetivo principal do melhoramento não é extrair informações quantitativas da imagem, mas **tornar a imagem mais adequada à interpretação humana ou a etapas posteriores de processamento**.

As técnicas de melhoramento atuam diretamente sobre os valores de intensidade dos pixels, buscando realçar detalhes, reduzir imperfeições ou evidenciar características de interesse, sem alterar o conteúdo essencial da cena.

3.1 O pixel

Em uma imagem digital, cada pixel pode ser identificado por suas coordenadas no plano da imagem. Seja p um pixel localizado na posição (i, j) . A partir dessa posição, é possível definir diferentes relações de vizinhança entre os pixels, que são amplamente utilizadas em operações de processamento de imagens, como filtragem, segmentação e detecção de bordas.

3.1.1 Vizinhança dos Pixels

A **vizinhança de 4 pixels**, também chamada de vizinhança ortogonal, considera apenas os pixels que estão imediatamente acima, abaixo, à esquerda e à direita do pixel (p) . Esses pixels compartilham um lado com (p) . Formalmente, o conjunto desses vizinhos é definido por

$$N_4(p) = \{(i+1, j), (i-1, j), (i, j+1), (i, j-1)\}.$$

Além desses, também podemos considerar os **vizinhos diagonais** de (p) . Esses pixels estão posicionados nas quatro diagonais ao redor do pixel central, compartilhando apenas um vértice com ele. O conjunto dos vizinhos diagonais é dado por

$$N_D(p) = \{(i+1, j+1), (i-1, j-1), (i-1, j+1), (i+1, j-1)\}.$$

Quando consideramos simultaneamente os vizinhos ortogonais e os vizinhos diagonais, obtemos a **vizinhança de 8 pixels**. Essa vizinhança inclui todos os pixels imediatamente

adjacentes ao redor de (p) , formando um bloco 3×3 com p no centro. O conjunto de vizinhos é definido pela união das duas vizinhanças anteriores:

$$N_8(p) = N_4(p) \cup N_D(p).$$

Essas diferentes definições de vizinhança são importantes porque determinam como os pixels são considerados conectados em uma imagem. Dependendo da aplicação, pode-se utilizar a vizinhança de 4 ou de 8 pixels para definir conectividade, propagação de regiões ou cálculo de operadores locais.

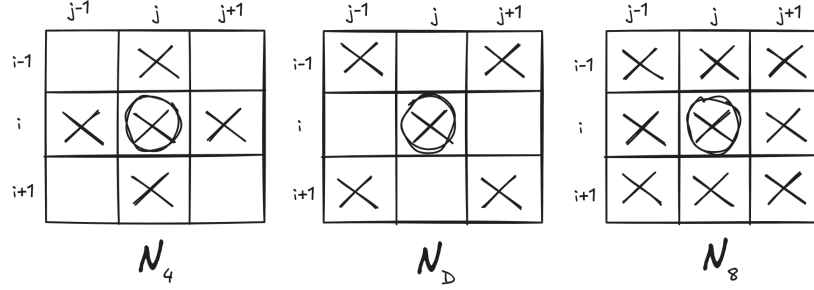


Figura 3.1: Ilustração das relações de vizinhança dos pixels

3.1.2 Medidas de Distância dos pixels

Em uma imagem digital, também é comum definir **medidas de distância entre pixels**. Essas medidas são importantes em diversas aplicações de processamento de imagens, como segmentação, crescimento de regiões, transformadas de distância e análise de conectividade. Considere dois pixels p e q , localizados nas coordenadas $p = (i_1, j_1)$ e $q = (i_2, j_2)$.

A **distância euclidiana** corresponde à distância geométrica tradicional entre dois pontos no plano cartesiano. Ela representa o comprimento do segmento de reta que liga os pixels p e q . Essa medida é definida por

$$d_e = \sqrt{(i_1 - i_2)^2 + (j_1 - j_2)^2}$$

Essa é a distância mais natural do ponto de vista geométrico, porém pode ser computacionalmente mais custosa, pois envolve o cálculo de raiz quadrada.

Outra medida bastante utilizada em processamento de imagens é a **distância city-block** (ou **distância Manhattan**). Nesse caso, a distância é calculada considerando apenas deslocamentos horizontais e verticais, como se o movimento ocorresse em uma grade de ruas ortogonais de uma cidade. A distância é dada por

$$d_c = |i_1 - i_2| + |j_1 - j_2|$$

Essa medida está diretamente associada à **vizinhança de 4 pixels**, pois o deslocamento entre os pontos ocorre apenas pelas direções horizontal e vertical.

Uma terceira medida é a **distância xadrez** (ou **distância de Chebyshev**). Nessa definição, a distância entre dois pixels corresponde ao maior deslocamento necessário entre as coordenadas horizontal e vertical. Essa medida é definida por

$$d_x = \max(|i_1 - i_2|, |j_1 - j_2|)$$

O nome distância xadrez vem do fato de que ela corresponde ao número mínimo de movimentos necessários para um **rei** se deslocar entre duas casas em um tabuleiro de xadrez. Essa métrica está associada à **vizinhança de 8 pixels**, pois permite movimentos tanto nas direções ortogonais quanto diagonais.

Essas três medidas de distância são amplamente utilizadas em algoritmos de processamento de imagens, e a escolha de qual utilizar depende do tipo de conectividade e da modelagem geométrica desejada para o problema.

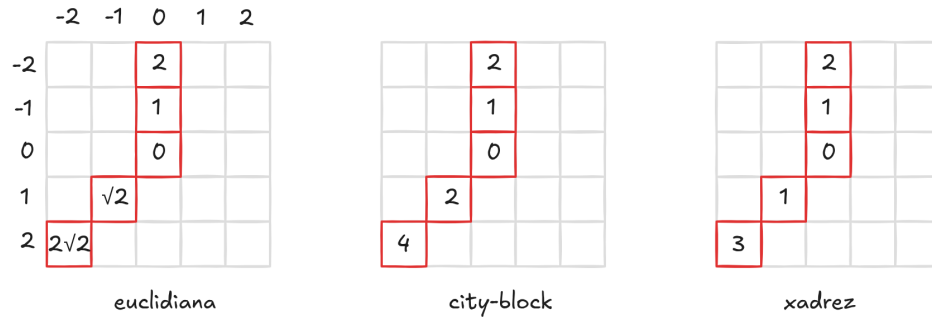


Figura 3.2: Medidas de distância em imagens digitais

3.1.3 Algoritmo: Transformada de Distância em Imagem Binária

Considere uma **imagem binária** na qual as regiões de pixels com valor **1** representam objetos, e deseja-se calcular a **distância de cada pixel do objeto até o fundo (pixels 0)**.

O algoritmo possui **duas fases** e o resultado será uma **imagem em tons de cinza**, onde o valor de cada pixel indica sua distância ao fundo.

Algoritmo TransformadaDistancia(I)

Entrada: imagem binária I[altura][largura]

Saída: imagem de distâncias D[altura][largura]

/* Inicialização */

```
for (y = 0; y < altura; y++)
{
    for (x = 0; x < largura; x++)
    {
        if (I[y][x] == 0)
            D[y][x] = 0;
        else
            D[y][x] = INF;
    }
}
```

/* Fase 1 - Varredura direta */

```
for (y = 0; y < altura; y++)
{
    for (x = 0; x < largura; x++)
    {
        p = D[y][x];
        a = D[y-1][x]; // vizinho acima
        b = D[y][x-1]; // vizinho à esquerda

        if (p != 0)
        {
            D[y][x] = min(a + 1, b + 1);
        }
    }
}
```

/* Fase 2 - Varredura inversa */

maxDist = 0;

```

for (y = altura-1; y >= 0; y--)
{
    for (x = largura-1; x >= 0; x--)
    {
        p = D[y][x];
        a = D[y][x+1];    // vizinho à direita
        b = D[y+1][x];    // vizinho abaixo

        if (p != 0)
        {
            D[y][x] = min(a + 1, b + 1, p);
        }

        if (D[y][x] > maxDist)
        {
            maxDist = D[y][x];
        }
    }
}

return D;

```

Ao final do processo o **maior valor atribuído a *maxDist*** corresponde ao **maior nível de cinza**, ou seja, à **maior distância ao fundo** encontrada na imagem.

Em sequência um simulador desse algoritmo:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/rp0vlNYIX>



3.1.4 Adjacência e Conectividade dos Pixels

Em processamento de imagens, o conceito de **adjacência** é usado para determinar quando dois pixels podem ser considerados vizinhos pertencentes a uma mesma região. Para isso, define-se inicialmente um conjunto V de valores de intensidade que serão utilizados para estabelecer essa relação. Esse conjunto depende do tipo de imagem analisada. Por exemplo, em uma imagem binária pode-se considerar $V = \{1\}$, representando os pixels do objeto. Em imagens em tons de cinza, V pode ser um subconjunto dos níveis de cinza, e em imagens coloridas pode corresponder a uma determinada cor ou faixa de cores.

Dados dois pixels p e q , dizemos que eles possuem **adjacência-4** ou **adjacência-8** quando satisfazem duas condições: ambos possuem valores de intensidade pertencentes ao conjunto V e, além disso, são vizinhos segundo a definição de vizinhança adotada. No caso da adjacência-4, os pixels devem pertencer à vizinhança N_4 . Já na adjacência-8, eles devem pertencer à vizinhança N_8 .

A partir do conceito de adjacência, pode-se definir um **caminho** entre dois pixels. Um caminho de $p = (i_1, j_1)$ até $q = (i_n, j_n)$ é uma sequência de pixels distintos

$$(i_1, j_1), (i_2, j_2), \dots, (i_n, j_n)$$

tal que cada par consecutivo de pixels

$$(i_x, j_x) \text{ e } (i_{x-1}, j_{x-1}), \quad \text{para } 1 < x \leq n$$

é adjacente segundo o critério de vizinhança adotado (adjacência-4 ou adjacência-8).

Considere agora S como um subconjunto de pixels de uma imagem. Dizemos que dois pixels p e q estão **conectados em S** se existir um caminho entre eles formado exclusivamente por pixels pertencentes ao conjunto S .

O conjunto de todos os pixels de S que estão conectados a um determinado pixel p recebe o nome de **componente conexo**. Em outras palavras, uma componente conexa é uma região da imagem formada por pixels adjacentes entre si e que satisfazem o critério de pertencimento ao conjunto V .

3.1.5 Região e Borda

O conjunto de pontos conectados em uma imagem forma uma **região** da imagem. Uma região corresponde, portanto, a um grupo de pixels adjacentes que compartilham determinadas propriedades, como o mesmo valor de intensidade ou pertencimento ao conjunto V . Em muitos contextos de análise de imagens, essa região também pode ser interpretada como uma **forma** presente na imagem.

Os pixels que **não pertencem a nenhuma região de interesse** são denominados **fundo da imagem**. O fundo representa a parte da imagem que não faz parte dos objetos ou estruturas que se deseja analisar.

A **fronteira** de uma região — também chamada de **borda** ou **contorno** — é formada pelos pixels que delimitam a separação entre a região e o fundo. Essa fronteira pode ser definida de duas maneiras. A **borda interna** é o conjunto de pixels da própria região que possuem pelo menos um vizinho pertencente ao fundo da imagem. Já a **borda externa** é composta pelos pixels do fundo que possuem pelo menos um vizinho pertencente à região.

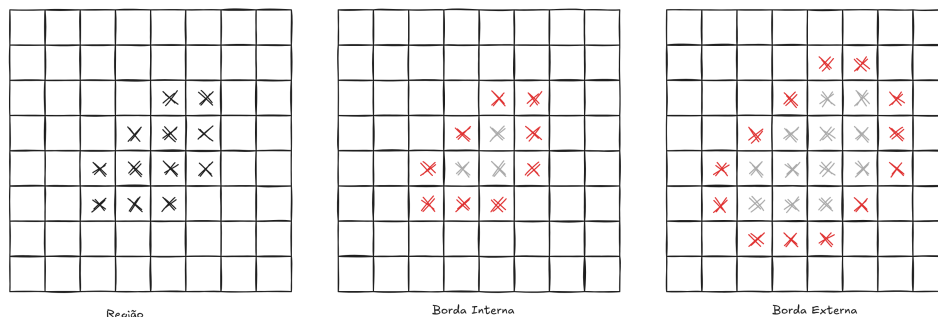


Figura 3.3: Ilustração de uma região, borda interna e borda externa

Ao definir regiões e fronteiras, é importante evitar o chamado **paradoxo da conectividade**, que ocorre quando diferentes critérios de vizinhança produzem ambiguidades na definição de conectividade entre pixels. Para evitar esse problema, utiliza-se uma regra complementar: se a **vizinhança de 8 pixels** (N_8) for utilizada para definir a borda, então a **vizinhança de 4 pixels** (N_4) deve ser utilizada para definir a região. De forma equivalente, se a borda for definida utilizando N_4 , a região deve ser definida utilizando N_8 .

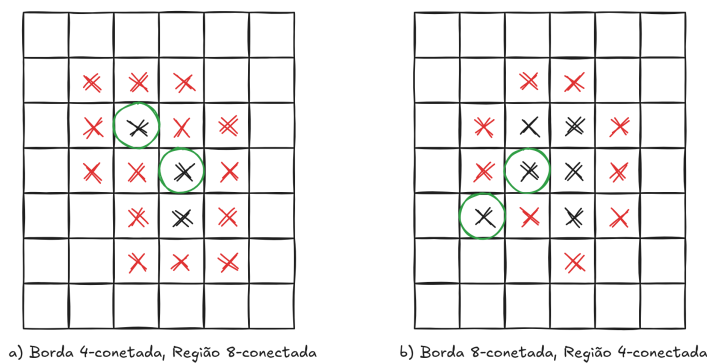


Figura 3.4: Ilustração do paradoxo da conectividade

Em sequência um algoritmo muito útil em processamento de imagens, utilizado para atribuir

um rótulo diferente para cada uma das regiões gráficas encontradas numa imagem binária (onde a forma é identificada por pixel igual a um). O resultado final é uma imagem em tons de cinza, com o número máximo de tons de cinza igual ao número de regiões encontradas na imagem.

3.1.6 Algoritmo: Rotulagem de Componentes Conectados

```
Algoritmo RotulacaoComponentes(I)

Entrada:  imagem binária I[altura][largura]
Saída:    imagem rotulada L[altura][largura]

/* Inicialização */

for (y = 0; y < altura; y++)
{
    for (x = 0; x < largura; x++)
    {
        L[y][x] = 0;
    }
}

proximoRotulo = 1;

/* Primeira varredura */

for (y = 0; y < altura; y++)
{
    for (x = 0; x < largura; x++)
    {
        p = I[y][x];

        if (p == 0)
            continue;

        r = L[y-1][x];    // vizinho acima
        t = L[y][x-1];    // vizinho à esquerda

        if (r == 0 && t == 0)
        {
```

```

        L[y][x] = proximoRotulo;
        proximoRotulo++;
    }
    else if (r != 0 && t == 0)
    {
        L[y][x] = r;
    }
    else if (r == 0 && t != 0)
    {
        L[y][x] = t;
    }
    else
    {
        if (r == t)
        {
            L[y][x] = r;
        }
        else
        {
            L[y][x] = r;
            registrarEquivalencia(r, t);
        }
    }
}

/* Resolução de equivalências */

determinarClassesDeEquivalencia();

/* Segunda varredura */

for (y = 0; y < altura; y++)
{
    for (x = 0; x < largura; x++)
    {
        if (L[y][x] != 0)
        {
            L[y][x] = representante(L[y][x]);
        }
    }
}

```

```
}  
}  
  
return L;
```

Em sequência um simulador desse algoritmo:

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/nAeHKMA9H>



3.2 Conceito de Melhoramento de Imagens

O melhoramento de imagens pode ser definido como o conjunto de técnicas aplicadas a uma imagem com o objetivo de melhorar sua aparência visual ou facilitar sua análise. O conceito de “melhor” é, em grande parte, **subjetivo**, pois depende do observador e da aplicação.

Em geral, as técnicas de melhoramento não seguem um critério único de otimização. Ao contrário, são escolhidas com base na experiência, no conhecimento do domínio da aplicação e nas características específicas da imagem a ser processada.

3.3 Domínios de Processamento

As técnicas de melhoramento de imagens podem ser classificadas de acordo com o domínio em que são aplicadas:

- **Domínio espacial:** as operações são realizadas diretamente sobre os pixels da imagem.
- **Domínio da frequência:** a imagem é transformada para um domínio alternativo, como o domínio da frequência, onde as operações são aplicadas e, posteriormente, a imagem é reconstruída.

Neste capítulo, o foco principal está nas técnicas de melhoramento no domínio espacial, que são conceitualmente mais simples e amplamente utilizadas. No próximo capítulo será apresentada a transformação para o Domínio da frequência, denominada **Transformada de Fourier**, que é mais importante e utilizada em Processamento de Imagens e outras aplicações.

3.4 Melhoramento no Domínio Espacial

No domínio espacial, o processamento é descrito por uma função da forma:

$$g(x, y) = T[f(x, y)]$$

em que $f(x, y)$ representa a imagem de entrada, $g(x, y)$ a imagem resultante e (T) um operador definido sobre os valores de intensidade.

Essas operações podem ser aplicadas a um único pixel ou a uma vizinhança de pixels, dependendo da técnica utilizada.

3.5 Transformações de Intensidade

As transformações de intensidade são operações pontuais, nas quais o valor de cada pixel da imagem de saída depende exclusivamente do valor do pixel correspondente na imagem de entrada.

Essas transformações mapeiam um nível de cinza de entrada (r) para um nível de saída (s), conforme representado no gráfico gerado no exemplo seguinte (onde L representa a quantidade de níveis de cinza):

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/N451LEXKV>



3.5.1 Transformação Linear

A transformação linear é uma das formas mais simples de melhoramento e pode ser expressa como:

$$g(x, y) = a f(x, y) + b$$

Os parâmetros (a) e (b) controlam, respectivamente, o contraste e o brilho da imagem. Valores maiores de (a) ampliam o contraste, enquanto o termo (b) desloca os níveis de intensidade.

3.5.2 Transformações Logarítmicas

Transformações logarítmicas são utilizadas para expandir valores de baixa intensidade e comprimir valores elevados, sendo especialmente úteis em imagens que apresentam grande variação dinâmica.

Uma forma típica é dada por:

$$g(x, y) = c \log(1 + f(x, y))$$

Essas transformações são comuns em aplicações como imagens médicas e espectrais. A constante c é definida para ajustar a transformação dentro de intervalo de intensidade. Um valor possível para a constante c na transformação logarítmica é:

$$c = \frac{max}{\log(1 + max)}$$

Onde $max = L-1$ é a máxima intensidade de pixel na imagem, ou seja, o maior valor retornado para a função imagem $f(x, y)$.

3.5.3 Transformações de Potência

As transformações de potência, também conhecidas como correção gama, são definidas por:

$$g(x, y) = c [f(x, y)]^\gamma$$

O parâmetro (γ) controla a forma da curva de transformação. Valores de ($\gamma < 1$) tem o efeito de clarear a imagem, enquanto valores de ($\gamma > 1$) escurecem a imagem. O exemplo interativo abaixo ilustra essa transformação:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/wtDvsM5Ym>



3.6 Histograma de Imagens

O histograma de uma imagem descreve a distribuição dos níveis de intensidade dos pixels. Ele fornece uma representação estatística que é amplamente utilizada para análise e melhoramento de imagens.

Imagens com baixo contraste tendem a apresentar histogramas concentrados em uma faixa estreita de valores, enquanto imagens com bom contraste exibem histogramas mais espalhados ao longo da escala de intensidades.

Algoritmo para calcular o histograma de uma imagem:

```
// atribuir valor zero a todos elementos do vetor
for (i = 0; i < L; i++)
  Histo[i] = 0;
// calcular distribuição dos níveis de cinza
// para cada pixel da imagem
for (i = 0; i < M; i++)
  for (j = 0; j < N; j++)
    Histo [Img[i][j]] = Histo [Img[i][j]]+1;
```

3.7 Equalização de Histograma

A equalização de histograma é uma técnica de melhoramento que visa redistribuir os níveis de intensidade de uma imagem de forma aproximadamente uniforme. O objetivo é maximizar o contraste global, especialmente em imagens cujos níveis estão concentrados em uma região específica.

Essa técnica é amplamente utilizada devido à sua simplicidade e eficácia, embora possa produzir resultados indesejados em algumas aplicações, como a amplificação de ruído.

O processo chamado de **Mapeamento de Intensidade** (Gonzalez e Woods (2010)) é dado por:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) = \frac{L - 1}{MN} \sum_{j=0}^k n_j$$

onde:

- L é o número de tons de cinza da imagem.
- k representa um tom de cinza, com $k = 0, 1, 2, \dots, L - 1$.
- n_k é o número de pixels de intensidade k na imagem (histograma).
- $p_r(r_k) = \frac{n_k}{MN}$ é a probabilidade de ocorrência do nível k .
- M é o número de linhas da imagem.
- N é o número de colunas da imagem.

Considere uma imagem digital de dimensão 64×64 pixels, totalizando $MN = 4096$ pixels, codificada com **3 bits por pixel**. Nesse caso, o número de níveis de cinza é dado por $L = 2^3 = 8$, com níveis de intensidade no intervalo $[0, L - 1] = [0, 7]$.

A Tabela a seguir apresenta o **histograma da imagem**, a **probabilidade de ocorrência de cada nível de cinza** e o **mapeamento de intensidade** obtido pelo processo de **equalização de histograma**.

r_k	n_k	n_k/MN	$\frac{L-1}{MN} \sum_{j=0}^k n_j = s_k$	s_k (arred.)
$r_0 = 0$	790	0,19	1,33	1
$r_1 = 1$	1023	0,25	3,08	3
$r_2 = 2$	850	0,21	4,55	5
$r_3 = 3$	656	0,16	5,67	6
$r_4 = 4$	329	0,08	6,23	6
$r_5 = 5$	245	0,06	6,65	7
$r_6 = 6$	122	0,03	6,86	7
$r_7 = 7$	81	0,02	7,00	7

Observa-se que o valor n_k corresponde ao **número de pixels com intensidade r_k** , enquanto n_k/MN representa a **probabilidade de ocorrência** desse nível de cinza na imagem.

Esse procedimento define uma **tabela de consulta (LUT)** que mapeia cada nível de entrada r_k para um novo nível s_k , redistribuindo os níveis de cinza de forma mais uniforme e aumentando o contraste global da imagem.

O exemplo interativo a seguir apresenta a implementação do algoritmo de **equalização de histograma**. No exemplo, pode-se carregar qualquer imagem, mas somente as intensidades (tons de cinza), são utilizadas. Para imagem carregada será apresentada a imagem original, o seu histograma correspondente, a imagem equalizada e o histograma equalizado.

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/nFphWvonZ>



3.8 Filtragem

A filtragem no Processamento Digital de Imagens é o conjunto de técnicas utilizadas para modificar ou extrair informações de uma imagem por meio da aplicação de operadores matemáticos que atuam sobre a vizinhança de cada pixel. Esses operadores analisam a relação entre um pixel e seus vizinhos, permitindo realçar características relevantes, como bordas e detalhes, ou atenuar informações indesejadas, como ruídos e variações abruptas de intensidade. A filtragem pode ser realizada no domínio espacial ou no domínio da frequência e envolve tanto filtros lineares, baseados em operações de **convolução**, quanto filtros não lineares, que não obedecem às propriedades de linearidade. Dessa forma, a filtragem é uma etapa fundamental no pré-processamento e na análise de imagens, sendo amplamente utilizada em aplicações de visão computacional, reconhecimento de padrões e análise visual automatizada.

3.8.1 Convolução

A **convolução** é o processo de calcular a intensidade de um determinado pixel em função da intensidade de seus vizinhos. O cálculo é baseado em ponderação, isto é, utilizam-se pesos diferentes para pixels vizinhos diferentes.

A convolução pode ser interpretada como o deslizamento de uma máscara (ou *kernel*) sobre a imagem, onde, a cada posição, é calculada uma soma ponderada entre os valores dos pixels da vizinhança e os coeficientes do kernel.

O exemplo interativo seguinte ilustra o processo de convolução para uma imagem 16x16 em tons de cinza. No exemplo é possível iniciar a simulação (clicando no botão “continuar”), pausar o processo (clicando no botão “pausar”), ou acelerar/desacelerar a simulação, clicando com o mouse sobre a imagem.

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/DvTNxm7gO>



3.8.2 Equação da Convolução

Matematicamente, a convolução bidimensional é expressa por:

$$g(i, j) = \sum_{y=-a}^a \sum_{x=-b}^b w(y, x) f(i + y, j + x)$$

onde:

- $f(i, j)$ é a imagem de entrada;
- $g(i, j)$ é a imagem resultante;
- $w(y, x)$ é a máscara de convolução;
- a e b definem o tamanho do kernel.

3.8.3 Pseudocódigo da Convolução

```
for (i = a; i < nl - a; i++)  
  for (j = b; j < nc - b; j++) {  
    int sum = 0;  
    for (y = -a; y <= a; y++)
```

```

    for (x = -b; x <= b; x++)
        sum += W[y][x] * F[i+y][j+x];
    G[i][j] = sum;
}

```

3.8.4 Operações lineares e não lineares

Operadores **lineares** e **não lineares** constituem classificações importantes no Processamento Digital de Imagens, sendo amplamente utilizadas, por exemplo, na classificação de filtros.

Seja um operador H que produz a imagem de saída $g(x, y)$ a partir da imagem de entrada $f(x, y)$, de acordo com a relação

$$H[f(x, y)] = g(x, y).$$

Diz-se que o operador H é **linear** se satisfaz a seguinte condição:

$$H[a_i f_i(x, y) + a_j f_j(x, y)] = a_i H[f_i(x, y)] + a_j H[f_j(x, y)]$$

ou, de forma equivalente,

$$= a_i g_i(x, y) + a_j g_j(x, y).$$

Essas relações expressam as propriedades de **aditividade** e **homogeneidade**, que caracterizam os operadores lineares.

3.8.5 Exemplos de Filtros

Os filtros passa-baixa, cuja a soma dos coeficientes do kernel é um (ou próxima de um) tem o efeito de atenuar as altas frequências e preservar as baixas frequências. Na prática significa suavização da imagem.

<table><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr></table> passa baixa	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	<table><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table> Identidade	0	0	0	0	1	0	0	0	0
1/9	1/9	1/9																	
1/9	1/9	1/9																	
1/9	1/9	1/9																	
0	0	0																	
0	1	0																	
0	0	0																	
<table><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>-4</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr></table> passa alta	0	1	0	1	-4	1	0	1	0	<table><tr><td>0</td><td>-1</td><td>0</td></tr><tr><td>-1</td><td>5</td><td>-1</td></tr><tr><td>0</td><td>-1</td><td>0</td></tr></table> sharpen	0	-1	0	-1	5	-1	0	-1	0
0	1	0																	
1	-4	1																	
0	1	0																	
0	-1	0																	
-1	5	-1																	
0	-1	0																	
<table><tr><td>-1</td><td>-1</td><td>-1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table> prewitt horizontal	-1	-1	-1	0	0	0	1	1	1	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table> prewitt vertical	-1	0	1	-1	0	1	-1	0	1
-1	-1	-1																	
0	0	0																	
1	1	1																	
-1	0	1																	
-1	0	1																	
-1	0	1																	
<table><tr><td>-1</td><td>-2</td><td>-1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>2</td><td>1</td></tr></table> sobel horizontal	-1	-2	-1	0	0	0	1	2	1	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table> sobel vertical	-1	0	1	-2	0	2	-1	0	1
-1	-2	-1																	
0	0	0																	
1	2	1																	
-1	0	1																	
-2	0	2																	
-1	0	1																	

Figura 3.5: Filtros

Um filtro passa-alta realça bordas e detalhes, atenuando as baixas frequências (regiões suaves). Em PDI, isso é feito com kernels cuja soma dos coeficientes é zero (ou próxima de zero).

Regra Prática:

- Passa-baixa = a soma dos coeficientes no kernel é 1 (um);
- Passa-alta = a soma dos coeficientes no kernel é 0 (zero);
- High-boost (realce) = preserva a imagem, a soma dos coeficientes no kernel é maior que 1 (um).

No exemplo interativo em sequência, esses diversos filtros podem ser testados:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/2T58I4gUQ>



3.9 Considerações sobre Ruído

O ruído é uma variação indesejada nos valores de intensidade da imagem, geralmente introduzida durante a aquisição ou transmissão. Técnicas de melhoramento devem considerar a presença de ruído, pois operações de realce podem amplificá-lo.

Assim, o melhoramento de imagens frequentemente envolve um compromisso entre **realce de detalhes** e **supressão de ruído**.

3.9.1 Ruído Sal e Pimenta

O **ruído sal e pimenta** é um tipo de ruído impulsivo muito comum em **processamento digital de imagens**. Ele se manifesta como pixels isolados com valores extremos, geralmente **preto (0)** e **branco (255)** em imagens em tons de cinza, lembrando grãos de sal e pimenta espalhados sobre a imagem.

Esse ruído costuma surgir devido a:

- falhas em sensores de aquisição;
- erros de transmissão de dados;
- conversões analógico-digitais defeituosas;
- pixels mortos ou saturados.

Diferentemente de ruídos aditivos, o ruído sal e pimenta **não afeta todos os pixels**, mas apenas uma fração deles, substituindo seus valores originais por valores extremos.

3.9.1.1 Modelo matemático

O modelo clássico do ruído sal e pimenta pode ser descrito por:

$$g(x, y) = \begin{cases} 0, & \text{com probabilidade } p_p \\ 255, & \text{com probabilidade } p_s \\ f(x, y), & \text{com probabilidade } 1 - (p_p + p_s) \end{cases}$$

onde:

- $f(x, y)$ é a imagem original;
- $g(x, y)$ é a imagem corrompida;
- p_p é a probabilidade de ocorrência da *pimenta* (pixel preto);
- p_s é a probabilidade de ocorrência do *sal* (pixel branco).

3.9.1.2 Características principais

- Ruído **não gaussiano**;
- Afeta pixels de forma **esparsa e abrupta**;
- Facilmente perceptível visualmente;
- Pode degradar significativamente bordas e detalhes finos.

3.9.1.3 Técnicas de remoção

Filtros lineares, como o **filtro da média**, não são adequados para esse tipo de ruído, pois tendem a borrar a imagem. A abordagem mais eficaz é o uso de **filtros não lineares**, em especial:

- **Filtro da mediana**: remove impulsos preservando bordas;
- **Filtro da mediana adaptativo**: ajusta o tamanho da janela conforme a densidade do ruído.

O filtro da mediana substitui o valor do pixel pelo valor mediano de sua vizinhança, sendo especialmente eficiente para eliminar pixels extremos isolados.

No exemplo interativo em sequência está um programa que permite carregar uma imagem (que é convertida em tons de cinza), gerar ruído sal e pimenta de acordo com probabilidades definidas e em seguida aplicar o filtro da mediana para remoção do ruído, para demonstração destes conceitos.

 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/xk5SXodH3>



3.10 Considerações Finais

O melhoramento de imagens desempenha um papel fundamental no processamento digital, servindo tanto para aprimorar a visualização quanto para preparar imagens para etapas posteriores, como segmentação e reconhecimento.

As técnicas apresentadas neste capítulo fornecem uma base conceitual para o entendimento do melhoramento no domínio espacial, sendo essenciais para o estudo de métodos mais avançados de processamento e análise de imagens.

4 Transformada de Fourier

A análise de imagens no domínio espacial, embora intuitiva e amplamente utilizada, apresenta limitações quando se deseja compreender a distribuição de padrões periódicos, texturas e estruturas repetitivas. Para superar essas limitações, recorre-se à análise no **domínio da frequência**, na qual a imagem é representada como uma combinação de componentes senoidais de diferentes frequências.

A **Transformada de Fourier** constitui a principal ferramenta matemática para essa mudança de domínio, sendo fundamental em diversas áreas do Processamento Digital de Imagens, como filtragem, compressão, restauração e análise de textura.

4.1 Domínio Espacial e Domínio da Frequência

No domínio espacial, uma imagem é representada diretamente por seus valores de intensidade, descritos por uma função bidimensional $f(x, y)$. Cada ponto (x, y) corresponde a um pixel da imagem.

No domínio da frequência, a mesma imagem é descrita em termos de suas componentes de frequência espacial. Frequências baixas estão associadas a variações suaves de intensidade, enquanto frequências altas representam transições abruptas, como bordas e detalhes finos.

Essa mudança de representação permite compreender e manipular características da imagem que não são facilmente observáveis no domínio espacial.

4.2 Fundamento

4.2.1 Mudança de domínio como estratégia de simplificação

A resolução de muitos problemas torna-se complexa quando realizada diretamente no domínio original. Operações como divisão, convolução ou correlação frequentemente exigem alto custo computacional ou manipulações algébricas trabalhosas. Uma estratégia clássica para contornar essa dificuldade consiste em **transformar o problema para outro domínio**, no qual as operações envolvidas se tornam mais simples.

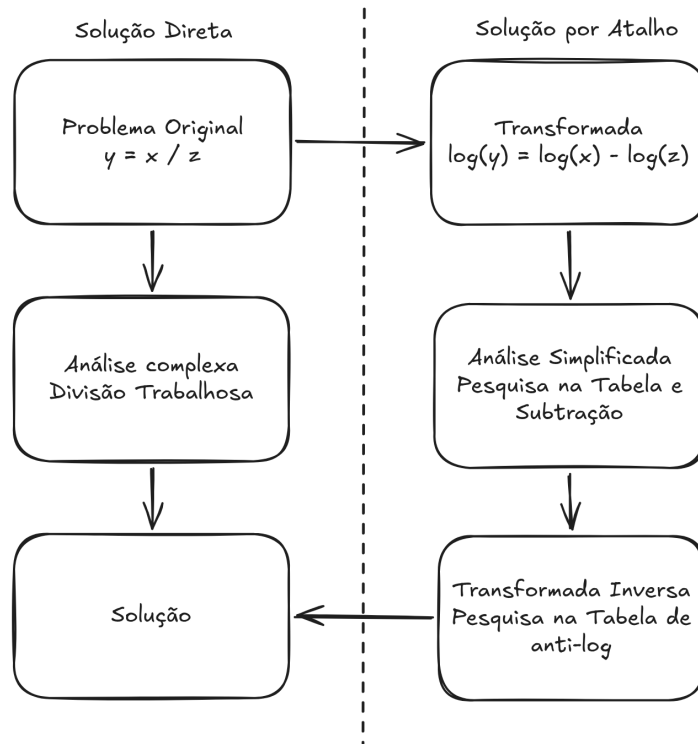


Figura 4.1: Mudança de domínio

O diagrama da Figura 4.1 ilustra esse princípio por meio de um exemplo elementar. No domínio original, a relação

$$y = \frac{x}{z}$$

exige uma operação de divisão, que pode ser inconveniente do ponto de vista analítico ou computacional. Ao aplicar uma **transformação logarítmica**, o problema é levado a um novo domínio, no qual a divisão é convertida em subtração:

$$\log(y) = \log(x) - \log(z).$$

Nesse domínio transformado, a análise torna-se significativamente mais simples, pois a subtração pode ser realizada de forma direta, inclusive com o auxílio de tabelas ou métodos mais eficientes.

Após a resolução no domínio transformado, aplica-se a **transformada inversa** (anti-logaritmo), retornando o resultado ao domínio original. Assim, obtém-se a solução do problema inicial por meio de um caminho indireto, porém mais eficiente.

Esse raciocínio fundamenta o uso de transformadas em diversas áreas, em especial no **Processamento Digital de Imagens**. Transformadas como a logarítmica, a de Fourier e a de Laplace permitem converter operações complexas no domínio espacial em operações mais simples em domínios alternativos, justificando o uso da expressão “solução usando um atalho”. A mudança de domínio, portanto, não altera o problema em si, mas revela uma forma mais conveniente de resolvê-lo.

4.2.2 Histórico

Em 1807, **Joseph Fourier** propôs uma ideia que, à época, foi recebida com grande ceticismo: **funções periódicas arbitrárias podem ser representadas como uma soma ponderada de senos e cossenos**. Em outras palavras, Fourier afirmou que sinais complexos podem ser decompostos em um conjunto de oscilações simples, cada uma contribuindo com uma parcela específica para a forma final do sinal.

Essa proposta contrariava a intuição matemática dominante do início do século XIX. Muitos matemáticos questionavam como funções com descontinuidades ou formas irregulares poderiam ser descritas por combinações de funções suaves e contínuas, como senos e cossenos. Apesar das críticas iniciais, a ideia mostrou-se extremamente poderosa.

Com o desenvolvimento da **série de Fourier**, tornou-se possível analisar um sinal não apenas em termos de sua variação no tempo ou no espaço, mas também em termos de suas **frequências constituintes**. Essa mudança de perspectiva — do domínio original para o domínio da frequência — revelou que operações complexas sobre sinais podem ser compreendidas e tratadas de forma muito mais simples.

Atualmente, a ideia de Fourier é um dos pilares da ciência e da engenharia, sendo fundamental em áreas como processamento de sinais, comunicações e **Processamento Digital de Imagens**, onde a decomposição de uma imagem em componentes de frequência permite compreender, filtrar e modificar sua estrutura de maneira eficiente.

4.2.3 A ideia fundamental de Fourier

A função denominada “Soma das senóides”, apresentada no exemplo interativo em sequência é obtida pela **soma das quatro funções** (Senoide 1, Senoide 2, Senoide 3, Senoide 4). Cada uma dessas funções elementares é uma senoide com frequência e amplitude distintas. Quando combinadas, elas produzem uma forma de onda mais complexa, que já não se parece, à primeira vista, com um seno simples.

Dica

O código completo e a demonstração interativa estão disponíveis em:

https://editor.p5js.org/luizedsilva/full/Dn_cYEuqj



O exemplo ilustrativo seguinte, adaptado do código de **Daniel Shiffman**¹, demonstra como uma série de Fourier pode ser utilizada para representar uma onda quadrada.

Nesse exemplo, cada **termo da série** é representado por um **círculo trigonométrico** (*epiciclo*). O **raio de cada círculo** corresponde à **amplitude** do termo, enquanto a **frequência** é representada pela **velocidade de rotação** do círculo. Os círculos são conectados de forma sequencial, de modo que o **centro de cada círculo** coincide com a **extremidade do raio do círculo anterior**, conforme ilustrado na Figura 4.2.

A **soma dos termos da série** é obtida pela **projeção vertical** (eixo y) do vetor resultante, formado pelo encadeamento dos raios de todos os círculos. À medida que mais termos são adicionados, a forma de onda resultante aproxima-se progressivamente de uma **onda quadrada**, evidenciando o princípio fundamental da série de Fourier: a representação de funções periódicas complexas como a soma de senos e cossenos.

¹(<https://thecodingtrain.com/CodingChallenges/125-fourier-series.html>)

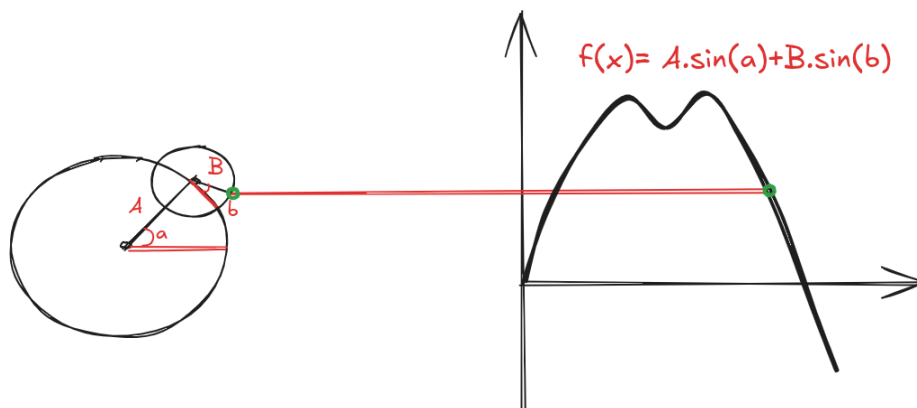


Figura 4.2: Representação dos termos como Epiciclos

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/kk4tjuY7W>



4.3 Transformada de Fourier Unidimensional

4.3.1 Definição Matemática

A **Transformada de Fourier contínua unidimensional** de uma função $f(t)$ é definida por:

$$F(\mu) = \int_{-\infty}^{\infty} f(t), e^{-j2\pi\mu t}, dt$$

onde:

- $f(t)$ é o sinal no domínio do tempo (ou espaço),
- $F(\mu)$ é a representação do sinal no domínio da frequência,

- μ representa a frequência,
- $j = \sqrt{-1}$ é a unidade imaginária.

O termo exponencial complexo pode ser interpretado como uma combinação de seno e cosseno, conforme a identidade de Euler:

$$e^{j\theta} = \cos \theta + j \sin \theta$$

Assim, a Transformada de Fourier projeta o sinal original sobre um conjunto de **funções base senoidais**, medindo o quanto cada frequência está presente no sinal.

4.3.2 Interpretação Física e Intuitiva

Cada valor de $F(\mu)$ é um **número complexo** que contém duas informações fundamentais:

- **Magnitude**

$$|F(\mu)| = \sqrt{\text{Re}^2 + \text{Im}^2}$$

indica a **energia ou intensidade** da frequência μ .

- **Fase**

$$\phi(\mu) = \tan^{-1} \left(\frac{\text{Im}}{\text{Re}} \right)$$

indica o **deslocamento** da componente senoidal no tempo.

Em muitas aplicações práticas, especialmente em Processamento de Imagens, a análise concentra-se principalmente na **magnitude do espectro**, pois ela revela onde a energia do sinal está concentrada.

4.3.3 Transformada Inversa de Fourier Unidimensional

A reconstrução do sinal original a partir de sua representação em frequência é realizada pela **Transformada Inversa de Fourier**, definida por:

$$f(t) = \int_{-\infty}^{\infty} F(\mu), e^{j2\pi\mu t}, d\mu$$

Essa relação garante que a Transformada de Fourier **não perde informação**: o sinal pode ser transformado para o domínio da frequência e recuperado integralmente, desde que todas as frequências sejam consideradas.

4.3.3.1 Um exemplo

Abaixo está um exemplo de uma cálculo no domínio contínuo para uma função da Figura 4.3 a.

$$\begin{aligned}
 F(\mu) &= \int_{-\infty}^{\infty} f(t)e^{-j2\pi\mu t} dt \\
 &= \int_{-W/2}^{W/2} Ae^{-j2\pi\mu t} dt \\
 &= \frac{-A}{j2\pi\mu} [e^{-j2\pi\mu t}]_{-W/2}^{W/2} \\
 &= \frac{-A}{j2\pi\mu} [e^{-j\pi\mu W} - e^{j\pi\mu W}] \\
 &= \frac{A}{j2\pi\mu} [e^{j\pi\mu W} - e^{-j\pi\mu W}] \\
 &= AW \frac{\sin(\pi\mu W)}{\pi\mu W}
 \end{aligned}$$

onde:

$$\sin \theta = \frac{e^{j\theta} - e^{-j\theta}}{2j}$$

A Figura 4.3 ilustra essa transformação contínua unidimensional:

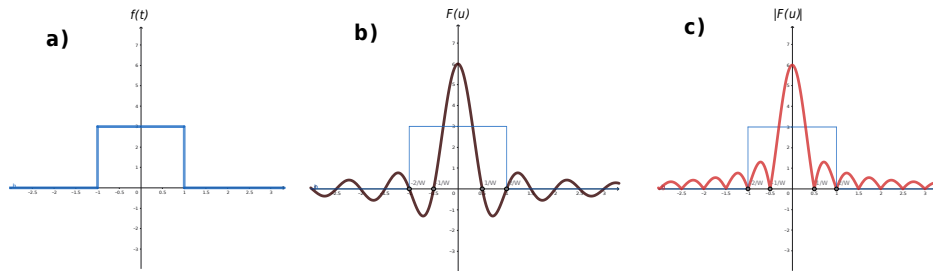


Figura 4.3: Transformação: a) função simples, b) transformada calculada e c) espectro

A simulação em Geogebra em sequência demonstra essa função pulso, o espectro calculado e o módulo do espectro:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://www.geogebra.org/calculator/f6gyatmc?embed>



4.3.4 Transformada Discreta de Fourier para uma Variável

Em aplicações computacionais, os sinais são representados de forma discreta. Nesse contexto, utiliza-se a **Transformada Discreta de Fourier (DFT)**, que opera sobre um conjunto finito de amostras. Seja $f(x)$ um sinal discreto definido para $x = 0, 1, 2, \dots, M-1$. A DFT é definida por:

$$F(\mu) = \sum_{x=0}^{M-1} f(x) e^{-j2\pi\mu x/M}, \quad \mu = 0, 1, 2, \dots, M-1$$

Cada coeficiente $F(\mu)$ representa a contribuição de uma frequência discreta específica no sinal original.

De forma análoga ao caso contínuo, o sinal pode ser reconstruído por meio da **Transformada Discreta Inversa de Fourier (IDFT)**, dada por:

$$f(x) = \frac{1}{M} \sum_{\mu=0}^{M-1} F(\mu) e^{j2\pi\mu x/M}, \quad x = 0, 1, 2, \dots, M-1$$

A DFT e sua inversa formam um par de transformações mutuamente inversas, assegurando que a representação no domínio da frequência contém toda a informação necessária para a recuperação do sinal original. Esses conceitos constituem a base para algoritmos eficientes, como a **Transformada Rápida de Fourier (FFT)**, amplamente utilizada no processamento digital de sinais e imagens.

4.4 A Transformada de Fourier Bidimensional

A **Transformada de Fourier bidimensional (2D)** é uma extensão natural da Transformada de Fourier unidimensional e constitui uma das ferramentas mais importantes do **Processamento Digital de Imagens (PDI)**. Enquanto a versão unidimensional analisa sinais ao longo de uma única variável independente, a versão bidimensional permite analisar funções definidas sobre duas variáveis espaciais, como imagens digitais.

Uma imagem pode ser interpretada como uma função de intensidade luminosa $f(x, y)$, em que x e y representam as coordenadas espaciais. A Transformada de Fourier 2D permite representar essa imagem no **domínio da frequência espacial**, revelando padrões, texturas e estruturas que nem sempre são evidentes no domínio espacial.

4.4.1 Motivação para o Domínio da Frequência

No domínio espacial, as informações estão organizadas de acordo com a posição dos pixels. Já no domínio da frequência, a imagem é descrita em termos de **variações espaciais de intensidade**.

De forma intuitiva:

- **Baixas frequências** correspondem a variações suaves de intensidade (regiões homogêneas).
- **Altas frequências** estão associadas a variações abruptas, como bordas, detalhes finos e ruído.

Muitas operações importantes, como filtragem, realce e supressão de ruído, tornam-se mais simples e intuitivas quando realizadas no domínio da frequência.

4.4.2 Definição Matemática

A **Transformada de Fourier bidimensional contínua** de uma função $f(x, y)$ é definida por:

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

onde:

- $f(x, y)$ é a imagem no domínio espacial;
- $F(u, v)$ é a representação no domínio da frequência;
- u e v representam as frequências espaciais nas direções x e y ;
- $j = \sqrt{-1}$.

Essa expressão mostra que a imagem é projetada sobre um conjunto de **funções base senoidais bidimensionais**, cada uma caracterizada por uma orientação e uma frequência espacial específicas.

4.4.3 Interpretação do Espectro de Fourier Bidimensional

O resultado da Transformada de Fourier 2D é uma função **complexa**, em que cada ponto (u, v) contém informações sobre uma frequência espacial específica.

Assim como no caso unidimensional, cada coeficiente possui:

- **Magnitude:**

$$|F(u, v)| = \sqrt{\text{Re}^2 + \text{Im}^2}$$

que indica a intensidade daquela frequência;

- **Fase**, que codifica a posição espacial relativa das estruturas da imagem.

Em aplicações práticas, a visualização do espectro geralmente utiliza a **magnitude em escala logarítmica**:

$$S(u, v) = \log(1 + |F(u, v)|)$$

Essa transformação é necessária porque a faixa dinâmica dos valores do espectro é muito ampla.

4.4.4 Frequência Espacial e Orientação

Na Transformada de Fourier 2D:

- A **distância ao centro** do espectro está relacionada à frequência espacial;
- A **direção em relação ao centro** indica a orientação das estruturas na imagem.

Por exemplo:

- Linhas verticais na imagem geram componentes dominantes ao longo do eixo horizontal do espectro;
- Texturas periódicas produzem picos bem definidos em posições específicas do domínio da frequência.

Assim, o espectro de Fourier fornece uma poderosa ferramenta para análise estrutural e direcional de imagens.

4.4.5 Transformada Inversa de Fourier Bidimensional

A imagem original pode ser recuperada a partir de seu espectro por meio da **Transformada Inversa de Fourier 2D**, dada por:

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{j2\pi(ux+vy)} du dv$$

Essa relação garante que a transformação é **reversível** e que nenhuma informação é perdida, desde que todas as frequências sejam preservadas.

No contexto de imagens digitais, que são funções discretas e bidimensionais, utiliza-se a **Transformada Discreta de Fourier (DFT)**, que fornece uma representação igualmente discreta no domínio da frequência.

4.4.6 Transformada Discreta de Fourier Bidimensional

Seja uma imagem digital $f(x, y)$ de dimensões $M \times N$. A Transformada Discreta de Fourier bidimensional é definida por:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y), e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

onde (u) e (v) representam as coordenadas no domínio da frequência e (j) é a unidade imaginária.

A transformada inversa permite recuperar a imagem original a partir de sua representação espectral:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v), e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

4.5 Transformada Rápida de Fourier (FFT)

A Transformada Discreta de Fourier (DFT) é uma ferramenta fundamental para análise no domínio da frequência. Entretanto, sua implementação direta possui alto custo computacional.

Para um sinal bidimensional de tamanho $M \times N$, a complexidade é da ordem de:

$$\mathcal{O}((MN)^2)$$

No caso de uma imagem de 1024×1024 pixels, isso implica aproximadamente um trilhão de multiplicações e somas, tornando o cálculo direto inviável para muitas aplicações práticas.

A **Transformada Rápida de Fourier (FFT)** reduz essa complexidade para:

$$\mathcal{O}(MN \log_2 MN)$$

Para a mesma imagem, o número de operações cai para cerca de 20 milhões, uma redução substancial.

A ideia central da FFT é reescrever a DFT como a combinação de DFTs menores.

4.5.1 Desenvolvimento Matemático

Considere a DFT unidimensional de tamanho M :

$$F(u) = \sum_{x=0}^{M-1} f(x) W_M^{ux}$$

onde

$$W_M = e^{-j\frac{2\pi}{M}}$$

Suponha que $M = 2K$, isto é, M é potência de 2.

Então:

$$F(u) = \sum_{x=0}^{2K-1} f(x) W_{2K}^{ux}$$

Separando os termos pares e ímpares:

$$F(u) = \sum_{x=0}^{K-1} f(2x) W_{2K}^{u(2x)} + \sum_{x=0}^{K-1} f(2x+1) W_{2K}^{u(2x+1)}$$

Utilizando a propriedade:

$$W_{2K}^{2ux} = W_K^{ux}$$

obtemos:

$$F(u) = \sum_{x=0}^{K-1} f(2x)W_K^{ux} + \sum_{x=0}^{K-1} f(2x+1)W_K^{ux}W_{2K}^u$$

Definindo:

$$F_{par}(u) = \sum_{x=0}^{K-1} f(2x)W_K^{ux}$$

$$F_{impar}(u) = \sum_{x=0}^{K-1} f(2x+1)W_K^{ux}$$

temos a expressão fundamental da FFT:

$$F(u) = F_{par}(u) + W_{2K}^u F_{impar}(u)$$

Além disso, usando a propriedade:

$$W_{2K}^{u+K} = -W_{2K}^u$$

obtemos:

$$F(u+K) = F_{par}(u) - W_{2K}^u F_{impar}(u)$$

Essa decomposição mostra que a DFT de tamanho $2K$ pode ser escrita como duas DFTs de tamanho K combinadas por fatores complexos.

4.5.2 Restrição

A FFT radix-2 exige que:

$$M = 2^n$$

ou seja, o número de pontos deve ser potência de 2.

4.5.3 Reordenação Bit-Reversa (Bit Reversal)

Durante a decomposição recursiva da FFT, cada termo pode ser classificado, em cada etapa, como par p ou ímpar i . Ao substituir p por 0 e i por 1, obtém-se uma sequência binária associada ao caminho percorrido pelo termo ao longo das divisões sucessivas. O índice original correspondente a esse termo é exatamente o número binário formado com a ordem dos bits invertida. Essa reorganização sistemática dos índices é conhecida como **bit reversal** (reordenação bit-reversa) e permite que o algoritmo seja implementado de forma iterativa e eficiente.

4.5.3.1 Exemplo para $N = 8$

Considere a seguinte função discreta:

$$\{f(0), f(1), f(2), f(3), f(4), f(5), f(6), f(7)\}$$

Invertendo os bits:

Original	Binário	Bit-reverso	Novo índice
f(0)	000	000	f(0)
f(1)	001	100	f(4)
f(2)	010	010	f(2)
f(3)	011	110	f(6)
f(4)	100	001	f(1)
f(5)	101	101	f(5)
f(6)	110	011	f(3)
f(7)	111	111	f(7)

A reordenação dos dados permite que o algoritmo seja implementado de forma iterativa e eficiente.

4.5.4 Estrutura Butterfly

A combinação dos termos pode ser representada por:

$$\text{saída superior} = a + Wb$$

$$\text{saída inferior} = a - Wb$$

Essa estrutura é conhecida como **butterfly** (imagem seguinte), pois o diagrama de fluxo assume esse formato gráfico. A aplicação sucessiva dessa decomposição reduz o problema até termos triviais de tamanho 1.

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/gdGhjn6LZ>



4.5.4.1 Um exemplo

A Figura 4.4 a seguir ilustra um conjunto discreto de amostras de um sinal unidimensional no domínio espacial. Esse exemplo simples permite compreender, de forma concreta, como a **Transformada Discreta de Fourier (DFT)** decompõe um sinal em suas componentes de frequência.

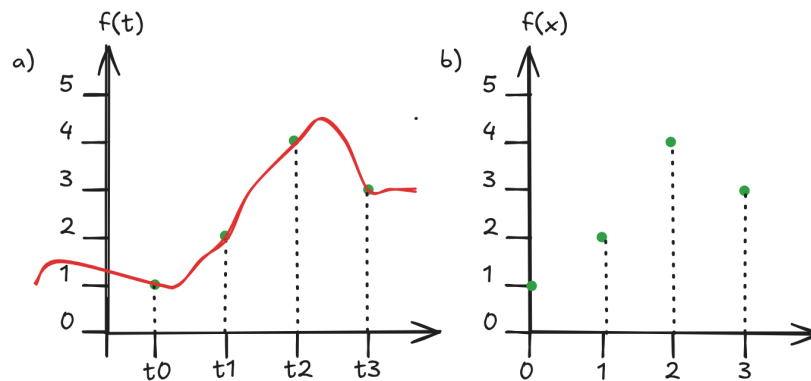


Figura 4.4: Amostras do sinal no domínio espacial

Considere o sinal discreto $f(x)$, definido para $x = 0, 1, 2, 3$, com valores $f(0) = 1$, $f(1) = 2$, $f(2) = 4$ e $f(3) = 3$, obtidos de uma função $f(t)$. O coeficiente de frequência zero da DFT, $F(0)$, corresponde à soma de todas as amostras do sinal e representa a **componente DC** (*Direct Current*), isto é, o valor médio escalado do sinal. Assim, tem-se:

$$\begin{aligned}
F(0) &= \sum_{x=0}^3 f(x) \\
&= f(0) + f(1) + f(2) + f(3) \\
&= 1 + 2 + 4 + 3 = 10
\end{aligned}$$

O coeficiente $F(1)$ é obtido multiplicando cada amostra do sinal por um termo exponencial complexo que representa uma frequência específica. Esse processo projeta o sinal sobre uma base senoidal complexa. Para $u = 1$, obtém-se:

$$\begin{aligned}
F(1) &= \sum_{x=0}^3 f(x)e^{-j2\pi(1)x/4} \\
&= 1e^0 + 2e^{-j\pi/2} + 4e^{-j\pi} + 3e^{-j3\pi/2} \\
&= -3 + 1j
\end{aligned}$$

De forma análoga, os demais coeficientes da DFT são calculados, resultando em:

$$F(2) = \sum_{x=0}^3 f(x)e^{-j2\pi(2)x/4} = 0, \quad F(3) = \sum_{x=0}^3 f(x)e^{-j2\pi(3)x/4} = -3 - 1j$$

Esses valores constituem a **representação do sinal no domínio da frequência**, na qual cada coeficiente complexo codifica a contribuição de uma frequência específica, por meio de sua magnitude e fase.

Uma vez conhecidos os coeficientes $F(u)$, é possível reconstruir o sinal original aplicando a **Transformada Discreta Inversa de Fourier (IDFT)** (considerando j ao invés de $-j$ nos coeficientes e multiplicando o resultado do somatório por $\frac{1}{N}$). Por exemplo, o valor da amostra $f(0)$ pode ser obtido por:

$$\begin{aligned}
f(0) &= \frac{1}{4} \sum_{u=0}^3 F(u)e^{j2\pi u(0)} \\
&= \frac{1}{4} \sum_{u=0}^3 F(u) \\
&= \frac{1}{4} [10 - 3 + 1j + 0 - 3 - 1j] \\
&= \frac{1}{4} [4] = 1
\end{aligned}$$

O exemplo interativo seguinte simula os mesmos cálculos com as entradas reordenadas e os cálculos sendo realizados considerando o *esquema butterfly*:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/2pBrQ6PYM>



Usando o *esquema butterfly* também é possível reconstruir o sinal original. Basta utilizar os valores dos coeficientes $F(u)$ reordenados como entrada, considerar j ao invés de $-j$ nos coeficientes e multiplicar o resultado do somatório por $\frac{1}{N}$.

4.5.5 Centralização do Espectro

Por definição, a frequência zero (componente DC) da Transformada de Fourier está localizada na origem $(u, v) = (0, 0)$, que aparece nos cantos da imagem do espectro.

Para facilitar a análise visual, é comum **centralizar o espectro**, deslocando a componente DC para o centro da imagem. Isso pode ser feito por meio da modulação espacial:

$$f_c(x, y) = f(x, y) (-1)^{x+y}$$

ou, alternativamente, pela troca dos quadrantes do espectro após a transformação. Essa etapa **não altera o conteúdo espectral**, apenas reorganiza sua visualização.

O exemplo interativo a seguir apresenta uma demonstração do cálculo do espectro de uma imagem (que na entrada é redimensionada para dimensão 256x256). Depois o espectro é calculado e apresentado ao lado da imagem selecionada:

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/RCKGkjFiN>



4.6 Filtragem no Domínio da Frequência

A filtragem no domínio da frequência baseia-se na Transformada de Fourier. Em vez de operar diretamente sobre os pixels da imagem, realiza-se a modificação do seu espectro. Essa abordagem permite manipular separadamente as diferentes componentes de frequência presentes na imagem.

Seja $f(x, y)$ uma imagem no domínio espacial. O processo de filtragem inicia-se com o cálculo da Transformada de Fourier da imagem, representado por

$$F(u, v) = \mathcal{F}\{f(x, y)\}.$$

Em seguida, define-se uma função filtro no domínio da frequência, denotada por $H(u, v)$. Esse filtro atua como uma máscara espectral que modifica seletivamente determinadas frequências.

A etapa seguinte consiste na multiplicação ponto a ponto entre o espectro da imagem e o filtro:

$$G(u, v) = H(u, v)F(u, v).$$

Por fim, aplica-se a transformada inversa de Fourier para reconstruir a imagem filtrada:

$$g(x, y) = \mathcal{F}^{-1}\{G(u, v)\}.$$

A função $g(x, y)$ corresponde à imagem resultante após a filtragem.

Os filtros no domínio da frequência geralmente são definidos em função da distância radial ao centro do espectro. Essa distância é dada por:

$$D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}.$$

O centro do espectro corresponde às baixas frequências, associadas às variações suaves de intensidade. À medida que se afasta do centro, encontram-se as altas frequências, responsáveis por bordas e detalhes finos.

O filtro ideal (representado na Figura 4.5 a) passa-baixa possui uma transição abrupta entre as frequências preservadas e as atenuadas. Sua definição é:

$$H(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases}$$

onde D_0 representa a frequência de corte. Embora simples, esse filtro pode introduzir artefatos como o **fenômeno de Gibbs**² devido à descontinuidade na resposta espectral.

O filtro gaussiano (representado na Figura 4.5 b) apresenta uma transição suave e contínua, evitando descontinuidades abruptas. Sua expressão é:

$$H(u, v) = e^{-\frac{D(u, v)^2}{2D_0^2}}.$$

Esse filtro não produz ringing significativo e resulta em suavização progressiva das altas frequências.

O filtro Butterworth (representado na Figura 4.5 c) representa um compromisso entre o ideal e o gaussiano. Sua formulação é:

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}},$$

onde D_0 é a frequência de corte e n é a ordem do filtro. Para valores pequenos de n , a transição é suave. À medida que n aumenta, o comportamento aproxima-se do filtro ideal.

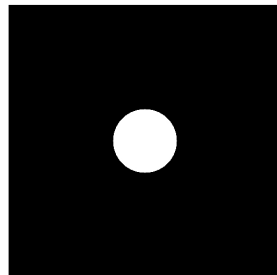
O exemplo interativo seguinte apresenta uma visualização 3D destes três filtros. Pode-se escolher o raio do Corte e a Ordem (para o caso do filtro Butterworth):

Dica

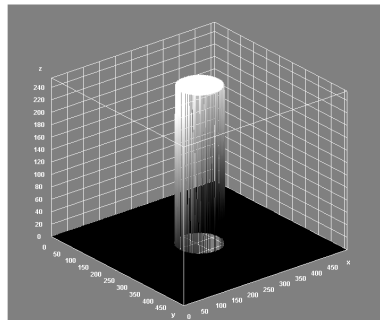
O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/kkrvpU7ra>

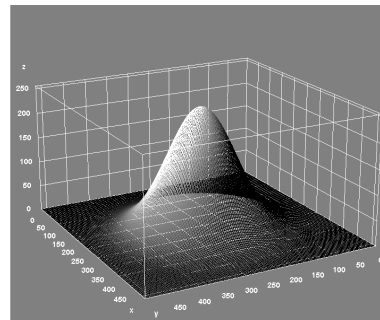
²(https://pt.wikipedia.org/wiki/Fen%C3%B4meno_de_Gibbs)



a) Ideal



b) Gaussiano



c) Butterworth

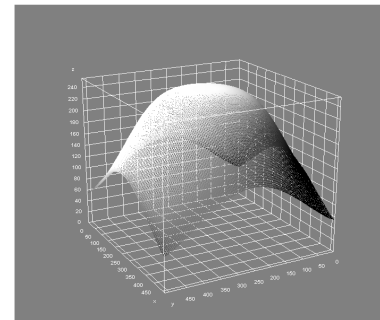


Figura 4.5: Filtros para Transformada de Fourier



Filtros passa-alta podem ser obtidos a partir dos passa-baixa por meio da relação:

$$H_{PA}(u, v) = 1 - H_{PB}(u, v).$$

Nesse caso, as altas frequências são preservadas, destacando bordas e estruturas finas da imagem.

O processo de restauração do espectro consiste em transformar a imagem para o domínio da frequência, modificar seletivamente suas componentes espectrais e reconstruí-la por meio da transformada inversa. Matematicamente, esse procedimento pode ser representado por:

$$g(x, y) = \mathcal{F}^{-1}\{H(u, v)\mathcal{F}[f(x, y)]\}.$$

Essa abordagem permite suavização, realce de detalhes, redução de ruído e correção de degradações, oferecendo maior controle matemático sobre o comportamento do filtro e eficiência computacional quando implementada via FFT.

O Exemplo interativo seguinte apresenta uma aplicação do processo de filtragem por Fourier.

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/3HBiKuxP2>



4.7 Vantagens e Limitações

A análise no domínio da frequência oferece uma poderosa ferramenta para o processamento de imagens, permitindo a implementação de operações que seriam complexas no domínio espacial. No entanto, o custo computacional da transformada e a perda de localização espacial direta são aspectos que devem ser considerados.

Na prática, o uso da **Transformada Rápida de Fourier (FFT)** torna a aplicação da DFT viável mesmo para imagens de grandes dimensões.

4.8 Considerações Finais

A Transformada de Fourier desempenha um papel central no Processamento Digital de Imagens, fornecendo uma base matemática sólida para a análise e manipulação de informações no domínio da frequência. A compreensão de seus fundamentos é essencial para o estudo de técnicas avançadas de filtragem, restauração e análise espectral de imagens.

5 Compressão de Imagens

O crescimento acelerado da produção e do armazenamento de imagens digitais impõe desafios significativos relacionados ao uso eficiente de memória e largura de banda. Imagens digitais, especialmente aquelas com alta resolução e profundidade de bits, demandam grandes quantidades de espaço para armazenamento e transmissão. Nesse contexto, a **compressão de imagens** surge como uma área fundamental do Processamento Digital de Imagens (PDI).

Considere um vídeo em resolução **SD** (720×480) com **30 quadros por segundo**, onde cada pixel é representado por **3 bytes** (RGB).

O número de bytes necessários para representar **um segundo de vídeo** pode ser calculado por:

$$30 \frac{\text{frames}}{\text{s}} \times (720 \times 480) \frac{\text{pixels}}{\text{frame}} \times 3 \frac{\text{bytes}}{\text{pixel}} = 31.104.000 \frac{\text{bytes}}{\text{s}}$$

Para um filme de **2 horas**, temos:

$$31.104.000 \frac{\text{bytes}}{\text{s}} \times (60^2) \text{ s} \times 2 \approx 2,24 \times 10^{11} \text{ bytes}$$

ou aproximadamente **224 GB** de dados.

Esse cálculo mostra que mesmo um vídeo relativamente simples, em resolução padrão e sem qualquer compressão, pode ocupar centenas de gigabytes de armazenamento. Na prática, armazenar ou transmitir esse volume de dados seria inviável para a maioria das aplicações, especialmente em sistemas de streaming, dispositivos móveis e transmissão em redes de comunicação.

A **compressão de imagens e vídeos** surge justamente para reduzir essa enorme quantidade de dados. Os algoritmos de compressão exploram diferentes tipos de **redundância presentes nas imagens**, como redundância espacial, temporal e estatística, permitindo representar a mesma informação visual com muito menos bits.

Graças a técnicas de compressão, formatos como **JPEG, MPEG e H.264** conseguem reduzir drasticamente o tamanho dos arquivos, tornando possível armazenar, transmitir e processar imagens e vídeos de forma eficiente em sistemas computacionais.

A compressão tem como objetivo principal reduzir a quantidade de dados necessária para representar uma imagem, preservando, tanto quanto possível, sua qualidade visual e informacional. Este capítulo apresenta os conceitos fundamentais da compressão de imagens, seus principais métodos e aplicações.

5.1 Conceitos Básicos de Compressão

O **modelo genérico de compressão de imagens** consiste em um processo utilizado para **reduzir o tamanho físico dos dados de uma imagem**, com o objetivo de **otimizar sua transmissão e armazenamento**. Imagens digitais geralmente ocupam uma quantidade significativa de memória, principalmente quando possuem alta resolução ou profundidade de bits. Dessa forma, técnicas de compressão tornam-se essenciais em diversas aplicações, como transmissão de imagens pela internet, armazenamento em dispositivos digitais e sistemas multimídia.

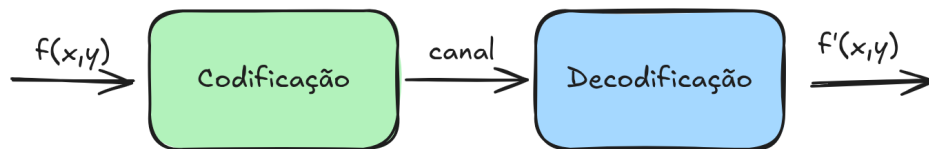


Figura 5.1: Modelo genérico de compressão de imagens

O processo de compressão normalmente envolve **duas fases principais: a codificação e a decodificação** (conforme ilustrada na Figura 5.1). Na fase de **codificação**, a imagem original é analisada e transformada em uma representação mais compacta, reduzindo a quantidade de bits necessária para armazenar ou transmitir os dados. Esse processo pode envolver diferentes estratégias, como remoção de redundâncias, transformação dos dados ou eliminação de informações consideradas menos relevantes para a percepção visual humana. Já na fase de **decodificação**, os dados comprimidos são processados para reconstruir a imagem, produzindo uma versão que pode ser idêntica à original ou apresentar pequenas diferenças, dependendo do método de compressão utilizado.

A ideia fundamental por trás da compressão é **explorar padrões e redundâncias presentes na imagem**. Em muitas imagens, determinados valores ou padrões de pixels aparecem com maior frequência que outros. Assim, é possível representar esses elementos mais frequentes utilizando **uma quantidade menor de bits**, enquanto elementos menos frequentes podem ser representados com códigos maiores. Esse princípio está presente em diversos métodos de compressão, como os esquemas de codificação estatística, nos quais símbolos mais frequentes recebem códigos mais curtos.

De maneira geral, os métodos de compressão podem ser classificados em **compressão sem perdas (lossless)** e **compressão com perdas (lossy)**. Na compressão sem perdas, a ima-

gem reconstruída após a decodificação é **exatamente igual à imagem original**, preservando todas as informações. Já na compressão com perdas, algumas informações são descartadas durante o processo de codificação, permitindo alcançar taxas de compressão maiores, porém introduzindo pequenas distorções na imagem reconstruída. A escolha do método depende da aplicação e do nível de fidelidade desejado.

5.1.1 Redundância de Dados

A **redundância de dados** é um conceito central na compressão de imagens. Ela mede quanto de informação adicional (ou repetitiva) existe em uma representação de dados em comparação com outra representação mais eficiente.

Considere duas formas de codificar o mesmo conjunto de dados:

- n_1 : espaço necessário para armazenar os dados na **representação original**;
- n_2 : espaço necessário para armazenar os dados na **representação comprimida**.

Define-se a **taxa de compressão** (*Compression Ratio*), denotada por CR , como:

$$CR = \frac{n_1}{n_2}$$

Essa medida indica quantas vezes o tamanho original é maior do que o tamanho após a compressão.

A **redundância de dados** (RD) pode então ser definida como:

$$RD = 1 - \frac{1}{CR}$$

ou, de forma equivalente,

$$RD = 1 - \frac{n_2}{n_1}$$

5.1.2 Interpretação

A redundância indica a **fração de dados que pode ser eliminada sem perda de informação essencial**. Quanto maior for a redundância presente nos dados, maior é o potencial de compressão.

Por exemplo, se um método de compressão produz uma taxa de compressão

$$CR = 4$$

então a redundância é

$$RD = 1 - \frac{1}{4} = 0.75$$

ou seja, **75% dos dados eram redundantes** e puderam ser removidos na compressão.

Em imagens digitais, a redundância pode aparecer de diferentes formas, como **redundância espacial**, **redundância espectral** e **redundância temporal** (no caso de vídeos). Os algoritmos de compressão exploram essas redundâncias para representar a informação visual de forma muito mais eficiente.

5.2 Tipos de Redundância em Imagens

A eficiência da compressão está diretamente relacionada à identificação e eliminação de redundâncias. Em imagens digitais, destacam-se três tipos principais:

5.2.1 Redundância Espacial

A redundância espacial decorre da alta correlação entre pixels vizinhos. Em regiões homogêneas, pixels adjacentes tendem a apresentar valores de intensidade semelhantes, permitindo que essa informação seja representada de forma mais compacta.

Muitas imagens apresentam uma característica importante: **pixels vizinhos frequentemente possuem valores iguais ou muito semelhantes**. Essa propriedade ocorre principalmente em regiões homogêneas da imagem, como áreas de fundo ou superfícies uniformes. Explorar essa característica permite reduzir a quantidade de dados necessária para representar a imagem.

5.2.2 Redundância Estatística

A redundância estatística está associada à distribuição não uniforme dos níveis de intensidade. Alguns valores ocorrem com maior frequência do que outros, possibilitando o uso de códigos mais curtos para símbolos mais prováveis.

Uma das formas mais simples de reduzir o tamanho de uma imagem digital é explorar a **redundância de codificação**. Esse tipo de redundância ocorre quando diferentes símbolos (por exemplo, valores de pixels) são representados por códigos com o **mesmo número de bits**, mesmo que alguns símbolos apareçam com muito mais frequência do que outros.

Considere a tabela apresentada, que mostra a quantidade de ocorrências de cada valor de pixel em uma imagem. Os valores variam de 1 a 8 e cada um aparece um número diferente de vezes na imagem.

No esquema **Cód. 1**, todos os valores de pixel são representados por códigos de **3 bits**, pois existem oito possíveis valores. Nesse caso, a codificação é fixa:

Pixel	Qtd.	Cód. 1	Bits	Cód. 2	Bits
1	19	000	57	11	38
2	25	001	75	01	50
3	21	010	63	10	42
4	16	011	48	001	48
5	8	100	24	0001	32
6	6	101	18	00001	30
7	3	110	9	000001	18
8	2	111	6	000000	12
Total			300		270

Como cada ocorrência utiliza 3 bits, o número total de bits necessários para representar todos os pixels da imagem é 300 bits.

Entretanto, observando a coluna **Qtd.**, percebe-se que alguns valores de pixel aparecem com muito mais frequência que outros. Por exemplo, os pixels **2, 3 e 1** aparecem muitas vezes, enquanto os valores **7 e 8** são raros. Essa diferença de frequência pode ser explorada para reduzir o tamanho da representação.

No esquema **Cód. 2**, são atribuídos **códigos mais curtos para os valores mais frequentes** e **códigos mais longos para os valores menos frequentes**. Por exemplo:

- pixels frequentes recebem códigos curtos, como 11 ou 10
- pixels menos frequentes recebem códigos mais longos, como 000001

Dessa forma, os símbolos que aparecem muitas vezes na imagem utilizam menos bits em média. Quando multiplicamos o número de ocorrências de cada pixel pelo tamanho do seu código, obtemos um total de 270 bits.

Assim, a **taxa de compressão** é

$$CR = \frac{300}{270} \approx 1.11$$

e a **redundância de dados** é

$$RD = 1 - \frac{1}{CR} \approx 0.099$$

ou seja, aproximadamente **9,9% dos bits eram redundantes** e puderam ser eliminados com uma codificação mais eficiente.

Esse princípio é a base de diversos métodos de compressão, como a **codificação de Huffman**, que constrói automaticamente códigos de comprimento variável de acordo com a frequência de ocorrência dos símbolos na imagem.

5.2.3 Redundância Psicovisual

A redundância psicovisual explora limitações do sistema visual humano. Certas informações podem ser descartadas ou representadas de forma aproximada sem causar impacto perceptível significativo na qualidade da imagem.



(a) 256 tons de cinza

(b) 16 tons de cinza

(c) 4 tons de cinza

Figura 5.2: Redundância psicovisual: a mesma imagem representada com **256, 16 e 4 tons de cinza**.

O sistema visual humano não é igualmente sensível a todas as variações de intensidade presentes em uma imagem. Pequenas diferenças entre níveis de cinza muitas vezes não são percebidas pelo observador.

Essa característica é chamada de **redundância psicovisual** e pode ser explorada em algoritmos de compressão de imagens. A ideia consiste em remover ou reduzir informações que possuem pouca relevância para a percepção humana, diminuindo a quantidade de dados necessária para representar a imagem.

A Figura 5.2 mostra a mesma imagem codificada com diferentes números de níveis de cinza: 256, 16 e 4 níveis. Observa-se que, mesmo com uma redução significativa no número de níveis disponíveis, a estrutura global da imagem ainda permanece reconhecível. Isso demonstra que parte da informação original pode ser descartada sem comprometer de forma significativa a percepção visual.

Essa propriedade é amplamente explorada em métodos de compressão com perda (*lossy compression*), como os utilizados em formatos de imagem e vídeo amplamente difundidos.

5.3 Compressão sem Perda

Na **compressão sem perda**, a imagem reconstruída após a descompressão é idêntica à imagem original. Esse tipo de compressão é essencial em aplicações onde a fidelidade absoluta dos dados é crítica, como imagens médicas, científicas e técnicas.

Métodos comuns de compressão sem perda incluem codificação por comprimento de execução, codificação preditiva e codificação entropia, como os algoritmos de Huffman e aritmético.

5.4 Compressão com Perda

A **compressão com perda** permite pequenas distorções na imagem reconstruída em troca de taxas de compressão significativamente maiores. Esse tipo de compressão é amplamente utilizado em aplicações multimídia, onde a percepção visual é mais importante do que a exatidão pixel a pixel.

Técnicas com perda geralmente envolvem transformações da imagem para domínios mais adequados, como o domínio da frequência, seguidas de processos de quantização que descartam informações menos relevantes perceptualmente.

5.4.1 Critério de Fidelidade

Em muitos métodos de compressão de imagens, especialmente nos métodos **com perda** (**lossy**), a imagem reconstruída pode não ser exatamente igual à imagem original. Por isso, é necessário definir **critérios de fidelidade** que permitam medir o quanto a imagem comprimida difere da imagem original.

Sejam:

- $f(x, y)$ a **imagem original**
- $f'(x, y)$ a **imagem reconstruída após a compressão**

Uma maneira simples de avaliar a diferença entre as duas imagens é calcular o **erro total**, definido como a soma das diferenças entre os valores de pixels correspondentes:

$$e_T = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f'(x, y) - f(x, y)]$$

Entretanto, essa medida pode não representar adequadamente a magnitude real do erro. Por isso, uma métrica mais utilizada é o **erro médio quadrático** (*Mean Squared Error – MSE*) ou sua raiz, conhecida como **erro raiz da média quadrática** (*Root Mean Square Error – RMSE*).

O erro raiz da média quadrática é dado por:

$$e_{MQ} = \sqrt{\frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f'(x, y) - f(x, y)]^2}$$

onde M e N representam as dimensões da imagem.

Essa medida fornece uma estimativa da **diferença média entre os pixels da imagem original e da imagem reconstruída**. Quanto menor o valor de e_{MQ} , maior será a fidelidade da imagem comprimida em relação à imagem original.

Outra métrica amplamente utilizada na avaliação de qualidade de imagens comprimidas é a **PSNR** (*Peak Signal-to-Noise Ratio*), que mede a relação entre o valor máximo possível do sinal da imagem e o erro introduzido pela compressão.

A PSNR é definida como:

$$PSNR = 10 \log_{10} \left(\frac{L^2}{MSE} \right)$$

onde:

- L é o **valor máximo possível do nível de intensidade** da imagem
- MSE é o **erro médio quadrático**

Para imagens de **8 bits**, o valor máximo de intensidade é

$$L = 255$$

e o MSE é calculado como:

$$MSE = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f'(x, y) - f(x, y)]^2$$

Valores mais altos de **PSNR** indicam maior similaridade entre a imagem original e a imagem reconstruída. Em muitas aplicações práticas, valores de PSNR entre **30 dB e 50 dB** são considerados aceitáveis, dependendo do tipo de imagem e da aplicação. (observação: O resultado da PSNR é dado em decibéis (dB) porque envolve uma relação logarítmica entre sinais, semelhante ao que ocorre em medidas de potência em telecomunicações.)

Apesar da utilidade dessas métricas matemáticas, elas nem sempre refletem perfeitamente a qualidade percebida pelo observador. Em muitos casos, **a avaliação visual humana continua sendo o critério mais importante**, pois o sistema visual humano possui sensibilidades diferentes para contraste, textura e detalhes da imagem.

5.5 Transformações Utilizadas na Compressão

Diversos métodos de compressão utilizam transformações matemáticas para concentrar a energia da imagem em um pequeno conjunto de coeficientes. Entre as mais utilizadas destacam-se:

- Transformada Discreta do Cosseno (DCT);
- Transformada de Fourier;
- Transformada Wavelet.

Essas transformações facilitam a identificação de componentes menos significativas, que podem ser quantizadas de forma mais agressiva.

5.6 Codificação Huffman

A codificação entropia é uma etapa essencial em muitos sistemas de compressão. Ela visa representar os dados de forma eficiente, utilizando códigos de comprimento variável baseados na probabilidade de ocorrência dos símbolos.

Algoritmos clássicos incluem a codificação de Huffman, que utiliza árvores binárias para atribuir códigos mais curtos a símbolos mais frequentes, e a codificação aritmética, que alcança taxas de compressão próximas ao limite teórico da entropia.

O **algoritmo de Huffman** é um método de compressão de dados amplamente utilizado baseado em **codificação estatística sem perdas**. Ele foi proposto por David A. Huffman em 1952 e tem como objetivo reduzir o número médio de bits necessários para representar um conjunto de símbolos. Esse algoritmo explora a frequência de ocorrência dos símbolos, atribuindo códigos menores aos símbolos mais frequentes e códigos maiores aos símbolos menos frequentes, resultando em uma representação mais eficiente dos dados.

A ideia central da codificação de Huffman é construir um **código de comprimento variável**, no qual cada símbolo é representado por uma sequência de bits cujo tamanho depende de sua probabilidade de ocorrência. Diferentemente dos códigos de comprimento fixo, em que todos os símbolos são representados pelo mesmo número de bits, a codificação de Huffman permite que símbolos mais comuns utilizem menos bits, reduzindo o tamanho total da informação codificada.

O processo de construção do código de Huffman inicia-se com a **determinação da frequência de ocorrência de cada símbolo** presente nos dados (conforme ilustrado na Figura 5.3). Em seguida, esses símbolos são organizados em uma estrutura conhecida como **árvore binária de Huffman**. Inicialmente, cada símbolo é representado como um nó da árvore com um peso correspondente à sua frequência. O algoritmo então combina repetidamente os dois nós de menor frequência, criando um novo nó cujo peso é a soma dos dois anteriores (iterações $i = 1, 2, 3$ e 4 na Figura 5.3). Esse processo continua até que todos os nós estejam conectados em uma única árvore.

Após a construção da árvore, os códigos binários são obtidos percorrendo-se os caminhos da raiz até cada folha da árvore. Normalmente, atribui-se o valor **0 para um ramo à esquerda** e **1 para um ramo à direita**. Assim, cada símbolo recebe um código binário correspondente ao caminho percorrido até ele. Uma característica importante desses códigos é que eles são **códigos prefixos**, ou seja, nenhum código é prefixo de outro. Isso garante que a sequência de bits possa ser decodificada de forma inequívoca.

O código na Listagem 5.1, apresenta de forma simplificada os passos do algoritmo de Huffman, onde n é o número de folhas, portanto o número de nós internos será $n - 1$. A cada iteração é preenchido um novo nó interno da árvore, onde os filhos $p1$ (menor frequência) e $p2$ (segunda menor frequência) são colocados à esquerda e direita do novo nó pai p e são removidos da fila de prioridades (ou lista, onde os nós são ordenados pela frequência). O novo nó p armazena a soma das frequências de $p1$ e $p2$ e é inserido na fila de prioridades. Para facilitar o percurso da folha para raiz, cada filho é ligado ao pai ($node[p1].father = p$).

Na fase de **codificação**, cada símbolo do conjunto de dados é substituído pelo código binário correspondente definido pela árvore de Huffman. Já na fase de **decodificação**, a sequência de bits é interpretada percorrendo a árvore binária desde a raiz até alcançar um nó folha,

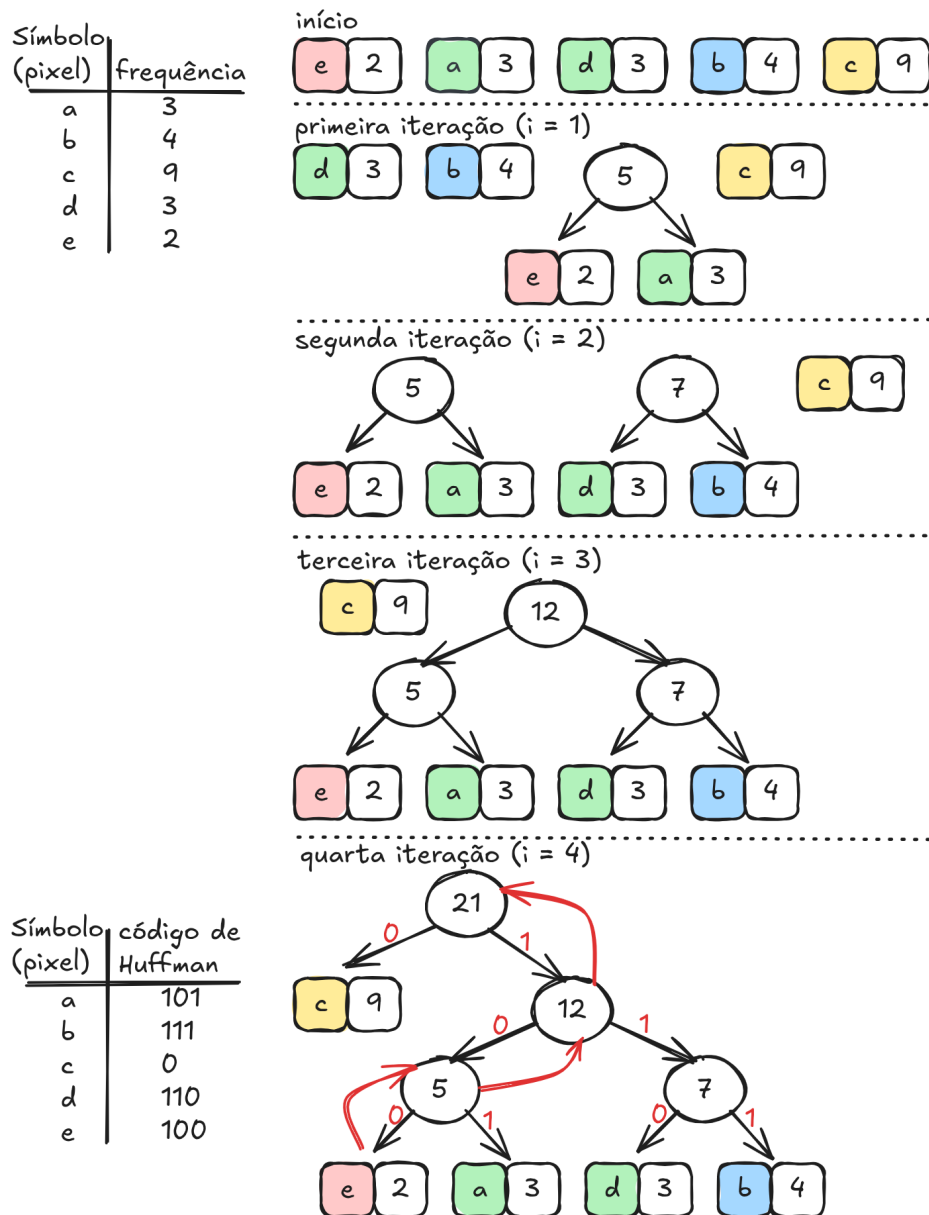


Figura 5.3: Iterações para construção de uma árvore de Huffman

Listagem 5.1

```
for (int i = 0; i < n-1; i++)
{
    p = n + i;
    p1 = pqMinDel(&root);
    p2 = pqMinDel(&root);
    node[p1].father = p;
    node[p2].father = p;
    node[p].freq = node[p1].freq + node[p2].freq;
    node[p].left = p1;
    node[p].right = p2;
    pqIns(&root, p);
}
```

identificando o símbolo correspondente. Esse processo é repetido até que toda a sequência codificada seja decodificada.

A codificação de Huffman é amplamente utilizada em diversos padrões e sistemas de compressão de dados, incluindo formatos de imagem, áudio e vídeo. Por ser um método **sem perdas**, ele garante que os dados reconstruídos após a decodificação sejam **idênticos aos dados originais**, tornando-o adequado para aplicações nas quais a preservação exata da informação é essencial. Em sistemas de compressão de imagens, o algoritmo de Huffman frequentemente aparece como uma etapa final de codificação, após processos de transformação e quantização dos dados.

A simulação em sequência apresenta a árvore construída a partir das frequências. No exemplo, as frequências 5, 9, 12, 13, 16 e 45, separados por espaço, são as frequências dos pixels (símbolos) 1, 2, 3, 4, 5 e 6, respectivamente.

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/RbVDL-7-g>



5.7 Codificação LZW

A **codificação de Lempel–Ziv–Welch (LZW)** é uma técnica de compressão de dados **sem perda** amplamente utilizada em arquivos e formatos de imagem, como GIF e TIFF. O método foi proposto por **Terry Welch** em 1984 como uma melhoria das técnicas de compressão desenvolvidas anteriormente por **Abraham Lempel** e **Jacob Ziv**. A ideia central da LZW é substituir sequências de símbolos por **códigos numéricos de tamanho fixo**, reduzindo a quantidade de dados necessária para representar a informação original.

Diferentemente de outros métodos de compressão estatísticos, como a codificação de Huffman, a técnica LZW **não requer conhecimento prévio das probabilidades de ocorrência dos símbolos da fonte**. Em vez disso, ela constrói dinamicamente um **dicionário de sequências de símbolos** durante o processo de compressão. Inicialmente, esse dicionário contém apenas os símbolos básicos do alfabeto da fonte (por exemplo, os valores possíveis de pixels ou caracteres). À medida que a sequência de entrada é analisada, novas combinações de símbolos são identificadas e adicionadas ao dicionário, cada uma associada a uma palavra-código.

O processo de compressão ocorre da seguinte forma. O algoritmo percorre a sequência de entrada procurando a **maior sequência de símbolos que já esteja presente no dicionário**. Quando essa sequência é encontrada, o algoritmo emite o código correspondente e acrescenta ao dicionário uma nova entrada formada pela sequência reconhecida seguida do próximo símbolo da entrada. Dessa forma, o dicionário cresce progressivamente, passando a representar padrões mais longos e frequentes presentes nos dados.

Uma característica importante da LZW é que **tanto o codificador quanto o decodificador constroem o mesmo dicionário de forma incremental**, seguindo exatamente as mesmas regras. Por esse motivo, **não é necessário transmitir o dicionário junto com os dados comprimidos**, o que contribui para a eficiência do método. Durante a descompressão, os códigos recebidos são utilizados para reconstruir as sequências originais, enquanto o dicionário é atualizado da mesma forma que ocorreu na compressão.

```
LZW_Encode(T)
{
    Dictionary D;
    int next;

    /* inicializa o dicionário com todos os símbolos */
    for each symbol a in alphabet  $\Sigma$ 
        D.insert(a, code(a));

    next = D.size();

    string w = "";
    List output;
```

```

for each symbol k in T
{
    if (D.contains(w + k))
    {
        w = w + k;
    }
    else
    {
        output.append(D[w]);
        D.insert(w + k, next);
        next = next + 1;
        w = k;
    }
}

if (w != "")
    output.append(D[w]);

return output;
}

```

Outra vantagem significativa da LZW é que **a compressão e a descompressão podem ser realizadas em uma única passada sobre os dados**. Isso significa que o algoritmo processa a informação sequencialmente, sem necessidade de múltiplas leituras ou análises prévias do conteúdo. Essa propriedade torna a técnica particularmente adequada para aplicações em tempo real ou em sistemas com recursos limitados.

```

LZW_Decode(C)
{
    Dictionary D;
    int next;

    /* inicializa o dicionário com todos os símbolos */
    for each symbol a in alphabet  $\Sigma$ 
        D.insert(code(a), a);

    next = D.size();

    int c = first(C);
    string w = D[c];
    string output = w;
}

```

```

for each code k in remaining(C)
{
    string entry;

    if (D.contains(k))
        entry = D[k];
    else
        entry = w + first(w);

    output = output + entry;

    D.insert(next, w + first(entry));
    next = next + 1;

    w = entry;
}

return output;
}

```

Além disso, o algoritmo apresenta **implementação relativamente simples e execução rápida**, pois baseia-se principalmente em operações de busca e inserção em um dicionário. Como consequência, a LZW tornou-se uma das técnicas de compressão mais utilizadas em aplicações práticas, especialmente em sistemas de armazenamento e transmissão de imagens e documentos digitais.

Em resumo, a codificação LZW destaca-se por combinar **simplicidade, eficiência e independência de conhecimento estatístico prévio**, utilizando um dicionário adaptativo para representar padrões repetidos nos dados e reduzir o volume de informação necessário para armazená-los ou transmiti-los.

💡 Dica

O código completo e a demonstração interativa estão disponíveis em:

https://editor.p5js.org/luizedsilva/full/FQDBrI9D_



5.8 Padrões de Compressão de Imagens

Ao longo das últimas décadas, diversos padrões de compressão de imagens digitais foram desenvolvidos com o objetivo de melhorar a eficiência de armazenamento e transmissão de dados visuais, ao mesmo tempo em que garantem interoperabilidade entre diferentes dispositivos e sistemas. A padronização desses métodos é fundamental para que imagens possam ser produzidas, armazenadas e visualizadas em diferentes plataformas sem perda de compatibilidade. Esses padrões buscam equilibrar fatores como taxa de compressão, qualidade visual da imagem reconstruída e complexidade computacional dos algoritmos envolvidos no processo de codificação e decodificação.

Entre os padrões mais conhecidos destaca-se o JPEG (Joint Photographic Experts Group), amplamente utilizado para compressão de fotografias e imagens com grande variação de tons e cores. O método JPEG baseia-se na Transformada Discreta do Cosseno (DCT) aplicada a blocos da imagem e emprega técnicas de quantização e codificação entropia para reduzir a quantidade de dados necessária para representar a imagem. Embora o processo de compressão seja geralmente com perda de informação, ele permite alcançar altas taxas de compressão com degradação visual relativamente pequena, o que explica sua ampla utilização em câmeras digitais, páginas da web e sistemas de armazenamento de imagens.

Outro padrão bastante difundido é o PNG (Portable Network Graphics), que utiliza compressão sem perda de informação. Diferentemente do JPEG, o PNG preserva exatamente os valores originais dos pixels, sendo especialmente adequado para imagens que contêm texto, gráficos, diagramas ou áreas de cores uniformes. A compressão no formato PNG baseia-se em técnicas de predição e codificação entropia, permitindo reduzir o tamanho do arquivo sem comprometer a fidelidade da imagem original.

O padrão JPEG2000 foi desenvolvido como uma evolução do JPEG tradicional, introduzindo uma abordagem baseada na transformada wavelet em vez da DCT aplicada a blocos. Essa técnica permite uma representação mais eficiente da imagem em múltiplas resoluções e pode oferecer melhor qualidade visual em altas taxas de compressão. Além disso, o JPEG2000 apresenta recursos adicionais, como codificação progressiva e melhor tratamento de regiões de interesse. Entretanto, sua maior complexidade computacional e menor adoção em dispositivos e navegadores limitaram sua popularização em comparação ao JPEG convencional.

Mais recentemente, o formato WebP foi desenvolvido com foco em aplicações na web, buscando reduzir o tamanho dos arquivos de imagem sem comprometer significativamente a qualidade visual. O WebP combina técnicas modernas de compressão, oferecendo suporte tanto para compressão com perda quanto sem perda, além de permitir transparência e animação. Devido à sua eficiência na redução do tamanho dos arquivos, esse formato tem sido amplamente adotado em aplicações web, contribuindo para o carregamento mais rápido de páginas e a economia de largura de banda.

Cada um desses padrões apresenta características específicas que os tornam mais adequados para determinados tipos de aplicação. Enquanto alguns priorizam altas taxas de compressão

com pequenas perdas perceptuais, outros enfatizam a preservação exata da informação original. Assim, a escolha do formato de compressão depende do equilíbrio desejado entre qualidade visual, eficiência de armazenamento e complexidade computacional do processo de codificação e decodificação.

5.8.1 Formato JPEG

As etapas da codificação JPEG estão representadas na Figura 5.4.

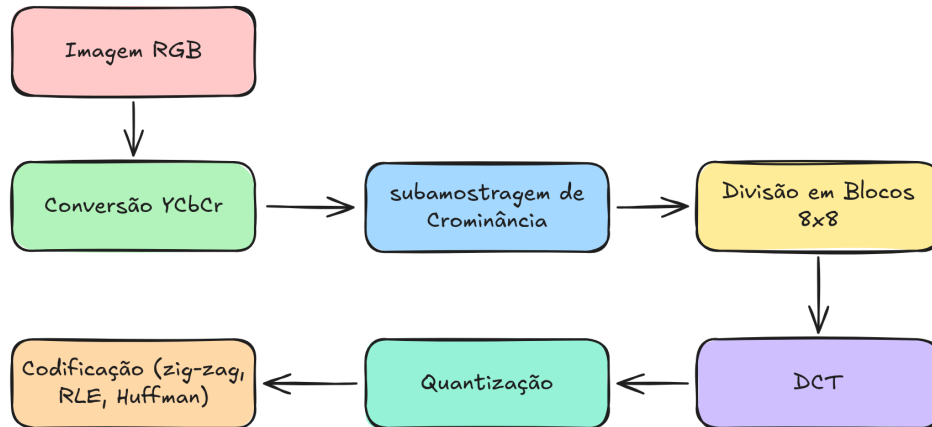


Figura 5.4: Etapas da codificação JPEG

Conversão YCbCr

A primeira etapa do processo de compressão JPEG consiste na conversão da imagem do espaço de cores RGB para o espaço de cores YCbCr. No formato RGB, cada pixel é representado pelas intensidades das três componentes de cor primárias: vermelho (R), verde (G) e azul (B). Embora esse modelo seja adequado para dispositivos de exibição, ele não é o mais apropriado para compressão, pois mistura informações de brilho e de cor. Para tornar o processo de compressão mais eficiente, o JPEG realiza uma transformação que separa essas duas informações.

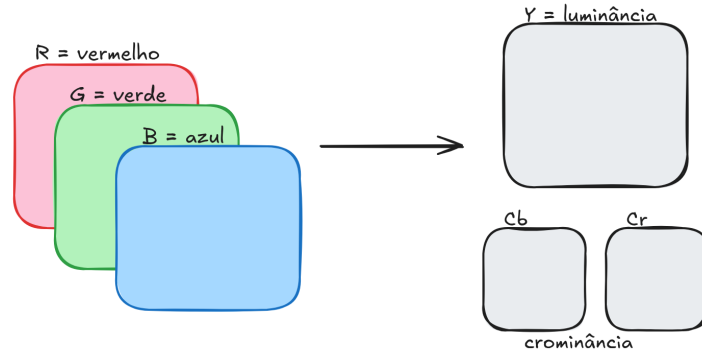


Figura 5.5: Conversão RGB para YCbCr

No espaço de cores YCbCr, cada pixel passa a ser descrito por três componentes distintas: luminância (Y) e duas componentes de crominância (Cb e Cr). A componente Y representa o nível de brilho da imagem, ou seja, a informação de intensidade luminosa que define os contornos e os detalhes estruturais da cena. As componentes Cb e Cr representam as informações de cor, correspondendo aproximadamente às diferenças entre o azul e a luminância, e entre o vermelho e a luminância, respectivamente. Essa separação é fundamental porque o sistema visual humano é muito mais sensível às variações de luminância do que às variações de cor.

A conversão de RGB para YCbCr é realizada por meio de uma transformação linear aplicada a cada pixel da imagem. Uma forma comum dessa transformação é dada pelas seguintes expressões:

$$Y = 0.299R + 0.587G + 0.114B$$

$$Cb = -0.1687R - 0.3313G + 0.5B + 128$$

$$Cr = 0.5R - 0.4187G - 0.0813B + 128$$

Nessas equações, observa-se que a componente de luminância é uma combinação ponderada das três cores primárias, com maior peso para o verde, refletindo a maior sensibilidade do olho humano a essa cor. As componentes de crominância recebem um deslocamento de 128 unidades para que seus valores permaneçam dentro do intervalo usual de representação digital de imagens, normalmente entre 0 e 255.

A principal vantagem dessa transformação é permitir que as componentes de crominância sejam representadas com menor resolução espacial do que a luminância, sem causar degradação perceptível significativa. Como o olho humano percebe mais detalhes de brilho do que de cor, o algoritmo JPEG pode reduzir a quantidade de dados das componentes Cb e Cr por meio de

um processo conhecido como subamostragem de crominância. Essa estratégia reduz o volume de informação que precisa ser codificada, contribuindo de forma significativa para a eficiência da compressão.

Após a conversão para o espaço YCbCr, cada componente da imagem é tratada separadamente nas etapas seguintes do algoritmo JPEG. Em geral, a componente de luminância preserva maior quantidade de detalhes, enquanto as componentes de crominância podem sofrer maior redução de informação. Essa etapa inicial de transformação de cores, portanto, prepara os dados da imagem para as fases posteriores do processo de compressão, que incluem a divisão da imagem em blocos, a aplicação da Transformada Discreta do Cosseno (DCT), a quantização e a codificação dos coeficientes.

Transformada do Cosseno Discreto (DCT)

A Transformada Discreta do Cosseno (DCT) é uma das etapas centrais do algoritmo de compressão JPEG. Após a conversão da imagem para o espaço de cores YCbCr e a divisão da imagem em blocos de 8×8 pixels, cada bloco é transformado do domínio espacial para o domínio da frequência. Essa transformação permite representar a informação da imagem em termos de diferentes componentes de frequência, separando as variações lentas (baixas frequências) das variações rápidas (altas frequências) da intensidade luminosa.

A DCT utilizada no JPEG é a chamada **DCT bidimensional**, aplicada a cada bloco de 8×8 . Essa transformação calcula um conjunto de coeficientes que representam a contribuição das diferentes frequências presentes no bloco. O coeficiente localizado na posição $(0, 0)$ corresponde à componente de frequência zero, também chamada de **coeficiente DC**, que representa o valor médio do bloco. Os demais coeficientes são chamados de **coeficientes AC** e descrevem variações espaciais da intensidade, correspondendo a diferentes padrões de frequência horizontal e vertical.

A expressão matemática da DCT bidimensional aplicada a um bloco de imagem pode ser escrita como:

$$F(u, v) = \frac{1}{4}C(u)C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos\left(\frac{(2x+1)u\pi}{16}\right) \cos\left(\frac{(2y+1)v\pi}{16}\right)$$

Nessa equação, $f(x, y)$ representa o valor do pixel localizado na posição (x, y) do bloco original, enquanto $F(u, v)$ representa o coeficiente transformado correspondente às frequências horizontal u e vertical v . Os índices x e y percorrem as posições do bloco de imagem no domínio espacial, enquanto u e v indicam as diferentes frequências no domínio transformado.

Os termos $(C(u))$ e $(C(v))$ são fatores de normalização utilizados para garantir a ortogonalidade da transformação e são definidos por:

$$C(k) = \begin{cases} \frac{1}{\sqrt{2}}, & k = 0 \\ 1, & k > 0 \end{cases}$$

Esses fatores ajustam a amplitude do coeficiente DC em relação aos coeficientes AC. O fator adicional $\frac{1}{4}$ presente na equação decorre da normalização da transformada para blocos de dimensão 8×8 .

A principal finalidade da DCT no JPEG é **concentrar a maior parte da energia da imagem em poucos coeficientes de baixa frequência**. Em imagens naturais, as variações de intensidade costumam ser suaves, o que faz com que os coeficientes associados às baixas frequências tenham valores maiores, enquanto muitos coeficientes de alta frequência tendem a assumir valores pequenos ou próximos de zero. Essa propriedade é fundamental para a etapa seguinte do algoritmo, a **quantização**, na qual os coeficientes de menor importância perceptual podem ser reduzidos ou eliminados, permitindo uma compressão significativa da imagem.

Após a aplicação da DCT, cada bloco de 8×8 pixels é transformado em uma matriz de 8×8 coeficientes de frequência. Esses coeficientes passam então pelo processo de quantização e, posteriormente, pelas etapas de reorganização em zig-zag e codificação entrópica. Dessa forma, a DCT desempenha um papel essencial no JPEG ao converter a informação da imagem para um formato no qual a redundância pode ser explorada de maneira eficiente pelo processo de compressão.

Quantização

Após a aplicação da Transformada Discreta do Cosseno (DCT), cada bloco de 8×8 pixels da imagem passa a ser representado por um conjunto de coeficientes de frequência. Esses coeficientes indicam a contribuição das diferentes frequências espaciais presentes no bloco. A etapa seguinte do algoritmo JPEG é a **quantização**, responsável por reduzir a precisão desses coeficientes e, conseqüentemente, diminuir a quantidade de informação necessária para representar a imagem.

A quantização é a principal etapa responsável pela perda de informação no processo de compressão JPEG. Nessa fase, os coeficientes obtidos pela DCT são divididos por valores previamente definidos em uma matriz de quantização. Após essa divisão, os resultados são arredondados para o inteiro mais próximo. Esse processo reduz a precisão dos coeficientes, especialmente aqueles associados às altas frequências, que têm menor impacto perceptual na qualidade visual da imagem.

Matematicamente, o processo de quantização pode ser expresso por:

$$F_Q(u, v) = \text{round} \left(\frac{F(u, v)}{k \cdot Q(u, v)} \right)$$

Nessa equação, $F(u, v)$ representa o coeficiente obtido pela DCT na posição (u, v) , enquanto $F_Q(u, v)$ corresponde ao coeficiente quantizado. A matriz $Q(u, v)$ é chamada de **matriz de quantização** e define o peso aplicado a cada coeficiente de frequência. O parâmetro k é um fator de compressão que controla o grau de redução aplicado aos coeficientes: valores maiores de k produzem maior compressão, porém com maior perda de qualidade na imagem reconstruída.

O padrão JPEG não impõe uma única matriz de quantização obrigatória. O comitê responsável pela padronização permitiu que diferentes matrizes fossem utilizadas, possibilitando ajustes conforme a aplicação desejada. Entretanto, juntamente com o padrão foram publicadas matrizes recomendadas, obtidas a partir de diversos experimentos com imagens reais. Essas matrizes foram projetadas de modo a explorar as características do sistema visual humano, aplicando quantização mais agressiva nas frequências altas, que são menos perceptíveis ao observador.

De modo geral, os elementos da matriz de quantização aumentam à medida que se afastam da posição $(0, 0)$, correspondente às baixas frequências. Isso significa que os coeficientes associados às altas frequências são divididos por valores maiores, tornando-se frequentemente iguais a zero após o arredondamento. Esse comportamento é fundamental para aumentar a eficiência das etapas posteriores de compressão, pois sequências de zeros podem ser representadas de forma muito compacta durante a codificação.

Uma família de matrizes de quantização pode ser gerada de forma paramétrica, permitindo controlar a intensidade da compressão por meio de um fator ajustável. Uma forma simples de definir essa família de matrizes é dada por:

$$Q(i, j) = 1 + [(i + j) \cdot \text{fator}], \quad i, j = 0, 1, \dots, N - 1$$

Nesse modelo, o valor da matriz cresce gradualmente à medida que se aumenta a distância da origem da matriz, representando o aumento da penalização aplicada às frequências mais altas. O parâmetro **fator** controla a intensidade dessa penalização e, portanto, influencia diretamente a taxa de compressão obtida.

Após a quantização, a matriz de coeficientes resultante contém muitos valores nulos, especialmente nas regiões correspondentes às altas frequências. Essa característica é explorada nas etapas seguintes do algoritmo JPEG, nas quais os coeficientes são reorganizados em ordem de zig-zag e codificados por métodos de compressão entrópica, como a codificação de Huffman. Dessa forma, a quantização desempenha um papel fundamental no JPEG ao reduzir a redundância perceptual da imagem e possibilitar taxas de compressão elevadas com perda visual relativamente pequena.

Varredura zig-zag

Após a etapa de quantização, cada bloco da imagem é representado por uma matriz de 8×8 coeficientes quantizados da DCT. Esses coeficientes ainda estão organizados na forma bidimensional correspondente às frequências horizontal e vertical. No entanto, para facilitar as etapas seguintes de compressão, o algoritmo JPEG reorganiza esses coeficientes em uma sequência unidimensional por meio de um processo conhecido como **varredura em zig-zag**.

A operação de zig-zag consiste em percorrer a matriz de coeficientes seguindo um padrão diagonal específico que começa no canto superior esquerdo da matriz e termina no canto inferior direito. Esse percurso é projetado de modo que os coeficientes de **baixa frequência**, localizados próximos da posição $(0,0)$, sejam visitados primeiro, enquanto os coeficientes de **alta frequência**, que normalmente aparecem nas regiões inferiores e à direita da matriz, sejam visitados posteriormente. Como resultado, a matriz bidimensional de coeficientes é convertida em um vetor de 64 elementos.

A principal motivação para esse tipo de ordenação está relacionada às propriedades estatísticas dos coeficientes após a quantização. Em imagens naturais, a maior parte da energia do sinal está concentrada nas baixas frequências. Assim, os primeiros coeficientes da matriz tendem a possuir valores relativamente grandes, enquanto muitos coeficientes de alta frequência tornam-se iguais a zero após a quantização. Ao organizar os coeficientes seguindo o padrão em zig-zag, o algoritmo agrupa os valores mais significativos no início da sequência e concentra longas sequências de zeros na parte final.

Essa reorganização é particularmente importante para as etapas seguintes de compressão. Após a varredura em zig-zag, os coeficientes são codificados utilizando técnicas de compressão entrópica, como a **codificação por comprimento de sequência (Run-Length Encoding – RLE)** combinada com **codificação de Huffman**. Como a varredura em zig-zag tende a produzir longas sequências consecutivas de zeros, essas sequências podem ser representadas de forma muito eficiente, reduzindo significativamente o número de bits necessários para armazenar o bloco.

Outro aspecto relevante é a distinção entre o primeiro coeficiente da sequência e os demais. O primeiro elemento da sequência corresponde ao coeficiente DC do bloco, que representa o valor médio da luminância no bloco. Esse coeficiente é tratado de forma especial na codificação JPEG, sendo codificado de maneira diferencial em relação ao coeficiente DC do bloco anterior. Os demais coeficientes, chamados de coeficientes AC, são codificados utilizando pares que indicam o número de zeros consecutivos e o valor do próximo coeficiente diferente de zero.

Dessa forma, a operação de zig-zag desempenha um papel fundamental no processo de compressão JPEG, pois reorganiza os coeficientes de frequência de maneira a explorar a concentração de energia nas baixas frequências e a presença de longas sequências de zeros nas altas frequências. Essa reorganização aumenta a eficiência das técnicas de codificação utilizadas nas etapas subsequentes, contribuindo para a obtenção de elevadas taxas de compressão com boa qualidade visual da imagem reconstruída.

Codificação da Corrida da Cadeia

Após a reorganização dos coeficientes quantizados por meio da varredura em zig-zag, obtém-se uma sequência unidimensional de 64 valores para cada bloco de 8×8 . Nessa sequência, o primeiro elemento corresponde ao coeficiente DC e os demais representam os coeficientes AC. Como resultado da quantização, muitos coeficientes de alta frequência tornam-se iguais a zero, especialmente na parte final da sequência. Para explorar essa característica e reduzir ainda mais a quantidade de dados, o algoritmo JPEG utiliza a técnica de **Run-Length Encoding (RLE)**, ou codificação por comprimento de sequência.

A codificação RLE consiste em representar sequências consecutivas de valores repetidos de forma compacta, armazenando apenas o valor repetido e o número de vezes que ele ocorre consecutivamente. No contexto do JPEG, essa técnica é aplicada principalmente às sequências de zeros presentes entre os coeficientes AC. Em vez de armazenar explicitamente cada zero, o algoritmo registra quantos zeros aparecem antes do próximo coeficiente diferente de zero.

Na prática, cada coeficiente AC diferente de zero é representado por um par de valores da forma $((r, v))$, onde (r) indica o número de zeros consecutivos que precedem o coeficiente e (v) corresponde ao valor do coeficiente diferente de zero. Por exemplo, considere a seguinte sequência de coeficientes após a varredura em zig-zag:

$$[15, 0, 0, -3, 0, 0, 0, 2]$$

Nesse caso, a codificação RLE produziria os pares:

$$(0, 15), (2, -3), (3, 2)$$

O primeiro par indica que o coeficiente 15 aparece imediatamente, sem zeros anteriores. O segundo par indica que existem dois zeros antes do coeficiente -3 , e o terceiro par indica três zeros antes do coeficiente 2.

Além disso, o padrão JPEG define símbolos especiais para lidar com casos particulares. Quando restam apenas zeros até o final do bloco, utiliza-se um símbolo denominado **End of Block (EOB)**, que indica que todos os coeficientes restantes são iguais a zero. Esse mecanismo evita a necessidade de representar explicitamente todos os zeros finais. Outro símbolo especial é o **Zero Run Length (ZRL)**, utilizado quando ocorre uma sequência de 16 zeros consecutivos.

Após a aplicação da RLE, os pares gerados ainda não constituem a forma final comprimida. Esses pares são então codificados por meio de **codificação de Huffman**, que atribui códigos binários mais curtos aos padrões mais frequentes. Dessa forma, a combinação entre varredura em zig-zag, codificação RLE e codificação de Huffman permite explorar tanto a redundância estrutural quanto a redundância estatística dos coeficientes da DCT, contribuindo significativamente para a eficiência do algoritmo de compressão JPEG.

Codificação Huffman

Após a aplicação da varredura em zig-zag e da codificação por comprimento de sequência (Run-Length Encoding – RLE), os coeficientes do bloco passam a ser representados por uma sequência de símbolos que descrevem tanto os valores dos coeficientes quanto a quantidade de zeros que os precedem. Entretanto, esses símbolos ainda podem ser representados de forma mais eficiente por meio de técnicas de compressão estatística. Para esse propósito, o padrão JPEG utiliza a **codificação de Huffman**, um método de codificação entrópica que atribui códigos binários de diferentes comprimentos aos símbolos de acordo com sua frequência de ocorrência.

A codificação de Huffman baseia-se no princípio de que símbolos que aparecem com maior frequência devem receber códigos binários mais curtos, enquanto símbolos menos frequentes recebem códigos mais longos. Dessa forma, a representação média dos dados tende a utilizar menos bits, reduzindo o tamanho final do arquivo comprimido. No JPEG, os códigos de Huffman são aplicados tanto aos coeficientes DC quanto aos coeficientes AC, porém de maneiras ligeiramente diferentes.

Para os **coeficientes DC**, o algoritmo JPEG utiliza codificação diferencial. Em vez de codificar diretamente o valor do coeficiente DC de cada bloco, o sistema codifica a diferença entre o coeficiente DC atual e o coeficiente DC do bloco anterior. Como essas diferenças tendem a assumir valores pequenos, sua representação se torna mais eficiente. Primeiramente determina-se a **categoria** do valor da diferença, que corresponde ao número de bits necessários para representar sua magnitude. Essa categoria é então codificada utilizando uma tabela de Huffman específica para coeficientes DC. Em seguida, são acrescentados os bits que representam a magnitude do valor.

Para os **coeficientes AC**, a codificação utiliza os pares gerados pela etapa de RLE. Cada símbolo é representado por um par (r, s) , onde r indica o número de zeros consecutivos antes do coeficiente e s representa a categoria (ou tamanho em bits) do coeficiente diferente de zero. Esse par (r, s) é codificado utilizando uma tabela de Huffman específica para coeficientes AC. Após o código de Huffman correspondente ao par, são acrescentados os bits que representam a magnitude do coeficiente.

O padrão JPEG define também alguns símbolos especiais utilizados durante a codificação dos coeficientes AC. Um deles é o **End of Block (EOB)** que representa o par $(0,0)$, que indica que todos os coeficientes restantes do bloco são iguais a zero. Outro símbolo é o **Zero Run Length (ZRL)**, que representa uma sequência de 16 zeros consecutivos $(0,15)$. Esses símbolos contribuem para aumentar a eficiência da compressão, pois sequências longas de zeros são muito comuns após a quantização.

Embora o padrão JPEG forneça tabelas de Huffman padrão, ele também permite que tabelas personalizadas sejam utilizadas. Essas tabelas podem ser geradas a partir das estatísticas de uma imagem ou de um conjunto de imagens, permitindo otimizar a codificação para determinados tipos de conteúdo. Entretanto, na prática, muitas implementações utilizam as tabelas

padrão (luminância e croma) devido à sua simplicidade e boa eficiência em uma ampla variedade de imagens.

Dessa forma, a codificação de Huffman constitui a etapa final do processo de compressão JPEG, responsável por transformar os símbolos gerados nas etapas anteriores em uma sequência compacta de bits. Ao explorar as probabilidades de ocorrência dos diferentes símbolos, essa técnica reduz ainda mais o tamanho dos dados, contribuindo para a alta eficiência do formato JPEG na compressão de imagens digitais.

O exemplo interativo seguinte demonstra as etapas de codificação para um bloco 8×8 . Com o botão “Passo”, as transformações do bloco 8×8 é modificado para apresentar o resultado da etapa realizada (deslocamento, DCT, Quantização, zig-zag, RLE e Huffman). As etapas zig-zag, RLE e Huffman são apresentadas concomitantemente. São apresentados a contagem de zeros antes de cada coeficiente DCT e para cada par (run, coeficiente), são mostrados a direita do bloco a diferença $DC_i - DC_{i-1}$ (considerasse $DC_{i-1} = -17$ nesta simulação) para codificação do coeficiente DC e para todos os coeficientes são mostrados a categoria, o prefixo e a mantissa consultadas nas tabelas DC e AC de Huffman:

Dica

O código completo e a demonstração interativa estão disponíveis em:

<https://editor.p5js.org/luizedsilva/full/crAHLJNHf>



5.8.2 Formato Base-64

Em algumas situações, dados binários precisam ser transmitidos ou armazenados em sistemas que foram projetados para manipular apenas **texto**. Nesses casos utiliza-se um mecanismo de codificação conhecido como **Base-64**. Embora essa codificação aumente o tamanho dos dados, ela permite representar informações binárias utilizando apenas **caracteres ASCII imprimíveis**, evitando problemas de interpretação ou corrupção durante a transmissão.

Esse tipo de codificação é frequentemente utilizado em protocolos de comunicação baseados em texto, como o SMTP, amplamente empregado para envio de correio eletrônico. Também é comum em formatos de troca de dados como JSON e XML. Nesses ambientes, a presença de bytes arbitrários pode causar problemas, pois determinados valores binários podem ser interpretados como **caracteres de controle**, responsáveis por indicar comandos especiais, quebras de linha ou final de mensagem. Assim, ao converter os dados binários para um

conjunto limitado de caracteres seguros, a codificação Base-64 garante que as informações possam ser transmitidas corretamente nesses sistemas.

O princípio da codificação Base-64 consiste em representar os dados binários utilizando um conjunto de **64 símbolos diferentes**. Cada um desses símbolos corresponde a um valor codificado em **6 bits**, pois $2^6 = 64$. Dessa forma, cada caractere da codificação Base-64 representa exatamente um grupo de seis bits da sequência original de dados.

O processo de codificação ocorre agrupando inicialmente os dados binários em blocos de **3 bytes**, o que corresponde a **24 bits**. Em seguida, esses 24 bits são reorganizados em **quatro grupos de 6 bits**. Cada um desses grupos é então convertido em um caractere da tabela Base-64. Como resultado, três bytes da sequência original passam a ser representados por quatro caracteres codificados. Isso implica um aumento aproximado de **33% no tamanho dos dados**, já que 24 bits passam a ser representados por 32 bits.

A tabela de conversão da Base-64 é formada pelos caracteres A–Z, a–z, 0–9, além dos símbolos + e /. Quando o número total de bytes do arquivo não é múltiplo de três, caracteres de preenchimento (=) são utilizados para completar a codificação final.

Tabela de Conversão

Valor	Caractere	Valor	Caractere
0–25	A–Z	52–61	0–9
26–51	a–z	62	+
63	/		

Para ilustrar o processo, considere a sequência de caracteres “**Man**”. Cada caractere é inicialmente convertido para sua representação binária em ASCII. Assim, os caracteres *M*, *a* e *n* correspondem respectivamente às sequências 01001101, 01100001 e 01101110. Ao concatenar esses valores obtemos uma sequência de 24 bits. Essa sequência é então dividida em quatro grupos de 6 bits, resultando nos valores 010011, 010110, 000101 e 101110. Convertendo esses grupos para seus valores decimais correspondentes e consultando a tabela Base-64, obtêm-se os caracteres T, W, F e u. Dessa forma, a sequência “**Man**” é representada na codificação Base-64 como “**TWFu**”.

A codificação Base-64 é amplamente utilizada em diversas aplicações computacionais. Entre suas aplicações mais comuns estão o envio de anexos em mensagens de correio eletrônico, a incorporação de imagens diretamente em documentos HTML ou CSS, o armazenamento de dados binários em estruturas JSON e a transmissão de informações em interfaces de programação de aplicações (APIs) baseadas em texto. Apesar do aumento no tamanho dos dados, essa técnica garante que informações binárias possam ser transportadas com segurança em ambientes que suportam apenas conteúdo textual.

Imagem dentro de HTML:

```

```

5.9 Considerações Finais

A compressão de imagens é uma área central do Processamento Digital de Imagens, possibilitando o armazenamento e a transmissão eficientes de grandes volumes de dados visuais. A escolha do método de compressão adequado depende da aplicação, do nível de fidelidade requerido e das restrições computacionais.

Os conceitos apresentados neste capítulo fornecem a base necessária para o estudo de técnicas modernas de compressão e codificação multimídia, bem como para a compreensão dos principais padrões utilizados na prática.

6 Imagens Coloridas

A maioria das informações visuais percebidas pelo ser humano envolve cor. Em Processamento Digital de Imagens, o uso da cor amplia significativamente a capacidade de análise, interpretação e representação de cenas, tornando sistemas computacionais mais próximos da percepção humana.

Este capítulo apresenta os fundamentos das **imagens coloridas**, abordando os princípios da percepção da cor, os principais modelos de cor utilizados em sistemas computacionais e as formas de representação e manipulação de imagens coloridas.

6.1 Percepção da Cor

A percepção da cor é resultado da interação da luz com o sistema visual humano. A luz visível corresponde a uma faixa restrita do espectro eletromagnético e, ao atingir a retina, estimula diferentes tipos de fotorreceptores.

O olho humano possui três tipos principais de cones, cada um sensível a uma faixa específica de comprimentos de onda, aproximadamente associada às cores vermelha, verde e azul. A combinação das respostas desses cones permite a percepção de uma grande variedade de cores.

Esse mecanismo fisiológico fundamenta muitos dos modelos de cor utilizados em Processamento de Imagens.

6.2 Conceitos Básicos de Cor

Uma cor pode ser descrita por atributos perceptuais fundamentais, tais como:

- **Matiz (hue):** corresponde à tonalidade dominante da cor;
- **Saturação:** indica o grau de pureza da cor;
- **Brilho ou intensidade:** representa a quantidade de luz associada à cor.

Esses atributos são utilizados para definir modelos de cor que facilitam a representação e o processamento computacional de imagens coloridas.

6.3 Modelo de Cor RGB

O modelo **RGB (Red, Green, Blue)** é o mais utilizado em dispositivos de aquisição e exibição de imagens, como câmeras, monitores e scanners. Nesse modelo, cada cor é representada como uma combinação aditiva das componentes vermelha, verde e azul.

Uma imagem colorida RGB pode ser representada por três matrizes bidimensionais, correspondentes aos canais R, G e B. Cada pixel é descrito por um vetor tridimensional:

$$\mathbf{c}(x, y) = [R(x, y), G(x, y), B(x, y)]$$

O modelo RGB é intuitivo e eficiente para dispositivos de hardware, mas não é sempre o mais adequado para tarefas de análise de imagens.

6.4 Modelo de Cor CMY e CMYK

Os modelos **CMY (Cyan, Magenta, Yellow)** e **CMYK (Cyan, Magenta, Yellow, Black)** são amplamente utilizados em sistemas de impressão. Diferentemente do RGB, esses modelos são baseados na mistura **subtrativa** de cores.

A conversão entre RGB e CMY pode ser expressa de forma simples, considerando valores normalizados:

$$C = 1 - R, \quad M = 1 - G, \quad Y = 1 - B$$

O canal adicional K no modelo CMYK é utilizado para melhorar o contraste e reduzir o consumo de tinta.

6.5 Modelo de Cor HSV

O modelo **HSV (Hue, Saturation, Value)** foi desenvolvido para representar cores de forma mais próxima à percepção humana. Nesse modelo:

- H representa a matiz;
- S representa a saturação;
- V representa o valor ou brilho.

O modelo HSV é particularmente útil em aplicações de segmentação e reconhecimento, pois permite separar informações de cor da intensidade luminosa.

6.6 Modelo de Cor HSI

O modelo **HSI (Hue, Saturation, Intensity)** é semelhante ao HSV, mas utiliza a intensidade como componente separada. Essa característica o torna adequado para aplicações em que se deseja processar a intensidade independentemente da cor.

Em muitos algoritmos de processamento, operações são realizadas apenas sobre o canal de intensidade, preservando as informações cromáticas.

6.7 Representação de Imagens Coloridas

Imagens coloridas podem ser representadas de diferentes formas, dependendo do modelo de cor adotado. Em geral, uma imagem colorida pode ser vista como um conjunto de imagens monocromáticas correlacionadas.

Essa representação permite que técnicas clássicas de processamento em tons de cinza sejam aplicadas individualmente aos canais de cor ou a componentes específicas do modelo adotado.

6.8 Aplicações de Imagens Coloridas

O uso de imagens coloridas é essencial em diversas aplicações, tais como:

- segmentação baseada em cor;
- reconhecimento de objetos;
- análise de cenas naturais;
- visão computacional em robótica;
- processamento de imagens médicas e industriais.

A escolha adequada do modelo de cor é determinante para o sucesso dessas aplicações.

6.9 Considerações Finais

As imagens coloridas desempenham um papel central no Processamento Digital de Imagens moderno. A compreensão dos modelos de cor e de suas características é fundamental para o desenvolvimento de sistemas eficientes e robustos.

O domínio desses conceitos permite selecionar representações adequadas e explorar de forma eficaz as informações cromáticas presentes nas imagens, ampliando as possibilidades de análise e interpretação.

7 Morfologia Matemática Binária

A **Morfologia Matemática** é uma área do Processamento Digital de Imagens dedicada à análise e ao processamento da forma geométrica dos objetos presentes em uma imagem. Diferentemente de técnicas baseadas diretamente em valores de intensidade, a morfologia foca na **estrutura espacial** dos objetos, sendo especialmente eficaz no tratamento de imagens binárias.

Este capítulo apresenta os fundamentos da **morfologia matemática binária**, introduzindo seus principais operadores e discutindo suas aplicações em tarefas como filtragem, segmentação e análise de formas.

7.1 Imagens Binárias

Uma imagem binária é uma imagem digital na qual cada pixel pode assumir apenas dois valores possíveis, geralmente representados por 0 e 1. Esses valores indicam, respectivamente, a ausência ou a presença de um objeto em determinada posição da imagem.

Imagens binárias são frequentemente obtidas a partir de imagens em tons de cinza por meio de processos de limiarização. Sua simplicidade torna-as adequadas para a aplicação de operações morfológicas, nas quais a forma e a conectividade dos objetos são os principais aspectos analisados.

7.2 Conceitos Fundamentais da Morfologia Matemática

A morfologia matemática baseia-se na teoria dos conjuntos. Uma imagem binária pode ser interpretada como um conjunto de pontos no plano bidimensional, correspondentes aos pixels de valor 1.

As operações morfológicas envolvem dois conjuntos principais:

- **Imagem:** o conjunto que representa os objetos de interesse;
- **Elemento estruturante:** um pequeno conjunto que define a forma e a escala da operação.

O elemento estruturante atua como uma sonda que percorre a imagem, permitindo analisar e modificar suas estruturas geométricas.

7.3 Elemento Estruturante

O **elemento estruturante** é um componente central da morfologia matemática. Ele pode assumir diferentes formas, como linhas, quadrados, discos ou cruzes, e possui um ponto de origem que define sua posição relativa durante as operações.

A escolha do elemento estruturante influencia diretamente o resultado das operações morfológicas, pois determina quais características geométricas serão preservadas, realçadas ou removidas.

7.4 Dilatação

A **dilatação** é uma operação morfológica que tende a expandir as regiões dos objetos em uma imagem binária. Formalmente, a dilatação de um conjunto (A) por um elemento estruturante (B) é definida como:

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\}$$

onde $\hat{B}$ representa a reflexão do elemento estruturante (B).

A dilatação é frequentemente utilizada para preencher pequenas lacunas, conectar componentes próximos e suavizar contornos internos.

7.5 Erosão

A **erosão** é a operação dual da dilatação e tem como efeito principal a redução das regiões dos objetos. A erosão de um conjunto (A) por um elemento estruturante (B) é definida como:

$$A \ominus B = \{z \mid B_z \subseteq A\}$$

Essa operação é útil para remover pequenas protuberâncias, separar objetos conectados por regiões estreitas e eliminar ruídos isolados.

7.6 Abertura

A **abertura** é uma operação composta, definida como a erosão seguida de uma dilatação utilizando o mesmo elemento estruturante:

$$A \circ B = (A \ominus B) \oplus B$$

A abertura é utilizada para remover objetos pequenos e detalhes finos, preservando a forma geral das estruturas maiores presentes na imagem.

7.7 Fechamento

O **fechamento** é a operação dual da abertura, definida como a dilatação seguida de uma erosão:

$$A \bullet B = (A \oplus B) \ominus B$$

Essa operação é eficaz para preencher pequenas lacunas e buracos nos objetos, além de suavizar contornos externos.

7.8 Propriedades das Operações Morfológicas

As operações morfológicas apresentam propriedades importantes, como:

- **Idempotência:** a aplicação repetida de abertura ou fechamento não altera o resultado após a primeira aplicação;
- **Dualidade:** erosão e dilatação, assim como abertura e fechamento, são operações duais;
- **Invariância por translação:** o resultado das operações não depende da posição absoluta dos objetos na imagem.

Essas propriedades fornecem uma base teórica sólida para o uso da morfologia matemática em sistemas de processamento de imagens.

7.9 Aplicações da Morfologia Binária

A morfologia matemática binária é amplamente utilizada em diversas aplicações, tais como:

- remoção de ruído impulsivo;
- preenchimento de regiões;
- extração de contornos;
- análise de conectividade;
- pré-processamento para segmentação e reconhecimento de padrões.

Sua simplicidade conceitual e eficiência computacional fazem da morfologia uma ferramenta essencial em PDI.

7.10 Considerações Finais

A morfologia matemática binária fornece um conjunto poderoso de operadores para a análise e manipulação da forma dos objetos em imagens digitais. A compreensão de seus princípios e operações fundamentais é essencial para o desenvolvimento de soluções robustas em Processamento Digital de Imagens.

Os conceitos apresentados neste capítulo servem como base para extensões mais avançadas, como a morfologia em imagens em tons de cinza e aplicações em visão computacional.

8 Morfologia Matemática em Tons de Cinza

A **Morfologia Matemática em tons de cinza** estende os conceitos da morfologia binária para imagens cujos pixels assumem valores de intensidade em um intervalo contínuo ou discreto mais amplo. Enquanto a morfologia binária opera sobre conjuntos, a morfologia em tons de cinza trabalha diretamente com **funções de intensidade**, permitindo a análise e o processamento da forma considerando variações graduais de brilho.

Este capítulo apresenta os fundamentos teóricos e operacionais da morfologia matemática em tons de cinza, destacando seus principais operadores e aplicações em processamento e análise de imagens.

8.1 Imagens em Tons de Cinza

Uma imagem em tons de cinza pode ser modelada como uma função bidimensional ($f(x,y)$), na qual cada ponto $((x,y))$ assume um valor de intensidade pertencente a um conjunto ordenado, normalmente $\{0,1,\dots,L-1\}$. Essa representação permite descrever variações suaves de brilho, essenciais para a análise de superfícies, texturas e estruturas naturais.

Diferentemente das imagens binárias, as operações morfológicas em tons de cinza consideram **relações de ordem** entre intensidades, e não apenas a presença ou ausência de pixels.

8.2 Fundamentos da Morfologia em Tons de Cinza

A morfologia matemática em tons de cinza baseia-se em operadores de **máximo** e **mínimo** aplicados localmente à imagem, utilizando um **elemento estruturante** que também pode ser definido como uma função.

Seja (f) a imagem de entrada e (b) um elemento estruturante. As operações morfológicas são definidas de forma a preservar e analisar a geometria das superfícies de intensidade da imagem.

8.3 Elemento Estruturante

Na morfologia em tons de cinza, o elemento estruturante pode ser:

- **Plano (flat)**: assume valor constante, geralmente zero;
- **Não plano (non-flat)**: definido por uma função com valores variados.

Elementos estruturantes planos são os mais utilizados na prática, pois simplificam as operações e facilitam a interpretação dos resultados.

8.4 Dilatação em Tons de Cinza

A **dilatação** em tons de cinza tende a realçar regiões claras e expandir máximos locais. Para um elemento estruturante plano, a dilatação é definida como:

$$(f \oplus b)(x, y) = \max_{(s, t) \in b} f(x - s, y - t)$$

Essa operação eleva os valores de intensidade nas vizinhanças definidas pelo elemento estruturante, sendo útil para realçar detalhes claros e preencher vales rasos.

8.5 Erosão em Tons de Cinza

A **erosão** em tons de cinza é a operação dual da dilatação e tem como efeito principal a atenuação de regiões claras e a expansão de mínimos locais. Para um elemento estruturante plano, a erosão é definida como:

$$(f \ominus b)(x, y) = \min_{(s, t) \in b} f(x + s, y + t)$$

Essa operação é utilizada para eliminar picos isolados e reduzir variações abruptas de intensidade.

8.6 Abertura em Tons de Cinza

A **abertura** em tons de cinza é definida como a erosão seguida de uma dilatação:

$$f \circ b = (f \ominus b) \oplus b$$

A abertura suaviza a imagem, removendo picos brilhantes menores que o elemento estruturante, enquanto preserva a forma geral das regiões maiores.

8.7 Fechamento em Tons de Cinza

O **fechamento** é a operação dual da abertura, definido como:

$$f \bullet b = (f \oplus b) \ominus b$$

Essa operação é eficaz para preencher vales escuros e suavizar descontinuidades negativas na superfície de intensidade.

8.8 Propriedades das Operações Morfológicas

As operações morfológicas em tons de cinza compartilham diversas propriedades com suas contrapartes binárias, tais como:

- **Dualidade** entre erosão e dilatação;
- **Idempotência** da abertura e do fechamento;
- **Invariância por translação**;
- **Extensividade** da dilatação e **antiextensividade** da erosão.

Essas propriedades garantem previsibilidade e consistência no comportamento dos operadores morfológicos.

8.9 Aplicações da Morfologia em Tons de Cinza

A morfologia matemática em tons de cinza é amplamente utilizada em aplicações como:

- realce e suavização de imagens;
- supressão de ruído impulsivo;
- extração de estruturas relevantes;
- pré-processamento para segmentação;
- análise de superfícies e texturas.

Sua capacidade de preservar características geométricas torna-a especialmente útil em imagens naturais e médicas.

8.10 Considerações Finais

A morfologia matemática em tons de cinza amplia significativamente o alcance das técnicas morfológicas, permitindo o processamento direto de imagens com variações graduais de intensidade. A compreensão de seus operadores fundamentais é essencial para o desenvolvimento de métodos avançados de análise e processamento de imagens.

Os conceitos apresentados neste capítulo estabelecem a base para aplicações mais complexas, incluindo morfologia multiescala e transformações morfológicas avançadas.

9 Segmentação de Imagens

A **segmentação de imagens** é uma das etapas mais importantes do Processamento Digital de Imagens, pois tem como objetivo particionar uma imagem em regiões ou objetos que sejam significativos para uma determinada aplicação. Diferentemente do melhoramento, cujo foco é a aparência visual, a segmentação busca **identificar e separar estruturas relevantes**, servindo como base para tarefas posteriores como reconhecimento, descrição e interpretação.

O sucesso de um sistema de visão computacional está fortemente relacionado à qualidade da segmentação realizada. Por essa razão, a segmentação é considerada uma das tarefas mais desafiadoras da área.

9.1 Conceito de Segmentação

Segmentar uma imagem consiste em dividi-la em subconjuntos disjuntos, de modo que cada região seja homogênea segundo um critério pré-definido, como intensidade, cor, textura ou forma. Em termos formais, a segmentação deve satisfazer propriedades como:

- cada pixel pertence a uma única região;
- regiões adjacentes são distintas segundo o critério adotado;
- a união de todas as regiões reconstrói a imagem original.

Esses critérios variam conforme o domínio da aplicação e as características da imagem.

9.2 Abordagens Gerais para Segmentação

As técnicas de segmentação podem ser agrupadas em duas grandes categorias:

- **segmentação baseada em descontinuidades**, que explora mudanças abruptas nos valores de intensidade;
- **segmentação baseada em similaridades**, que agrupa pixels com características semelhantes.

Cada abordagem apresenta vantagens e limitações, sendo comum a combinação de múltiplas técnicas em aplicações práticas.

9.3 Segmentação Baseada em Descontinuidades

As técnicas baseadas em descontinuidades identificam pontos, linhas ou bordas na imagem. A presença de uma borda geralmente indica uma transição entre regiões distintas.

9.3.1 Detecção de Bordas

A detecção de bordas baseia-se na identificação de variações significativas de intensidade. Operadores diferenciais, como gradientes e Laplacianos, são amplamente utilizados para esse fim.

O resultado desse processo é uma imagem que destaca as bordas dos objetos, que podem ser posteriormente utilizadas para definir regiões.

9.3.2 Operadores de Detecção

Diversos operadores foram propostos para detecção de bordas, incluindo operadores clássicos como Roberts, Prewitt e Sobel. Esses operadores utilizam máscaras convolucionais para estimar derivadas espaciais da imagem.

9.4 Segmentação Baseada em Similaridades

Na segmentação baseada em similaridades, pixels com características semelhantes são agrupados em regiões. Essa abordagem assume que objetos de interesse apresentam propriedades relativamente homogêneas.

9.4.1 Limiarização

A **limiarização** é uma das técnicas mais simples e utilizadas de segmentação. Ela consiste em classificar pixels com base em um ou mais valores de limiar.

Em sua forma mais simples, um único limiar (T) é utilizado:

$$g(x, y) = \begin{cases} 1, & f(x, y) \geq T \\ 0, & f(x, y) < T \end{cases}$$

A escolha adequada do limiar é crucial para o sucesso da segmentação.

9.4.2 Limiarização Global e Adaptativa

Na limiarização global, um único valor de limiar é aplicado a toda a imagem. Já na limiarização adaptativa, o limiar varia conforme regiões locais da imagem, sendo mais adequada para imagens com iluminação não uniforme.

9.5 Segmentação por Crescimento de Regiões

O crescimento de regiões inicia-se a partir de pixels-semente e expande as regiões incorporando pixels vizinhos que satisfaçam critérios de similaridade.

Essa técnica é intuitiva e permite bom controle sobre a segmentação, mas depende fortemente da escolha das sementes iniciais.

9.6 Segmentação por Divisão e Fusão

A segmentação por divisão e fusão combina duas estratégias complementares. Inicialmente, a imagem é dividida em regiões menores até que um critério de homogeneidade seja satisfeito. Em seguida, regiões adjacentes semelhantes são fundidas.

Esse método é frequentemente implementado por meio de estruturas hierárquicas, como árvores quadtree.

9.7 Segmentação Baseada em Morfologia

Operações morfológicas podem ser utilizadas para auxiliar na segmentação, especialmente em imagens binárias ou em tons de cinza. Técnicas como abertura, fechamento e reconstrução morfológica permitem remover ruídos e separar objetos conectados.

A morfologia matemática é particularmente útil como etapa de pré-processamento ou pós-processamento em sistemas de segmentação.

9.8 Desafios da Segmentação

Apesar dos inúmeros métodos existentes, a segmentação continua sendo um problema aberto em muitas aplicações. Fatores como ruído, variações de iluminação, oclusões e diversidade de formas tornam difícil a definição de critérios universais.

Por esse motivo, a segmentação é frequentemente ajustada ao problema específico e integrada a outras etapas do processamento de imagens.

9.9 Considerações Finais

A segmentação de imagens constitui um passo essencial para a interpretação automática de cenas. A escolha da técnica adequada depende das características da imagem e dos objetivos da aplicação.

Os métodos apresentados neste capítulo fornecem uma base conceitual para o desenvolvimento de sistemas de segmentação robustos, que podem ser combinados com técnicas avançadas de aprendizado de máquina e visão computacional.

10 Representação e Descrição de Imagens

Após a segmentação, o próximo passo em um sistema de Processamento Digital de Imagens consiste em **representar** e **descrever** as regiões ou objetos identificados. Enquanto a segmentação define *onde* estão os objetos, a representação e a descrição determinam *como* esses objetos serão caracterizados para análise posterior, classificação ou reconhecimento.

Este capítulo aborda os conceitos fundamentais de representação e descrição de imagens, apresentando técnicas clássicas utilizadas para extrair características relevantes de regiões segmentadas.

10.1 Representação de Regiões

A representação de uma região tem como objetivo codificar sua forma e estrutura de maneira adequada ao processamento computacional. Uma boa representação deve ser compacta, discriminativa e robusta a variações irrelevantes.

As técnicas de representação podem ser classificadas em duas categorias principais:

- **Representações por contorno**, que descrevem apenas a fronteira do objeto;
- **Representações por região**, que utilizam todos os pixels pertencentes ao objeto.

A escolha do tipo de representação depende da aplicação e das propriedades que se deseja analisar.

10.2 Representação por Contorno

Na representação por contorno, apenas os pixels que delimitam a região são considerados. Essa abordagem é eficiente quando a forma do objeto é o principal atributo de interesse.

10.2.1 Codificação por Cadeia

A **codificação por cadeia** descreve um contorno como uma sequência de direções discretas, geralmente baseadas na conectividade 4 ou 8. Cada segmento do contorno é representado por um símbolo que indica a direção do próximo pixel.

Essa técnica produz uma representação compacta e é particularmente útil para análise de forma, embora seja sensível a ruídos e pequenas variações no contorno.

10.2.2 Assinaturas de Forma

Uma assinatura de forma representa o contorno por meio de uma função unidimensional, geralmente baseada na distância entre pontos do contorno e um ponto de referência, como o centroide.

Assinaturas facilitam a comparação entre formas, mas podem perder informações locais importantes.

10.3 Representação por Região

As representações por região utilizam todos os pixels que compõem o objeto. Essa abordagem é mais adequada quando propriedades internas, como textura ou intensidade média, são relevantes.

Entre as representações por região mais comuns estão:

- matrizes de pixels;
- histogramas;
- momentos geométricos.

Essas representações tendem a ser mais robustas a ruídos locais no contorno.

10.4 Descrição de Regiões

A descrição de uma região consiste na extração de **atributos quantitativos**, também chamados de *features*, que caracterizam propriedades geométricas, topológicas ou estatísticas do objeto.

Esses atributos formam um vetor de características utilizado em etapas posteriores, como classificação ou reconhecimento de padrões.

10.5 Descritores Geométricos

Descritores geométricos caracterizam a forma e o tamanho das regiões. Exemplos comuns incluem:

- **área**, definida pelo número de pixels da região;
- **perímetro**, correspondente ao comprimento do contorno;
- **razão de aspecto**, que relaciona largura e altura;
- **circularidade**, que mede o quão próxima a forma está de um círculo.

Esses descritores são amplamente utilizados devido à sua simplicidade e interpretação intuitiva.

10.6 Momentos de Imagem

Os **momentos de imagem** fornecem uma descrição global da distribuição espacial dos pixels em uma região. O momento de ordem $((p,q))$ é definido como:

$$m_{pq} = \sum_x \sum_y x^p y^q f(x, y)$$

Momentos podem ser utilizados para calcular propriedades como área, centroide e orientação principal do objeto.

Momentos invariantes, como os **momentos de Hu**, são particularmente importantes por serem invariantes a transformações geométricas como translação, rotação e escala.

10.7 Descritores de Textura

Descritores de textura capturam padrões locais de variação de intensidade dentro de uma região. Esses descritores são úteis para diferenciar objetos com formas semelhantes, mas texturas distintas.

Entre os métodos clássicos de descrição de textura destacam-se matrizes de coocorrência, filtros estatísticos e medidas baseadas em frequência.

10.8 Normalização e Invariância

Para que descritores sejam eficazes em sistemas de reconhecimento, é desejável que sejam invariantes a transformações irrelevantes, como mudanças de posição, orientação ou escala.

Técnicas de normalização são frequentemente aplicadas para garantir comparabilidade entre objetos extraídos de imagens diferentes.

10.9 Aplicações de Representação e Descrição

A representação e a descrição de imagens são etapas fundamentais em aplicações como:

- reconhecimento de padrões;
- classificação de objetos;
- análise de formas;
- visão computacional em robótica;
- sistemas de inspeção automática.

A qualidade dos descritores influencia diretamente o desempenho dessas aplicações.

10.10 Considerações Finais

A representação e a descrição de imagens constituem o elo entre a segmentação e o reconhecimento de padrões. A escolha adequada de representações e descritores é essencial para capturar as características relevantes dos objetos e garantir o sucesso das etapas posteriores.

Os conceitos apresentados neste capítulo fornecem uma base sólida para o estudo de técnicas avançadas de análise de imagens e aprendizado de máquina aplicado à visão computacional.

11 Reconhecimento e Interpretação de Imagens Coloridas

A interpretação de imagens digitais é uma área central do processamento de imagens e da visão computacional, especialmente quando se trabalha com imagens coloridas. Diferentemente das imagens em tons de cinza, as imagens coloridas carregam informações adicionais que ampliam significativamente a capacidade de análise, reconhecimento e classificação de objetos e cenas.

Este capítulo apresenta os conceitos fundamentais envolvidos no reconhecimento e na interpretação de imagens coloridas, destacando a importância da cor como atributo informacional e discutindo suas aplicações práticas.

11.1 Imagens Digitais e Informação de Cor

Uma imagem digital pode ser entendida como uma matriz de pixels, em que cada elemento representa uma amostra espacial da cena original. No caso das imagens coloridas, cada pixel é composto por múltiplos valores que representam diferentes componentes de cor.

A cor não é apenas um elemento estético, mas uma característica essencial para distinguir objetos, identificar padrões e melhorar a robustez de algoritmos de análise visual. Em muitos casos, informações cromáticas permitem separar regiões que seriam indistinguíveis em imagens monocromáticas.

11.2 Modelos de Cor

Para representar imagens coloridas, utilizam-se modelos de cor, que organizam matematicamente as informações cromáticas. O modelo mais comum é o RGB, no qual cada pixel é descrito pelas componentes vermelho (Red), verde (Green) e azul (Blue). Esse modelo é amplamente utilizado em dispositivos de captura e exibição.

Outros modelos, como HSV e HSI, são frequentemente empregados em tarefas de interpretação, pois separam melhor atributos perceptuais como tonalidade, saturação e intensidade. Essa separação facilita operações como segmentação, detecção de objetos e análise semântica da imagem.

11.3 Reconhecimento de Padrões em Imagens Coloridas

O reconhecimento de padrões envolve a identificação automática de estruturas, objetos ou regiões de interesse em uma imagem. Quando se utilizam imagens coloridas, a cor passa a ser um atributo adicional no processo de decisão, juntamente com forma, textura e posição.

A combinação de informações espaciais e cromáticas aumenta a eficiência dos métodos de classificação, permitindo distinguir objetos semelhantes em forma, mas diferentes em cor, ou vice-versa.

11.4 Segmentação Baseada em Cor

A segmentação é uma etapa fundamental na interpretação de imagens, pois divide a imagem em regiões homogêneas. Em imagens coloridas, a segmentação baseada em cor é amplamente utilizada, explorando a similaridade entre pixels em determinados espaços de cor.

Essa abordagem é especialmente útil em aplicações como visão robótica, análise de cenas naturais, imagens médicas e sistemas de inspeção visual, onde a cor é um critério discriminante relevante.

11.5 Aplicações da Interpretação de Imagens Coloridas

Os conceitos discutidos neste capítulo são aplicados em diversas áreas, como: - sistemas de vigilância e monitoramento; - reconhecimento facial e biometria; - análise de imagens médicas; - visão computacional em robótica; - sensoriamento remoto e imagens de satélite.

Em todos esses contextos, a correta interpretação da informação de cor contribui para resultados mais precisos e confiáveis.

11.6 Considerações Finais

O uso de imagens coloridas amplia significativamente as possibilidades de análise e interpretação no processamento digital de imagens. A escolha adequada do modelo de cor, aliada a técnicas eficientes de reconhecimento e segmentação, é essencial para o sucesso das aplicações práticas.

Este capítulo consolidou os fundamentos necessários para compreender o papel da cor na interpretação de imagens, servindo como base para estudos mais avançados em visão computacional e análise de imagens.

12 Introdução ao OpenCV

O OpenCV (Open Source Computer Vision Library) é uma biblioteca amplamente utilizada para o desenvolvimento de aplicações em visão computacional e processamento digital de imagens. Criado originalmente pela Intel, o OpenCV é distribuído como software livre e oferece suporte a múltiplas linguagens, com grande destaque para Python e C++.

Este capítulo apresenta os conceitos fundamentais do OpenCV, sua estrutura básica e exemplos práticos de uso.

12.1 Visão Computacional e OpenCV

A visão computacional tem como objetivo permitir que sistemas computacionais interpretem informações visuais provenientes do mundo real. Para isso, utiliza técnicas de processamento de imagens, reconhecimento de padrões e aprendizado de máquina.

O OpenCV disponibiliza implementações eficientes desses algoritmos, facilitando o desenvolvimento de aplicações práticas.

12.2 Estrutura da Biblioteca OpenCV

Em Python, o OpenCV é acessado principalmente por meio do módulo `cv2`. Após a instalação, a biblioteca pode ser importada da seguinte forma:

```
import cv2
```

12.3 Leitura e Exibição de Imagens

A leitura de imagens é feita com a função `imread`, e a exibição com `imshow`.


```
import cv2

imagem = cv2.imread("imagem.jpg")
cv2.imshow("Imagem Original", imagem)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.4 Representação de Imagens no OpenCV

No OpenCV, imagens coloridas são representadas no formato **BGR** (Blue, Green, Red), e não RGB.

A conversão para tons de cinza pode ser feita da seguinte forma:

```
imagem_cinza = cv2.cvtColor(imagem, cv2.COLOR_BGR2GRAY)
cv2.imshow("Imagem em Cinza", imagem_cinza)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.5 Redimensionamento de Imagens

O redimensionamento é uma operação comum no processamento de imagens:

```
imagem_redimensionada = cv2.resize(imagem, (300, 200))
cv2.imshow("Imagem Redimensionada", imagem_redimensionada)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.6 Filtragem e Redução de Ruído

Filtros são usados para suavizar imagens e reduzir ruídos. Um dos mais comuns é o filtro Gaussiano:

```
imagem_suavizada = cv2.GaussianBlur(imagem, (5, 5), 0)
cv2.imshow("Imagem Suavizada", imagem_suavizada)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.7 Detecção de Bordas

A detecção de bordas é fundamental para a identificação de objetos. Um método amplamente utilizado é o detector de bordas de Canny:

```
bordas = cv2.Canny(imagem_cinza, 100, 200)
cv2.imshow("Bordas Detectadas", bordas)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.8 Limiarização (Threshold)

A limiarização converte uma imagem em tons de cinza para uma imagem binária:

```
_, imagem_binaria = cv2.threshold(imagem_cinza, 127, 255, cv2.THRESH_BINARY)
cv2.imshow("Imagem Binária", imagem_binaria)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

12.9 Aplicações Práticas do OpenCV

Com essas operações básicas, é possível desenvolver aplicações como:

- reconhecimento facial;
- sistemas de vigilância;
- análise de imagens médicas;
- visão computacional em robótica;
- processamento de vídeos em tempo real.

12.10 Considerações Finais

O OpenCV fornece uma base sólida para o desenvolvimento de aplicações em visão computacional. A compreensão das operações fundamentais — leitura, conversão, filtragem, segmentação e detecção de bordas — é essencial para avançar em aplicações mais complexas.

Este capítulo apresentou os conceitos iniciais e exemplos práticos que servirão como base para os próximos estudos em processamento e interpretação de imagens.

13 Conclusão

O Processamento Digital de Imagens consolidou-se como uma área fundamental da computação moderna, com aplicações que permeiam desde sistemas simples de visualização até soluções complexas baseadas em inteligência artificial. Ao longo deste livro, foram apresentados os principais conceitos, técnicas e ferramentas que permitem a análise, a melhoria e a interpretação automática de imagens digitais.

Inicialmente, foram discutidos os fundamentos das imagens digitais, incluindo sua representação matemática, os modelos de cor e os aspectos físicos relacionados à formação da imagem. Esses conceitos são essenciais para compreender como a informação visual é capturada, armazenada e manipulada em sistemas computacionais.

Na sequência, foram exploradas técnicas clássicas de processamento, como filtragem, realce, transformações geométricas, detecção de bordas e segmentação. Essas operações constituem a base para a maioria das aplicações práticas, permitindo reduzir ruídos, destacar características relevantes e separar regiões de interesse em uma imagem.

O livro também abordou métodos de reconhecimento e interpretação de imagens, destacando o papel das características visuais, da cor, da textura e da forma na identificação de padrões. Nesse contexto, ferramentas como o OpenCV demonstraram-se essenciais para a implementação eficiente de algoritmos, aproximando a teoria da prática.

Com o avanço recente do aprendizado de máquina e das redes neurais profundas, o processamento de imagens passou a integrar sistemas cada vez mais inteligentes e autônomos. Embora este livro tenha enfatizado os fundamentos, os conceitos apresentados servem como base sólida para o estudo de técnicas mais avançadas, como visão computacional baseada em aprendizado profundo.

Por fim, espera-se que este livro tenha fornecido ao leitor não apenas o conhecimento técnico necessário, mas também uma visão crítica sobre o uso e as limitações das técnicas de processamento de imagens. O domínio desses conceitos é indispensável para profissionais e pesquisadores que atuam em áreas como ciência de dados, engenharia, saúde, robótica e sensoriamento remoto.

O Processamento de Imagens continua em constante evolução, impulsionado por novas demandas e avanços tecnológicos. Assim, o aprendizado nessa área deve ser contínuo, combinando fundamentos teóricos sólidos com experimentação prática e atualização constante.

Referências

- Gonzalez, Rafael C., e Richard E. Woods. 2010. *Processamento Digital de Imagens*. 3º ed. São Paulo: Pearson Prentice Hall.
- . 2018. *Digital Image Processing*. 4º ed. New York: Pearson.
- Jain, Anil K. 1989. *Fundamentals of Digital Image Processing*. Englewood Cliffs: Prentice Hall.
- Pratt, William K. 2007. *Digital Image Processing: PIKS Scientific Inside*. 4º ed. Hoboken: Wiley-Interscience.
- Sonka, Milan, Vaclav Hlavac, e Roger Boyle. 2014. *Image Processing, Analysis, and Machine Vision*. 4º ed. Stamford: Cengage Learning.
- Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2º ed. Cham: Springer.